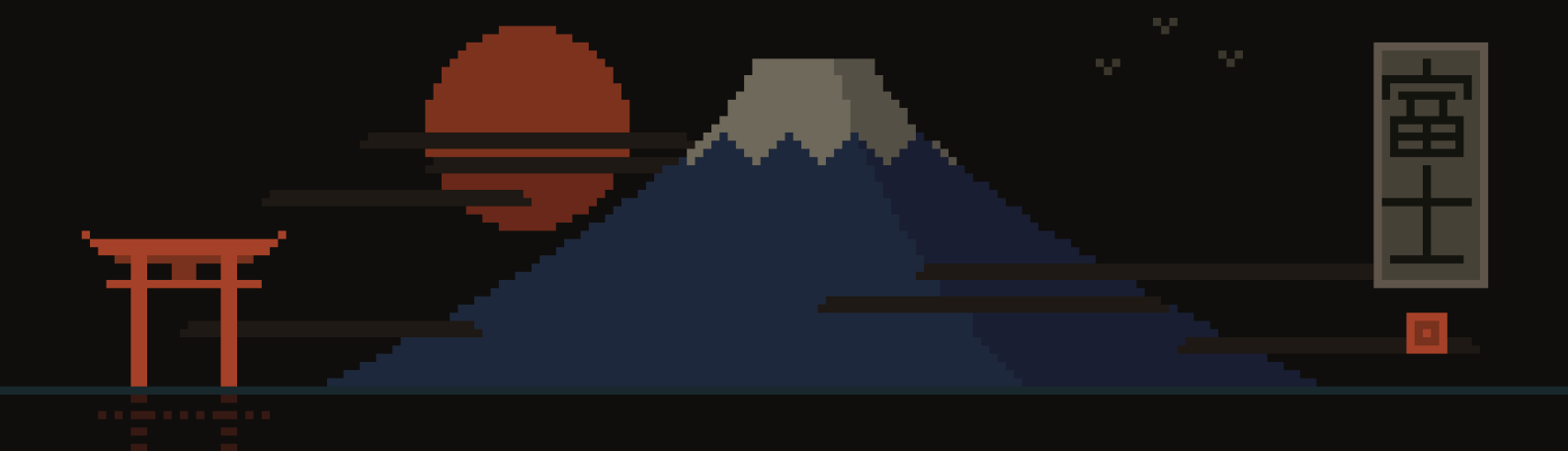
Fedokawai — the handbook

A Fedora Server install turned into a hand-drawn dwl desktop: how to build it, use it, change it, and keep it healthy

October 2026 · Fedora 44 / 45 · dwl 0.9 · wlroots 0.20



Contents

1. What this is
2. The scripts
3. Installing
   - 3.1 Fedora Server installer
   - 3.2 Run it
   - 3.3 `fedokawai.conf`
4. Using the desktop
   - 4.1 The bar
   - 4.2 Menus
   - 4.3 Keybindings
   - 4.4 Themes: dark and light
     - Frames
   - 4.5 Wallpapers
   - 4.6 The terminal and the prompt
   - 4.7 Lock, login, boot
   - 4.8 Copy and paste
   - 4.9 Files, file dialogs and default programs
     - File dialogs
   - 4.10 Screen sharing — picture and sound
   - 4.11 Books — Super+R
     - How it keeps track
   - 4.12 Notes — Super+N
5. How the look is built
   - 5.1 Fonts
   - 5.2 Icons
   - 5.3 Transitions
   - 5.4 Where every app gets its colours
   - 5.5 Pointers
6. Keeping it healthy
   - 6.1 The control — Super+U
   - 6.2 The season's update
   - 6.3 Pinned, and why
   - 6.4 A new Fedora — upgrade
   - 6.5 Built to be left alone
   - 6.6 Kernels, DNF, snapshots
   - 6.7 What verify proves
   - 6.8 Between seasons
     - If an update breaks neovim
7. Keeping it clean
   - 7.1 Where things go
     - The one rule for configuration
     - Your own files
   - 7.2 Snapshots
     - What makes a snapshot
     - Looking and comparing
     - Undoing
     - Deleting
   - 7.3 Keeping packages lean
     - Adding a package properly
     - Taking one out
     - Kernels
     - The rest of what this desktop installs
   - 7.4 Disk space and logs
     - The journal
     - Caches you can clear
     - Btrfs care
   - 7.5 A seasonal routine
8. Software, and why each piece
9. Troubleshooting
10. Where everything lives
    - 10.1 Your files
    - 10.2 Each program's files
    - 10.3 On the system
11. Using the software
    - Who owns which config
    - 11.1 foot — the terminal
      - A notice when a long command ends
      - Safe clipboard
    - 11.2 The shell
      - Fast to start
      - History rules
      - Secrets
    - 11.3 fuzzel — the launcher
    - 11.4 yazi — files
      - Opening
      - Bulk rename
      - Trash
      - Commands on files
    - 11.5 neovim and LazyVim — the editor
      - Modes
      - Never stuck
      - The first hour
      - The grammar: verb, count, motion
      - The dot
      - Text objects
      - Visual mode and block mode
      - Registers and the clipboard
      - Macros
      - Files, buffers and windows
      - Finding
      - The file tree
      - The leader key
      - Search and replace across a project
      - Language servers
      - Completion
      - Formatting
      - Extras and Mason: adding a language
      - Keeping it updated and healthy
      - A daily workflow
      - A learning path
    - 11.6 lazygit — git
      - What git is
      - Once, before the first commit
      - Opening it
      - The screen
      - The colours here
      - A first commit, step by step
      - Staging part of a file
      - Commit, amend, undo
      - Stash
      - Branches
      - Pull and push
      - Resolving a conflict
      - Rewriting history: interactive rebase
      - History and diffs
      - Cherry-pick
      - Recipes

11.7 bat, ripgrep, fd, fzf, eza, zoxide and tldr
11.8 btop — the system monitor
11.9 Brave Origin — the browser
What this desktop sets up
Worth checking once
11.10 mpv and yt-dlp — video
yt-dlp
11.11 imv — pictures
11.12 zathura — PDF, EPUB and comics
11.13 satty — annotating screenshots
11.14 Sound — pavucontrol and EasyEffects
11.15 spotify_player and cava — music
11.16 presenterm — slides in the terminal
11.17 qalc — the calculator
11.18 Documents — Xournal++, AbiWord, Gnumeric and the PDF tools
11.19 Hardware — OpenRGB, Solaar, liquidctl, LACT
11.20 podman — the Prowlarr container
11.21 snapper — snapshots
11.22 Claude Code — an assistant in the terminal
Signing in
11.23 Steam — games
GE-Proton

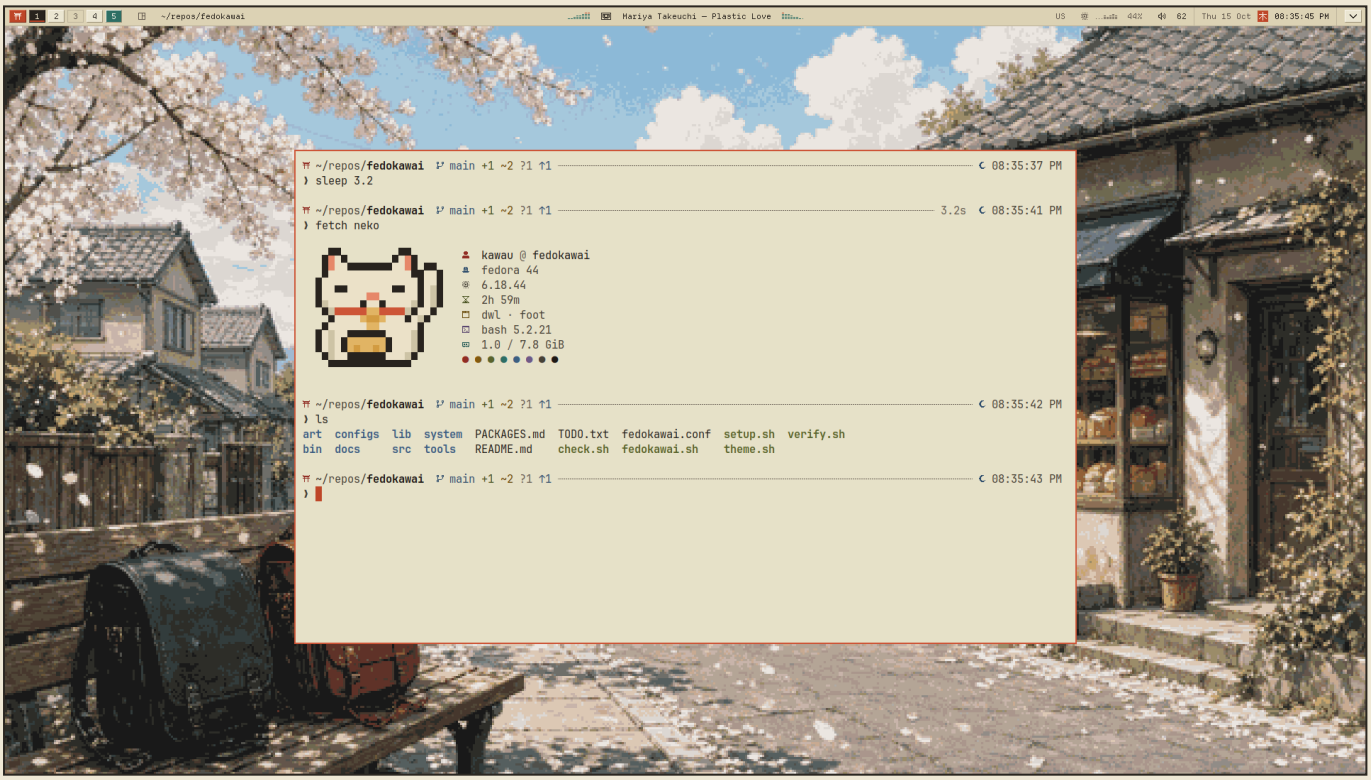
12. Torrents: searching and downloading
12.1 The first time: Prowlarr
12.2 Searching and downloading with torrra
Magnet links in Brave
12.3 How it behaves
12.4 When something is wrong

13. Everyday recipes
13.1 Upload a file from Brave
13.2 Share your screen, with sound, in a call
13.3 Take, annotate and paste a screenshot
13.4 Switch the theme and the wallpaper
13.5 Choose which program opens a type of file
13.6 Install a program and keep `verify` happy
13.7 Update everything
13.8 Move to a new Fedora
13.9 Undo a bad update
13.10 Free disk space
13.11 Plug in a second monitor
13.12 Watch or keep a video from the web
13.13 Change a setting for good

14. Do it yourself: what websites charge for
14.1 Before you start
14.2 PDFs: join, split, pick and turn pages
14.3 PDFs and pictures
14.4 PDFs: smaller, locked, unlocked
14.5 PDFs: text, scans and OCR
14.6 Fill in and sign a PDF
14.7 Pictures
14.8 Video and sound
14.9 Download and record
14.10 Files: archives, checksums, trash, renaming, text
14.11 Office documents and spreadsheets
14.12 Quick conversions: units, money, time



The desktop on the dark theme: the bar on top, a terminal with the prompt, fetch and a listing.



The same desktop on the light theme — switched live with Super+Shift+W.

1. What this is

Fedokawai takes a **minimal Fedora Server** install and turns it into a small, fast Wayland desktop built around **dwl**, a tiling compositor of about three thousand lines. Everything you see on screen was drawn for it: the icons, the Fuji wallpaper scene, the lock screen, the boot menu, the pictures in `fetch`. Nothing comes from a theme pack. The name is Fedora + *kawaii* (かわいい), the Japanese word for cute.

Four ideas run through every file:

Idea	What it means in practice
One tool per job	one editor (neovim), one browser (Brave Origin), one terminal (foot), one launcher (fuzzel, also every menu). `verify.sh` fails if a second one appears
Nothing hidden	every package is listed in `lib/manifest.sh`, every release binary in `lib/upstream.sh`, every colour in `tools/build-themes.py`, every icon in `art/icons.txt`
Proof, not hope	`verify.sh` checks the machine against the lists; `theme.sh --check` checks every config against the repository
Pixel-exact	the UI fonts and icons sit on an 11-pixel grid and are drawn at whole multiples of it only (11, 22, 33 — the boot menu 44 on a 5K screen); pixel art is enlarged by whole numbers only, so every pixel stays a sharp square

A first tour. Super+Space starts programs; Super+Return opens a terminal; Super+B is the browser and Super+E the files. The ⌄ key at the right end of the bar (or Super+X) holds quick settings — the theme switch is the first line. Super+/ shows every key. Super+V brings back anything you copied. Everything else in this handbook is detail.

2. The scripts

File	Owns	Touches
`fedokawai.sh`	the menu for the install	runs the others; nothing of its own
`check.sh`	**readiness**	nothing — is this Fedora, online, on Btrfs, with room to work?
`setup.sh`	**software**	repositories, packages, drivers, kernel arguments, the dwl binary and its session launcher, services, the performance profile, snapshots — and what software needs before it can run: the LazyVim starter, the Prowlarr container file and key, torrra's settings and its key wrapper
`theme.sh`	**look and configuration**	every file it ships under `~/.config` and `~/.local` (`bin/kawa-*`, `share/fedokawai`, `share/applications`, `share/themes`, fonts), `~/.bashrc.d`, `~/.inputrc`, the pixel art, `/etc/dconf` defaults, the boot menu theme and `/etc/default/grub`, the console font and colours, dwl rebuilt from `src/dwl/config.h`, two C helpers built from `src/`, Brave's colours (set once in its profile) and its launcher entry (so a screen share can carry sound)
`verify.sh`	**proof**	nothing of yours — checks against the manifest and the repository; it only wakes Prowlarr to ask it, and writes its result to `/var/lib/fedokawai/last-verify`
`system/fedokawai`	**upkeep** — the system's own control, installed as the `fedokawai` command (Super+U)	nothing itself: it reads the records and runs the scripts above through `sudo` (§6.1)

`theme.sh` never installs a package, and `setup.sh` writes no desktop look of its own. The menu offers them separately — **Setup** alone gives you dwl with upstream defaults, **Theme** can be run (and re-run) whenever you like, and **Both** runs them in order. The only file you edit is `fedokawai.conf`.

The toolkit. At its first reboot, setup installs the project folder for good as `/usr/local/lib/fedokawai`, owned by root; from then on everything runs from there, and the folder you installed from can go. Where this book says *the repository* after the install, it means that copy: to change one of its files, `sudoedit /usr/local/lib/fedokawai/FILE`, then `fedokawai theme` (a config) or `fedokawai update` (a package).

3. Installing

3.1 Fedora Server installer

Screen	Set	Why
Software Selection	**Minimal Install** (or *Fedora Custom Operating System*), nothing extra	the scripts add exactly what is needed, so `verify.sh` can prove nothing else crept in
Installation Destination	**Custom** → **Btrfs** → **create automatically**	Server's *Automatic* is LVM + XFS; snapshots need Btrfs. Result: `/boot/efi`, `/boot`, subvolumes `root` and `home`
Network & Host Name	ethernet on, hostname of your choice	NetworkManager is part of `@core`
Time & Date	your zone, Network Time on	
KDUMP	off	a desktop; saves memory
Root / user	root **disabled** (no password); your user **administrator** — setup, Theme and update all run through `sudo`	if the desktop ever fails to start, log in on a text console (Ctrl+Alt+F3) as your user: `fedokawai` works there; undo a bad update with snapper (§7.2)

3.2 Run it

The project is yours alone and lives nowhere online: it comes to the new machine on a USB stick, as `fedokawai.tar.gz` (a tar archive keeps every file's permissions, and the minimal install has `tar`).

```
lsblk -o NAME,SIZE,LABEL,FSTYPE          # find the stick, say sdb1
sudo mount /dev/sdb1 /mnt
tar -xzf /mnt/fedokawai.tar.gz -C ~      # → ~/fedokawai
sudo umount /mnt
cd ~/fedokawai
sudo bash fedokawai.sh           # the menu: Check · Setup · Theme · Both · Verify · Update · Drift · Status · Repair
```

Choose **Both** for a first install. Setup walks through three phases and reboots twice between them; after each reboot it takes tty1 by itself (`fedokawai-resume.service`) and carries on, so you can walk away. It stops with the failing line and the log (`/var/log/fedokawai.log`) if anything goes wrong; fix it and run it again — finished steps are recorded in `/var/lib/fedokawai` and skipped. When it is done the login screen comes up, and from then on the system looks after itself through one menu: **Super+U** (§6.1).

Phase	Steps	Why this order
1	readiness check → record what is installed → DNF tuning → boot-menu and two-kernel policy → full update → base tools → the performance profile → remove nano, vim, rescue kernel → SSH policy → **reboot**	the record lets `verify.sh` spot anything extra later; tuning first so the big update downloads ten at a time; update before any third-party repository so RPM Fusion matches the base libraries; SSH closed before the first reboot, so a run that stops early never leaves it open
2	RPM Fusion, Brave, Claude Code (key fingerprint checked), COPRs → GPU drivers → full ffmpeg and media tools → **reboot**	drivers before the programs that use them, so nothing pulls the wrong variant; the reboot activates the GPU kernel argument
3	build tools → dwl (pinned commit + patch) → greetd login → desktop → fonts and toolkit themes → apps → release binaries → the yazi file dialog → Prowlarr + torrra → LazyVim → Steam → groups → theme (if chosen) → **verify gate** → enable the login, snapshots, baseline	the login is enabled last, only after verification, so a failure never leaves you at a black screen

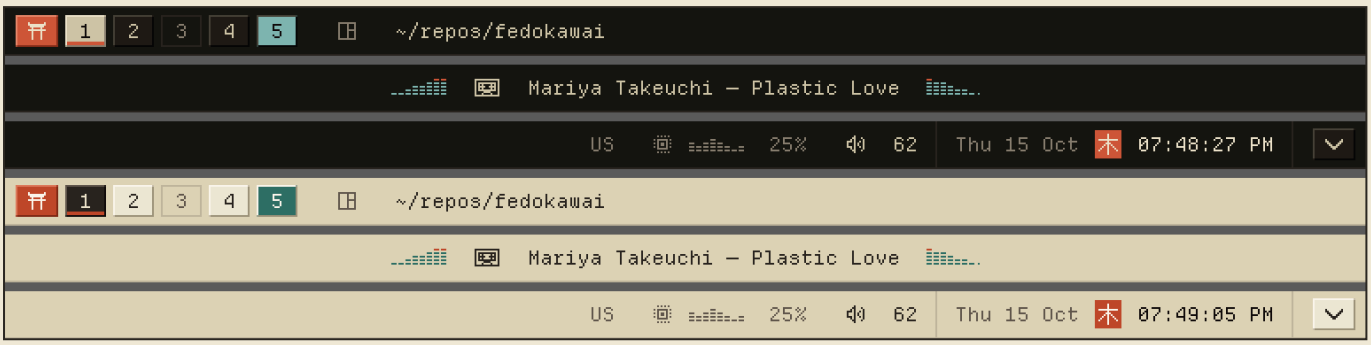
3.3 `fedokawai.conf`

Every value is validated before anything runs; a wrong value stops the script with a message that names it.

Setting	Default	Meaning
`USERNAME`	empty	the desktop user; empty = whoever ran `sudo`
`GPU`	`auto`	`auto`, `amd` or `intel` — drivers, video decoding, and mpv's scaling
`STEAM`	`yes`	Steam, gamescope, gamemode, MangoHud, GE-Proton; switched to `no`, the next update hands those packages back to dnf (`sudo dnf autoremove` removes them; your games stay in your home)
`SSH_SERVER`	`no`	`no` disables sshd and closes ssh/cockpit in the firewall
`MONITOR_HZ`	`144`	the highest refresh rate at or below this is used
`KB_LAYOUTS`	`us,ara`	keyboard layouts; the first one is always used for shortcuts
`KB_TOGGLE`	`grp:alt_shift_toggle`	Alt+Shift switches layout; the bar shows US / AR
`TIMEZONE`	`Asia/Dubai`	
`WEATHER_CITY`	`Sharjah`	the weather line in quick settings (wttr.in, no key)
`NIGHT_LIGHT`	`25.35:55.40`	latitude:longitude for gammastep
`THEME`	`dark`	the theme a new install starts in; afterwards it is whatever you last switched to
`WALLPAPER_PACK`	`yes`	fetch the wallpaper pack once (70 pictures, about 210 MB to download; kept redrawn as pixel art, about 4 MB)
`BOOT_MENU_SECONDS`	`2`	how long the themed boot menu shows; `0` = Fedora's hidden menu
`DNF_PARALLEL`	`10`	parallel downloads (1–20)
`DWL_TAG`, `DWL_COMMIT`, `DWL_WLROOTS`	`v0.9`, pinned, `0.20`	dwl is built from exactly this commit
`TFC_TAG`, `TFC_COMMIT`	`v1.4.3`, pinned	the yazi file dialog's portal is built from exactly this commit

4. Using the desktop

4.1 The bar



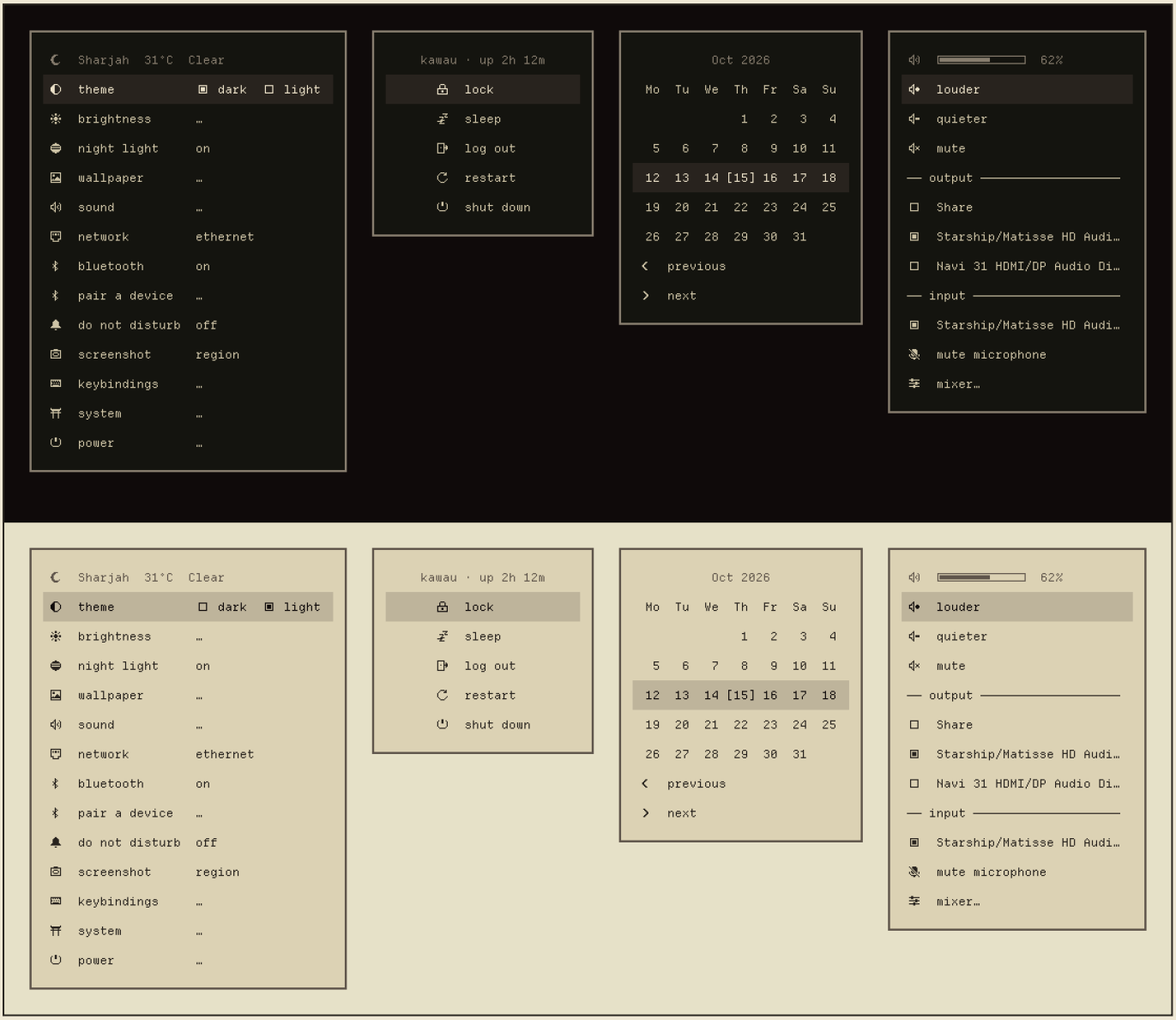
The bar, dark and light, at twice its size: the left end, the middle, the right end.

Part	What it shows	Click
torii key	the vermilion key at the far left	opens the launcher
workspaces	1, 2, 3 always; 4–9 only while they hold a window, are the one you are on or want attention. A raised key = windows on it; the lit key with a vermilion stamp = the one you are on; a teal key = something there wants attention	goes to it
layout icon	tiled, floating, or monocle (with the number of stacked windows)	
title	the focused window	
now playing	artist — title, with a cassette whose reels turn while sound plays, between the two halves of a spectrum meter drawn like an old hi-fi's fluorescent display: lowest notes nearest the title, a full column lights its top segment in vermilion	play/pause · right: next · middle: previous
mode seal	a small vermilion seal while something is on that you should not forget: **静** (*quiet*) while do-not-disturb holds notices back, **共有** (*sharing*) while the screen is being shared; nothing otherwise	quick settings
US / AR	the keyboard layout (vermilion while it is not the first)	
CPU	the chip icon, the last eight readings (one every two seconds, the newest on the right) in the same segment style, then the load now in per cent; vermilion above 80 %	opens btop
volume	speaker icon and level; vermilion when muted	the audio menu · right: mute · scroll: ±2 %
date, day, time	`Wed 14 Oct` 水 `08:52:07 PM`: the date, the day of the week as a kanji seal (月 火 水 木 金 土 日, Monday to Sunday), the time to the second	the calendar
⌄ key	quick settings	

Every player that speaks MPRIS shows in *now playing* and answers the media keys: spotify_player, Brave (a video or a tab playing sound), and mpv (through the mpv-mpris plugin).

The bar costs almost nothing at rest: the workspaces, layout, title and keyboard layout update only when dwl reports a change, and only the part that changed is redrawn (no polling); the clock is waybar's own and wakes once a second; the CPU module is pure bash; and the music meter's analyser (cava) runs only while a player is playing — paused, it is not running at all; silent, it sleeps.

4.2 Menus



Quick settings, the power menu, the calendar and the audio menu, on both themes.

Every menu is the same fuzzel window, so they look and behave alike: type to filter, arrows or mouse to choose, **Esc** or a click outside to close.

Menu	Open with	Inside
quick settings	⌵ key, **Super+X**	**theme** first, as a two-way switch (■ dark □ light) · brightness · night light · wallpaper · sound · network · Bluetooth on/off and pair a device (only when the PC has a Bluetooth adapter) · do not disturb · screenshot · keybindings · **system** (the control, §6.1) · power; the weather on top
power	**Super+Shift+E**	lock · sleep · log out · restart · shut down — centred, with your uptime
audio	volume, **Super+A**	the level as a framed bar at the top · louder / quieter / mute (stay open for repeated taps), every output and input — picking one moves playing sound to it — microphone mute, the full mixer
calendar	the clock, **Super+C**	this month (Monday first, today in brackets), previous / next
brightness	quick settings	the monitor's real backlight over DDC/CI, like Twinkle Tray on Windows, each level drawn as a small framed bar; **Super+]** / **Super+[** step by 10 %
share what?	a screen share	the screens and windows you can share (§4.10)
help	**Super+/**	every keybinding, searchable

The windows that show more than a list are small terminal windows of their own rather than menus, all in one look — the name in a rule across the top, a search line where there is something to search, the list, and a pane beside or below it: the wallpaper picker (**Super+W**, the wallpaper itself shown as you move — §4.5), the clipboard history (**Super+V** — §4.8), the books (**Super+R** — §4.11) and the system's control (**Super+U** — §6.1). Each opens once: a second press while it is open does nothing.

The volume, microphone and brightness keys show a **level display** for a moment at the bottom centre — also over fullscreen windows and while *do not disturb* is on. It is drawn like the volume scale of an old hi-fi: a wedge of twenty pixel columns rising left to right, lit in asagi with the topmost lit one vermilion (the peak lamp), the number beside it at twice the pixel size; it reads *off* when muted, and holding a key updates it in place.



The level display, a notice from an app and a critical notice, dark and light.

4.3 Keybindings

Super is the Windows key. Shortcuts always read the US layout, so they work the same while Arabic is active.

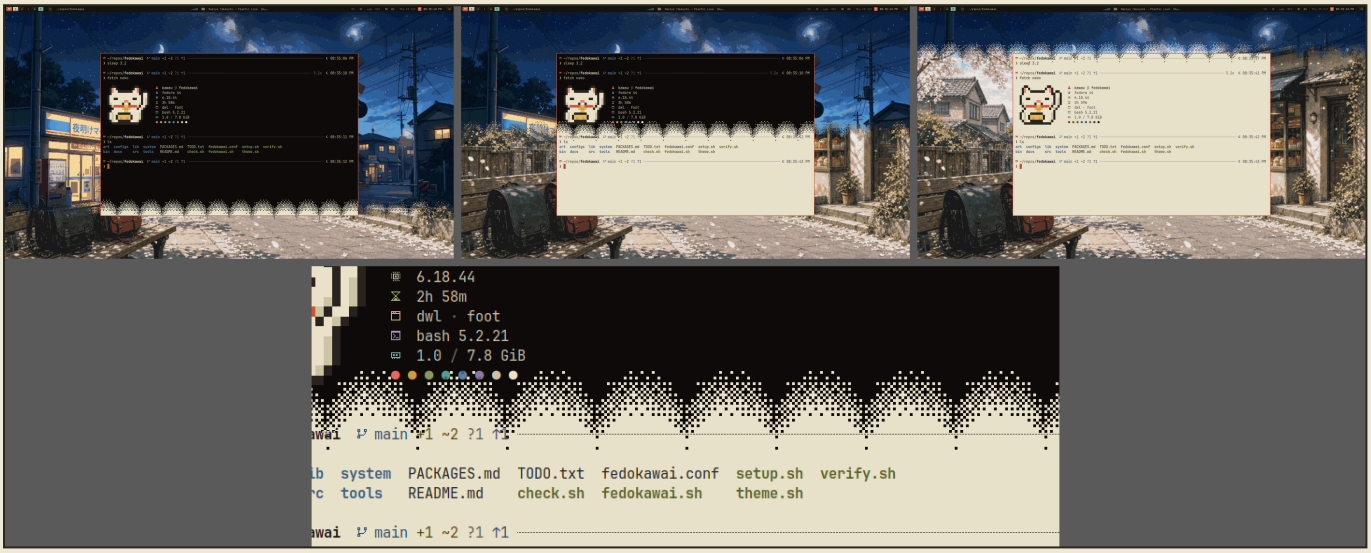
Keys	Does
Super+Return / Space / B / E	terminal · launcher · Brave Origin · files (yazi)
Super+X / A / C / W / V / /	quick settings · audio · calendar · wallpaper · clipboard history · help
Super+Shift+W	**dark ⇄ light theme**
Super+R · Super+N	books (§4.11) · notes, and again to put them away (§4.12)
Super+U	the system: update, a new Fedora, repair (§6.1)
Super+Shift+N	dismiss notifications
Super+Escape · Super+Shift+E	lock · power menu
Print · Shift+Print · Super+Print	screenshot a region · the whole screen · a region, then annotate it (Enter copies)
Super+Q	close the window
Super+J / K · H / L · I / D	focus next/previous · shrink/grow the main area · more/fewer windows in it
Super+Shift+Return · F · Shift+Space	move to the main area · fullscreen · float/unfloat (also Super+middle-click)
Super+drag / right-drag	move / resize
Super+T / Y / M · Shift+T	tiled · floating · monocle · previous layout
Super+1…9 · Shift+1…9 · Ctrl+1…9 · Ctrl+Shift+1…9	go to · send to · also show a workspace · also show the window there
Super+Tab · Super+0 · Shift+0	previous workspace · all of them · the window on every one
Super+, / . · Shift+, / .	focus · send to the monitor on the left / right
media keys	volume, mute, microphone mute — the level display shows for a moment, also in fullscreen · play/pause, next, previous
Super+scroll	volume ±2 % (as scrolling over the volume in the bar)
Super+] / [· brightness keys	monitor brightness (the same level display)
Alt+Shift	keyboard layout
Right Shift+F12	the game overlay (MangoHud) on / off, in games started with it

Keys	Does
Super+Ctrl+Shift+Q · Ctrl+Alt+F2…F6	log out · text consoles (Ctrl+Alt+F1 comes back)

Windows carry no title-bar buttons anywhere — not in GTK apps, not in Brave: dwl tiles them, Super+Q closes them.

4.4 Themes: dark and light

Press **Super+Shift+W** (or quick settings → *theme*). The screen holds still for a moment while the terminals and everything in them, the bar, menus, notifications, GTK and Qt apps, Brave, mpv, btop, zathura and the wallpaper all change underneath; then the new theme arrives as a row of seigaiha wave crests with a dithered fringe — rising from the bottom like dawn for the light theme, falling from the top like nightfall for the dark one. The lock screen and the next menus follow from then on.



A switch to the light theme, three moments of it: the crests rise from the bottom edge.

	dark — *sumi* (ink)	light — *washi* (paper)
ground	`#0f0e0c` ink	`#e9e1cb` paper
text / strongest	`#cfc3a6` / `#ece3cc`	`#2b2721` / `#0f0e0c`
quiet text	`#877f6f`	`#5f594d`
bar / raised key / hairline	`#161411` / `#1f1c18` / `#2b2721`	`#dcd2b8` / `#efe8d5` / `#c2b79b`
vermilion (accent)	`#d0583a`	`#c0462a`
beni · matcha · yamabuki	`#d64b4b` · `#8e9a5b` · `#d29f45`	`#a8342a` · `#5c682d` · `#845b0f`
ai · fuji · asagi	`#5f7fa0` · `#9079a9` · `#7fb8b0`	`#3f6185` · `#6f5a8a` · `#2f6e67`

The accents keep their character on both grounds; on paper they are deepened so they keep their contrast, and on paper the terminal's "bright" colours are the deeper ones. Every colour used for reading — quiet grey and vermilion included — reaches 4.5 : 1 or better against the surface it sits on (the WCAG AA level for body text). Decoration that is meant to recede — an empty workspace key, a hairline, the terminal's deliberately faint grey for comments — sits lower on purpose. Terminal programs see the usual 16 colours, arranged the way gruvbox does it, so a program that uses "black" for faint lines and "white" for text reads correctly on paper too.

Frames

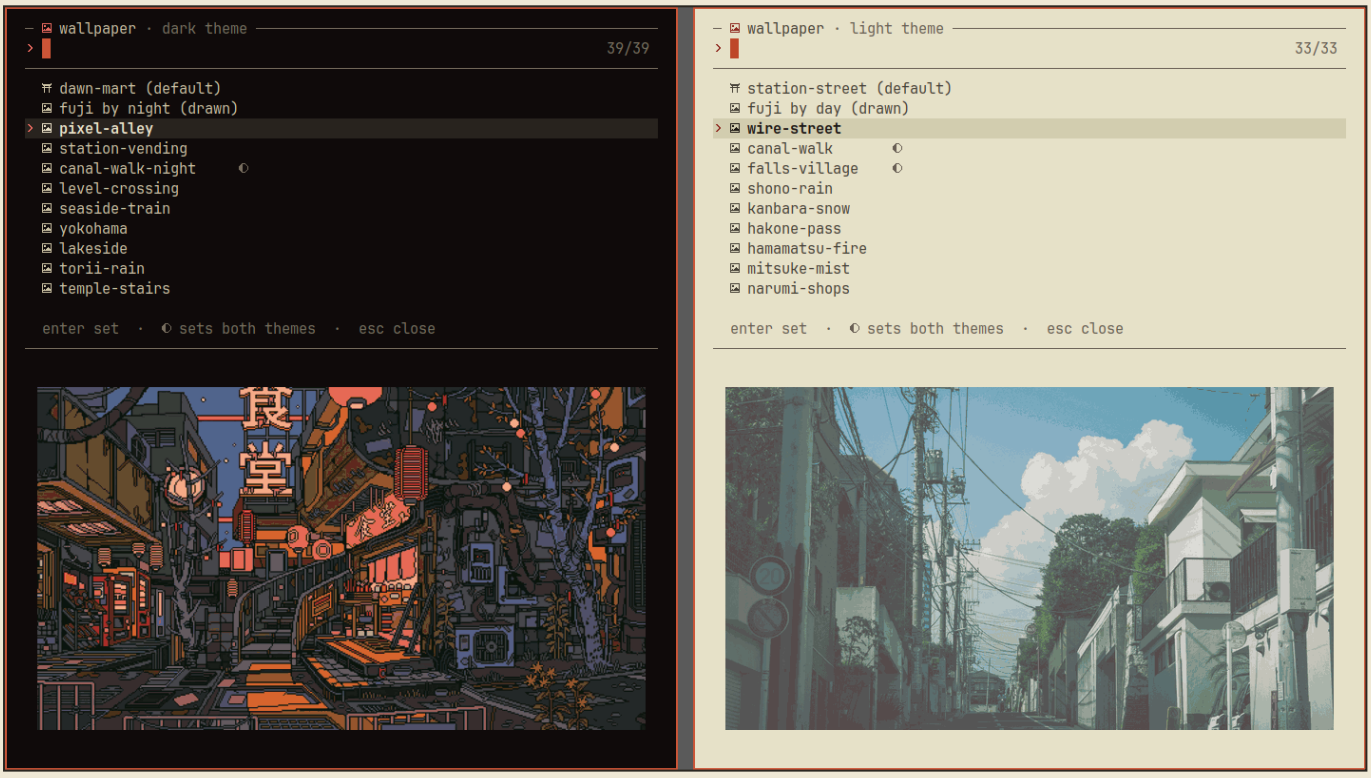
Everything that floats has the same plain, full border, 2 pixels wide. Windows get theirs from dwl (compiled in, so it suits both themes): vermilion around the focused window, warm grey around the others, asagi around one that asks for attention. Menus, notices and the level display are solid cards (nothing of the wallpaper shows through) with a warm-grey border of their own theme; a critical notice's border is vermilion.

4.5 Wallpapers



The two default wallpapers, redrawn as pixel art: dawn-mart on the dark theme, station-street on the light.

Super+W (or *wallpaper* in quick settings) opens the picker: a small window with the current theme's wallpapers listed on top and, underneath, the one under the cursor — the picture itself, as your screen will show it, scaled down by a whole number so the pixel art stays crisp. Move with ↑/↓ or the mouse, type to search; **Enter** sets it, **Esc** leaves everything as it was. A ◖ after a name means the place also comes in the other theme's light (see the pairs below), and choosing it sets both.



The wallpaper picker (Super+W) on both themes: the list on top, the picture under the cursor as the screen will show it.

Each theme remembers its own wallpaper. The choices:

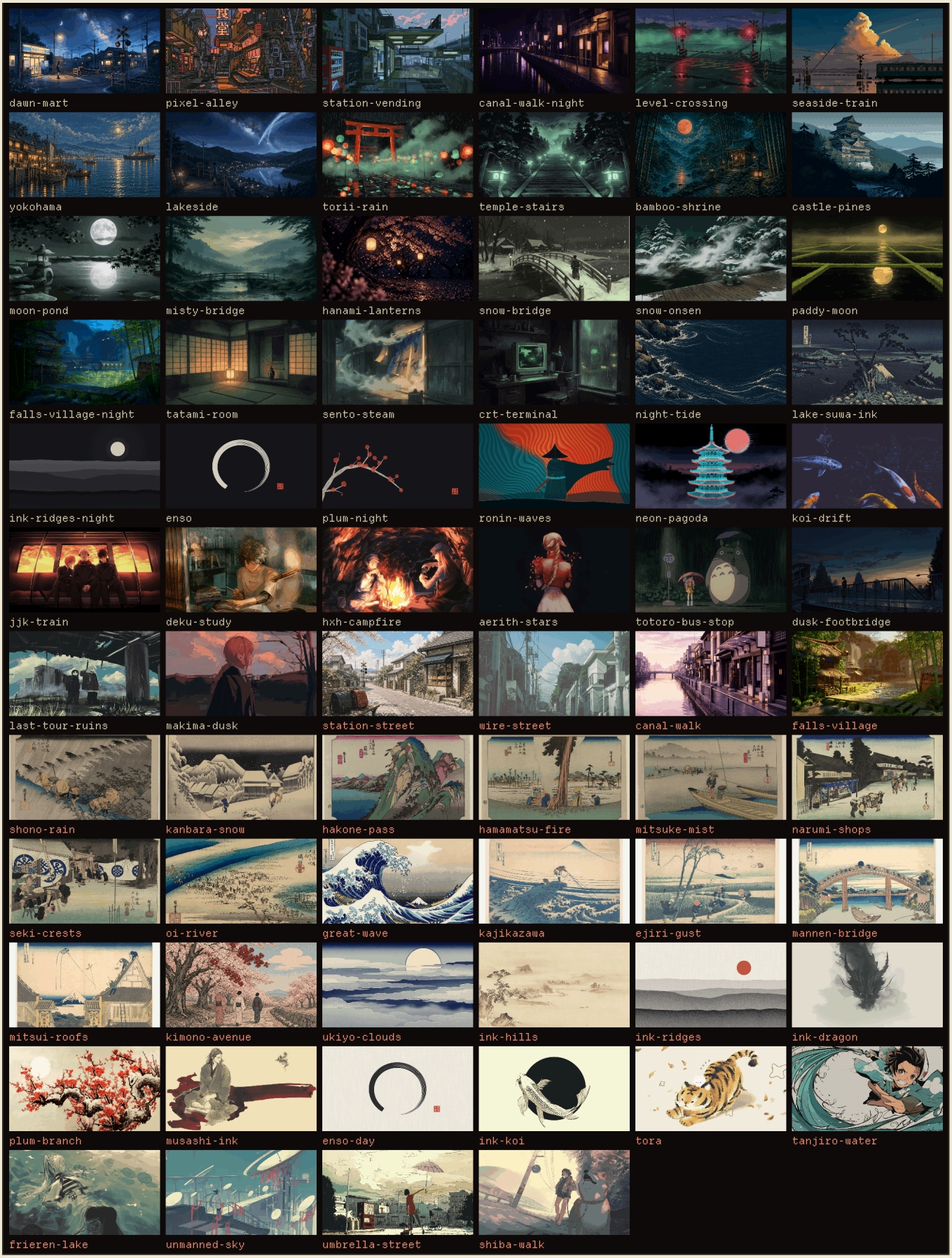
- **default** — a retro railway town from the pack: **dawn-mart** on the dark theme (a konbini glowing by a level crossing under the stars) and **station-street** on the light one (the same kind of crossing on a shop street under cherry blossom). Without the pack, the default is *fuji*.
- **fuji** — drawn here: the seigaiha (blue-sea wave) pattern tiled 1:1, with Mt Fuji, a torii in the water, birds and a woodblock-print cartouche along the bottom; by night on the dark theme, by day on the light one.
- **the pack** (`WALLPAPER_PACK="yes"`) — 70 pictures (38 for the dark theme, 32 for the light) fetched once from GitHub at pinned commits and checked by SHA-256, then redrawn as 640×360 pixel art with 32 colours. Every one is Japanese and retro, and sits in the themes' own colours — ink and paper, vermilion, teal and indigo (each was measured against the two palettes; nothing neon, nothing off-key) — and no two look alike. The picker lists them grouped by kind:

- *dark*: towns and streets by night (the konbini at the crossing, a lantern-lit pixel alley, a station's vending machines, a two-car train by the sea, Yokohama's Meiji harbour); shrines, gardens, rain and snow (a torii in the rain, a castle in the pines, a stone lantern by a moonlit pond, an onsen in snow); rooms and baths (a tatami room, a sentō, a CRT by a rainy window); ink and woodblock (Hokusai's waves and Lake Suwa, an ensō, a plum branch at night, a ronin); and anime and film (Jujutsu Kaisen, My Hero Academia, Hunter × Hunter, Final Fantasy VII, Totoro, Your Name, Girls' Last Tour, Chainsaw Man).
- *light*: streets and villages by day; Hiroshige's Tōkaidō road (driving rain at Shōno, night snow at Kanbara, the Hakone pass, the mist at Mitsuke, the crests at Seki, the crowd fording the Ōi); Hokusai (the Great Wave, the fisherman at Kajikazawa, the gust at Ejiri, Mannen bridge, the roofers at Mitsui); ukiyo style, ink and brush (kimono walkers under cherry trees, an ink dragon, a plum branch, an ensō, koi, a tiger, Musashi); and anime (Demon Slayer, Frieren and three illustrations).

Five come in pairs — the same picture by day and by night (a canal walk, a waterfall village, ink ridges, the ensō, the plum branch) — and choosing one sets the other theme's too. Sources and licences, one line each: `~/.local/share/fedokawai/wallpapers.list`. Nothing is stored in the repository; the files are fetched for your own use. A picture dropped from a later list is deleted from your pack the next time it is fetched. * **your own** — anything in `~/Pictures/Wallpapers`, shown as it is on either theme.

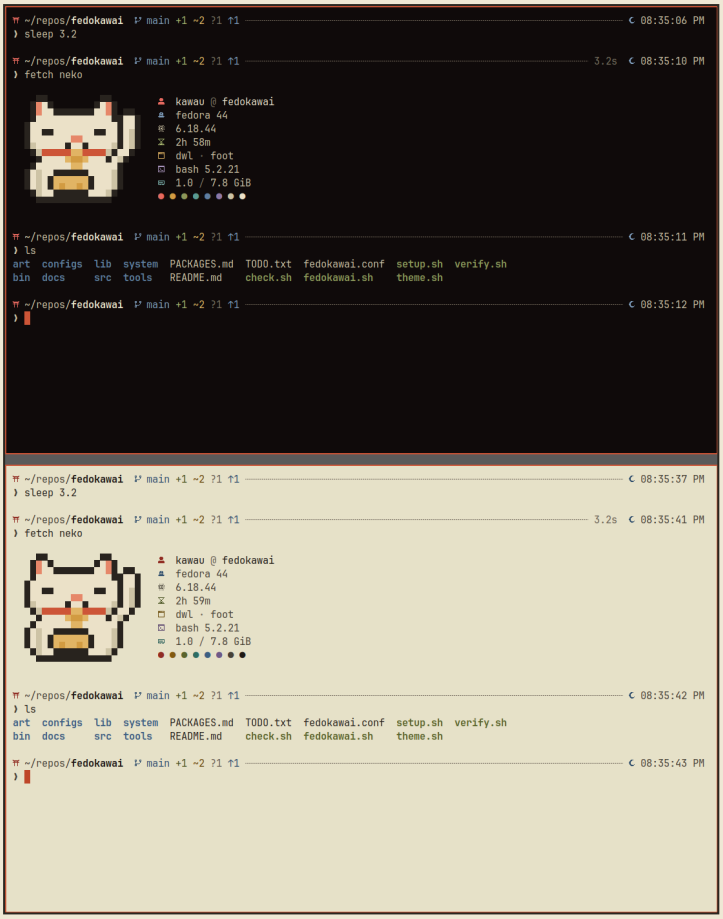
Pixel art is enlarged by a whole number to cover each monitor (1080p ×3, 1440p ×4, 4K ×6), so it never blurs; the results — and the picker's previews — are cached in `~/.cache/kawa-wallpaper`. A new choice dissolves in over the old one in an ordered dither whose grain is exactly one pixel of that art.

Tip: `~/.cache/kawa-wallpaper` can be deleted at any time; `kawa-wallpaper --restore` draws the current wallpaper again.



The whole pack: the dark theme's pictures first, then the light theme's.

4.6 The terminal and the prompt



A real terminal: the prompt's ledger line per command, a command that took over three seconds, fetch, and ls in a git repository with changes.

```
⛩ ~/repos/site   main +2 ~1 ····· 4.2s   × 2   ☾ 08:52:07 PM
❱
```

Part	Shows
torii	where every entry starts
path	the folders above in the text colour, the one you are in bright and bold (the last three; `…/` for the rest); `user@host` first over SSH
git	branch, then only what is non-zero: `+` staged, `~` modified, `?` untracked, `!` conflicted, `↑` ahead, `↓` behind — one `git status` call, parsed in bash
rule	a dotted line to the right edge, so each command reads as a new entry
right end	time taken (3 s or more) · the exit code in a vermilion seal when the command failed · sun by day, moon by night · the time to the second
`❱`	the text colour; vermilion after a failure

(The terminal's "vermilion" and "paper" are its ANSI red and white, so they follow the theme like everything else in it.) Nothing on the header line has a background, so the cursor below never touches it. Outside a git repository the prompt runs entirely inside bash — not one process is started to draw it.

Feature	How
new terminal in the current folder (Ctrl+Shift+N)	the prompt reports the folder (OSC 7)

Feature	How
jump between prompts (Ctrl+Shift+Z / X)	the prompt marks itself (OSC 133)
last command's output in neovim (Ctrl+Shift+G)	OSC 133 marks around the output + foot's `pipe-command-output`
history	50 000 entries, duplicates erased, each command saved as it runs, so every new window has it; Up/Down search by what you typed
instant terminals	`foot --server` at login; Super+Return opens a client

`fetch` prints one of ten pictures drawn for this desktop — a beckoning cat, a daruma, a torii at sunset over the sea, a beige computer showing Mt Fuji, Fuji at dawn, an onigiri, a paper lantern, a cassette, a fox mask and a bowl of ramen — beside the system facts. `fetch NAME` picks one (`neko`, `daruma`, `torii`, `pc`, `fuji`, `onigiri`, `chochin`, `cassette`, `kitsune`, `ramen`). It is pure bash and prints in a few milliseconds.

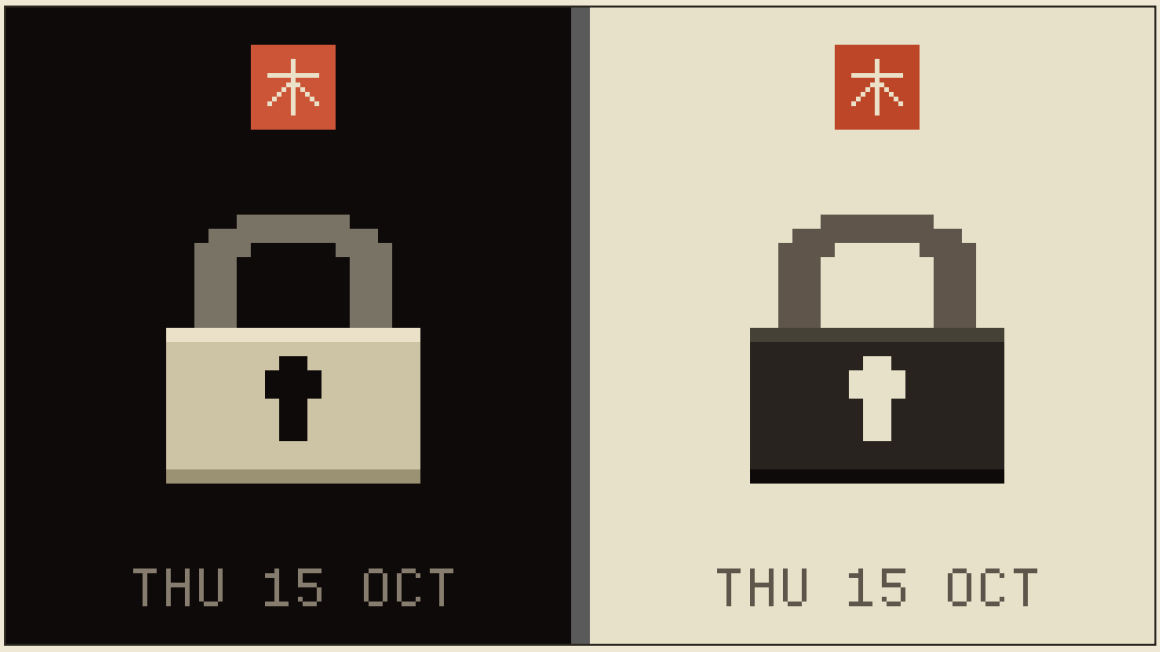


The ten fetch pictures, on ink and on paper: two pixels per character cell, drawn with half blocks.



The terminal's sixteen colours on both themes, with bold, dim, italic, underline, reverse and strike.

4.7 Lock, login, boot



The middle of the lock screen on both themes, at twice its size: the day's kanji seal, the pixel padlock (the typing indicator appears in its keyhole) and the date.

The **lock screen** is the theme's plain ground with a pixel padlock in the exact centre — paper on ink, or ink on paper. Above it the day of the week sits in a small vermilion seal, in kanji (月 Monday, 火 Tuesday, 水 Wednesday, 木 Thursday, 金 Friday, 土 Saturday, 日 Sunday), and below it the date in the bar's pixel letters; both are drawn into the picture as it locks, in a few tenths of a second. Before sleep it locks with the bare padlock at once, so the lid can close. swaylock's indicator is shrunk to a 15-pixel ring that sits inside the padlock's keyhole, clear of its face: it is invisible until you type, then the keyhole glows — vermilion while typing, teal while checking, red when wrong. It locks after 10 minutes idle, before sleep, and with Super+Escape; at 15 minutes the screens power off (windows stay where they were) and wake on the first key. The password goes in in the keyboard layout that was active (the bar showed it before the lock); if the keyhole turns red although the password is right, press Alt+Shift and type it again.

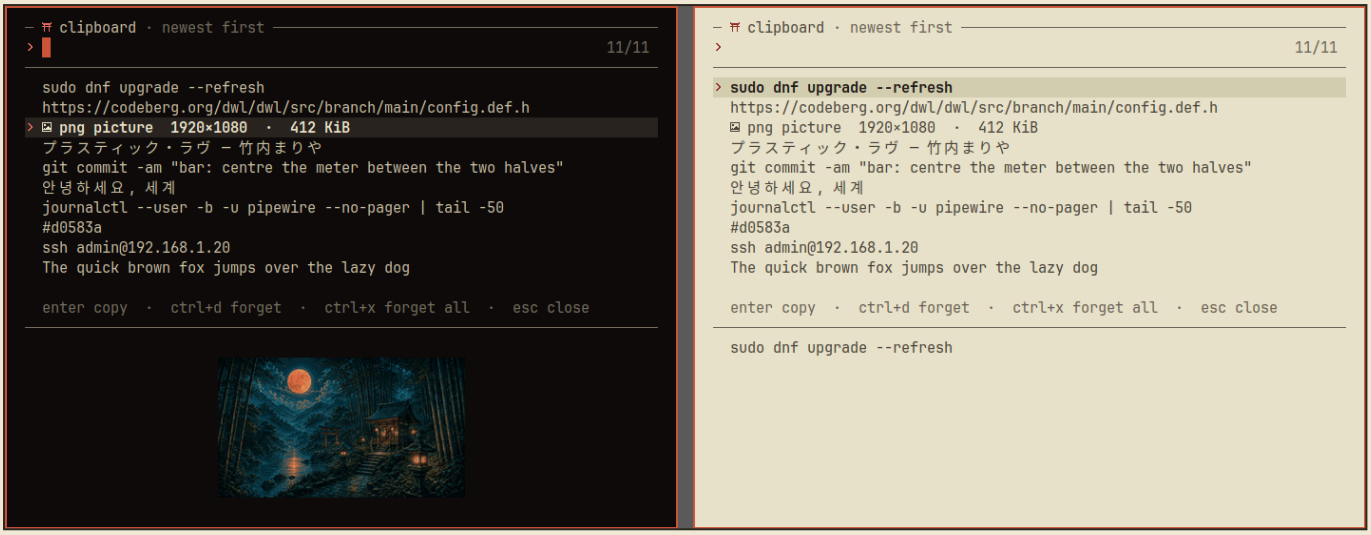
The **login** is greetd's plain text prompt (agreety) on the console: user name, password, nothing else. That console (tty1) always uses the dark theme's 16 colours (set as kernel parameters) and Terminus bold — 12×24 below 1440p, 16×32 above. On Fedora 44 the spare consoles (Ctrl+Alt+F2…F6) look the same; Fedora 45 runs kmscon there, which keeps its own colours and font.



The boot menu at 1080p: sumi ink, a hem of seigaiha waves fading upwards in a dither, Departure Mono at twice its grid.

The **boot menu** is a GRUB theme in the same language, at your monitor's native resolution: plain sumi ink with a hem of seigaiha waves along the bottom edge, fading upwards in an ordered dither; the torii above a quiet box of entries, the chosen one marked by a vermilion bar, a thin vermilion line counting down. Its text is Departure Mono at two to four times its grid (22, 33 or 44 px, by the screen's height). It shows for `BOOT_MENU_SECONDS`; with `0` Fedora's hidden menu returns (it then appears only after a failed boot, or with Esc held). Re-run **Theme** after changing monitors so it matches the new resolution.

4.8 Copy and paste



The clipboard history (Super+V) on both themes: newest first, the full text or the picture itself underneath.

Copying works the same way in every program: Ctrl+C / Ctrl+V (Ctrl+Shift+C / V in the terminal), and selecting text in the terminal copies it as well. Two things make it dependable:

- **A copy outlives its program.** On Wayland a copy is only a promise — the program that copied hands the data over when something pastes — so normally, closing it empties the clipboard. Here `kawa-clipkeep` (C, in `src/kawa-clipkeep/`, built by **Theme**) reads every format of each copy the moment it is made, and when the clipboard is then emptied because its owner closed, it offers the same copy again, in all its formats. While the program that copied is open nothing changes: it keeps serving its own copy. Copies a password manager marks as secret are never kept, nor copies over 64 MiB, nor the primary selection (select + middle-click). One limit, from Wayland itself: clearing the clipboard on purpose while the program that copied is still open looks exactly like that program closing, so the copy comes back once — clear it again, or close that program first.
- **Between Wayland and X11 programs** (Steam, older games) the compositor passes copies both ways by itself.

Super+V opens the clipboard history in a small floating window: the last copies, newest first, pictures included. Type to search; ↑/↓ or the mouse move; the full text — or the picture itself — shows underneath.

Keys	Does
Enter	put that entry back on the clipboard and close; then paste as usual
Ctrl+D	forget that entry
Ctrl+X	forget them all (it asks first; the clipboard itself stays)
Esc	close, the clipboard exactly as it was

The history is cliphist's (`~/.cache/cliphist`), fed by `wl-paste --watch` from the moment you log in; it keeps the last 750 copies. Its window uses the terminal's own colours, so it follows dark and light like foot.

Tip: copied a password by hand from somewhere that does not mark it secret? Open Super+V and press Ctrl+D on it, so it does not stay in the history.

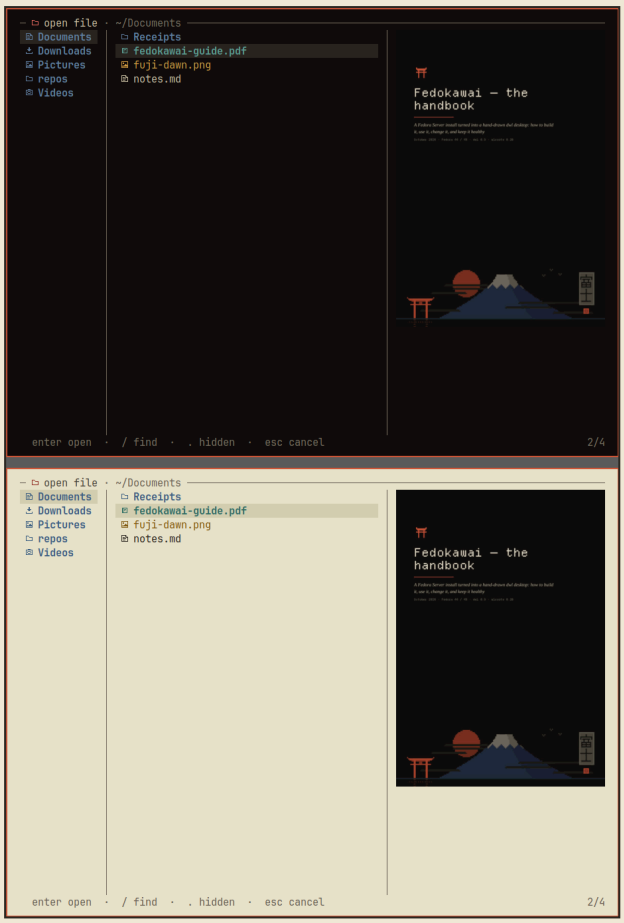
4.9 Files, file dialogs and default programs

Files — Super+E opens yazi in a window of its own; folders opened from anywhere (a download's *Show in folder*, `xdg-open ~/Pictures`) open the same way (§11.4).

File dialogs

Every program's *Open*, *Upload* and *Save as* — Brave's uploads and downloads, every GTK program's dialogs — opens **yazi** (§11.4) in a small floating terminal: the same file manager, keys and previews you already use, instead of a separate dialog with its own ways. It is dressed like the other popups — what it is for in a rule along the top (*open file*, *open files*, *open folder*, *save file*, with the

folder you are in), the keys that matter along the bottom. It starts in the folder the program suggests (Brave: where you last saved or uploaded), else your home.



The file dialog an upload opens: yazi in a floating window, the PDF under the cursor previewed.

In the dialog	Open a file	Open a folder	Save
move · into a folder · back	↑ ↓ (or j k) · → or Enter (l) · ← (h)	the same	the same
choose	**Enter** on a file	**Enter** on a folder, or **q** in the one you are in	**Enter** on the file the program suggested (it is already there, under its name; **r** renames it first)
jump	**z** a few letters of a name below here · **Z** a folder you use often · **g d** Downloads	the same	the same
several at once	**Space** marks each, then **Enter** (where the program accepts several)	Space, then Enter	—
save into another folder	—	—	go there and press **Enter** on the folder
replace a file	—	—	**Enter** on it; it asks first
search · hidden files	`/` · `.`	the same	the same
cancel	**Esc**	**Esc**	**Esc** (the suggested file goes again)

Saving shows the file before it is written: the program's suggested name is waiting in the folder, standing in for the file until you choose — its preview says *the file being saved* — and Enter keeps it there. Cancel, and the stand-in is removed again: nothing is left behind.

How it is wired, should you look: the portal `xdg-desktop-portal-termfilechooser` (built by setup from a pinned release, `/usr/local/libexec/`) hands every dialog to `~/.local/bin/kawa-pick`, which opens yazi; `~/.config/xdg-desktop-portal/wlroots-portals.conf` routes dialogs there, with GTK's own dialog as the fallback — should the portal ever be missing, programs show that one, in the desktop's colours, instead. The portal's settings are in `~/.config/xdg-desktop-portal-termfilechooser/config`.

Default programs come from `~/.config/mimeapps.list` (set by Theme), so `xdg-open`, yazi, Brave's downloads and every other program agree:

Type	Opens in
web pages, links	Brave Origin
folders	yazi (in a terminal window)
text, code, config files, Markdown	neovim (in a terminal window)
pictures	imv
video, music, playlists	mpv
PDF, EPUB, XPS, FictionBook	zathura
comic books (CBZ/CBR)	zathura
Word / OpenDocument text, RTF	AbiWord
spreadsheets, CSV	Gnumeric
Xournal++ notes	Xournal++
magnet links, `.torrent` files	torrra (§12)

Programs that ask for a terminal (yazi's own launcher entry, say) get foot.

4.10 Screen sharing — picture and sound

A call in Brave (or any program) that shares a screen asks the desktop portal, which asks what to share with the **share what?** menu: every screen with its size, and every open window by its title. With only one screen and no choice to make, it is taken without asking. Sharing runs at up to 60 frames a second. While anything is shared, notifications are held back so they never appear in the call — critical ones and the level display still show.



"share what?" — two screens with their sizes, and the open windows by title.

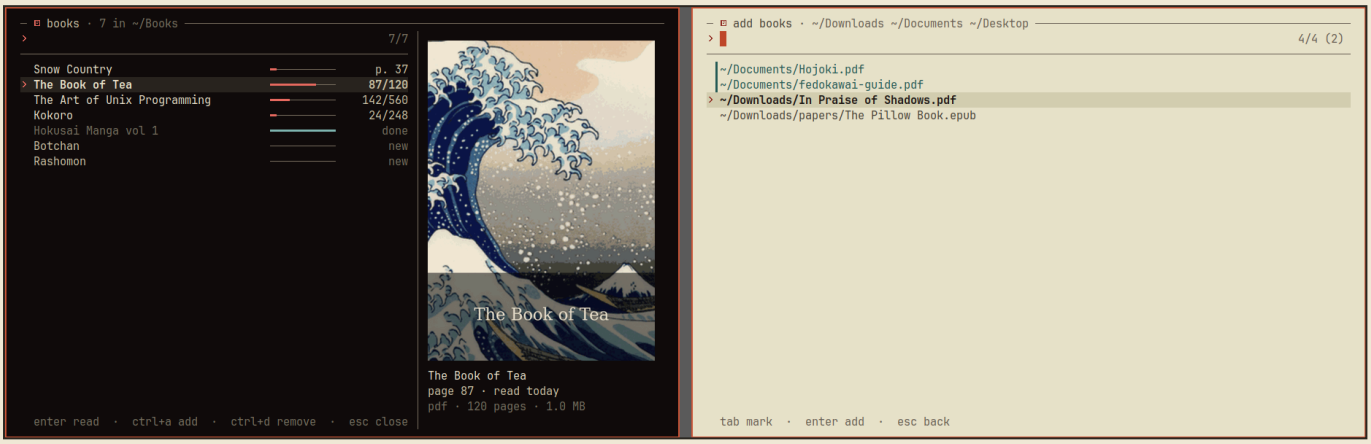
Sound has three ways in, depending on what the call needs:

You want the others to hear	Do
the sound of a **Brave tab** (a video you are showing)	share the tab and keep *Also share tab audio* ticked — Brave captures it itself
the **computer's sound** with a whole screen or a window	tick *Also share system audio* in Brave's picker (Theme turned this Brave feature on). It carries everything you hear — including the call itself, so others may hear an echo of their own voices. Headphones do not help; use it for a moment, or use the next row instead

You want the others to hear	Do
only certain programs (a game, a player), in any call program	in pavucontrol (audio menu → mixer → *Playback*) move those programs to the **Share** output — you still hear them — then pick **Share (as microphone)** as the microphone, or as a second audio input, in the call program. Only what you moved there is sent, never the call

Your microphone itself travels as usual; check it in the audio menu (Super+A → input).

4.11 Books — Super+R



The books window (Super+R). Dark: the ones being read first, each with its page and a bar, the cover of the one under the cursor beside. Light: adding books (Ctrl+A) from Downloads, Documents and Desktop.

Super+R opens your library: every book in `~/Books` (and its folders) — PDF, EPUB, MOBI, FB2, XPS and comic books (CBZ, CBR, CB7) — in a small floating window, the ones you are reading at the top, the most recent first. Each line shows how far in you are — a bar (vermilion while reading, teal once finished) and, at the right, one of four marks:

Mark	Means
`87/120` `0`	reading: on page 87 of 120 (the page zathura remembered when you last closed it)
`p. 37`	reading a book without fixed pages — EPUB, MOBI, FB2 flow their text to the window's size, so there is a page but no total (its bar shows one step: begun)
`done`	finished: you closed it on its last page
`new`	not opened yet (its kind, page count and size show under its cover)

The cover of the book under the cursor shows beside the list, centred in its pane, with the title, where you are and when you last read it under it.

Keys	Does
type	search by title
Enter	read it in zathura — it opens on the page where you stopped (§11.12)
Ctrl+A	add books: every book in `~/Downloads`, `~/Documents` and `~/Desktop` is listed; **Tab** marks several, **Enter** moves them into `~/Books`, **Esc** goes back
Ctrl+D	remove it: the file goes to the trash (yazi: **g t** brings it back)
Esc	close

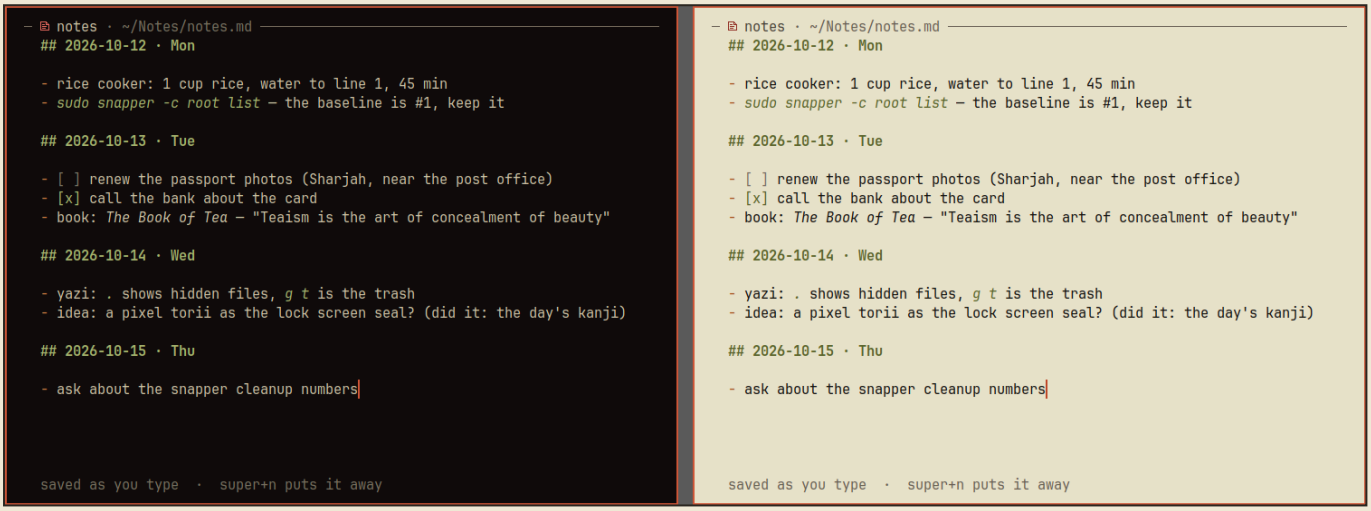
How it keeps track

Nothing new runs in the background and nothing is stored twice: the page comes from zathura's own memory, which zathura writes when you close a book; the window only reads it. Page counts and covers are worked out once per book and kept.

What	Where
your books	**`~/Books/`** — folders inside it are fine
the page you are on, per book	`~/.local/share/zathura/bookmarks.sqlite` (zathura's; read, never written)
covers and page counts	`~/.cache/kawa-books/` — safe to delete, drawn again when needed
the window	`~/.local/bin/kawa-books` (Super+R)

Tips. Folders inside `~/Books` (`~/Books/Japanese`, `~/Books/Work`) keep a large library tidy; the search matches titles anywhere in it. A book moved to another folder keeps its page: the window finds it by its name and opens it there. In zathura, **Ctrl+r** turns the pages into the theme's colours for reading at night (§11.12).

4.12 Notes — Super+N



The notes window (Super+N): one file, a heading per day, open at the end and ready to type.

Super+N opens your notes in a small floating window; **Super+N again** puts them away. It is one plain file, **`~/Notes/notes.md`**, that reads top to bottom like a diary: the first time you open it on a day, that day's date becomes a heading, and the window opens at the end, ready to type. Write anything — a phone number, a command, a thought — and press Super+N; it is saved as you type, so closing the window any other way (Super+Q included) loses nothing. A day you opened but wrote nothing under leaves no empty heading behind.

What	Where
your notes	**`~/Notes/notes.md`** — plain Markdown, one file
the window	`~/.local/bin/kawa-notes` (Super+N): neovim, the desktop's editor, dressed as a page — the name in a rule across the top and a quiet line of keys along the bottom, like every popup; no line numbers, the text three columns in (`~/.config/fedokawai/notes.lua`)

It is neovim (§11.5), so its keys work: **Esc** then `/word` finds something, `gg` goes to the top, `u` undoes. Markdown is optional, but it makes notes easy to scan later: `- [ ] call the bank` for a to-do, `**bold**` for what matters.

Tips. To find something from any terminal: `rg -i rice ~/Notes`. Notes are plain text in your home, so the snapshots of `/home` keep their history too (§7.2). Want a separate file for a project? Start one in `~/Notes` from neovim (`:e ~/Notes/japan-trip.md`); Super+N always opens `notes.md`.

5. How the look is built

5.1 Fonts

Font	Where	Why
Departure Mono (Nerd Font build)	bar, menus, notifications, boot menu, the game overlay	a pixel font on an 11 px grid — set at 11 px (or 22, 33), never in points, with smoothing off
Sumi Icons	every icon in the bar, menus and notifications — and, as *Sumi Icons Term*, in the prompt, fetch and the clipboard and wallpaper windows	the desktop's own icon font, built from `art/icons.txt`
PixelMplus 12	Japanese in the pixel UI (song titles in the bar, menus, notices) and in the terminal	a 12 px Japanese pixel font, so a Japanese title in the bar stays sharp
JetBrains Mono (Nerd Font)	the terminal and neovim	readable code at 11.5 pt
Noto Sans CJK	Japanese, Chinese and Korean in programs, web pages and subtitles	a full, smooth face where text is read at length
Terminus	the text console	sharp bitmap font for the login
Noto Sans, Noto Naskh Arabic, Amiri, Liberation, Noto Color Emoji	GTK and Qt programs, documents, Arabic, emoji	

Fontconfig switches smoothing, hinting and colour fringes **off** for the three pixel fonts (the rule comes last, because in fontconfig the last matching rule wins). PixelMplus is drawn on a 12-pixel grid, so it stays unsmoothed only at 12 and 24 px; at any other size (Japanese inside the 11 px bar, say) it keeps the normal smoothing, which reads better than broken pixels.

5.2 Icons



The icons of Sumi Icons, eight times their size: each square is one pixel on screen. Here each is centred in its tile; in the font it sits on the text's middle line.

Every icon is drawn as text in `art/icons.txt` — `#` for a pixel, `.` for nothing — on the same 11 px grid as Departure Mono, ten pixels tall and centred on the middle of the text line. `tools/build-icons.py` merges each drawing into a few rectangles and writes `SumiIcons.ttf` (code points U+FE000 onward, clear of every Nerd Font), its terminal cut `SumiIconsTerm.ttf` ("Sumi Icons Term": the same drawings two pixels higher, centred on the terminal font's taller capitals) and `~/.config/fedokawai/icons.sh`, which gives the scripts names like `$ICON_SPEAKER`. Each font carries one harmless hinting instruction, so FreeType never auto-hints it: every pixel lands exactly where it is drawn. To change an icon: edit its drawing, run `python3 tools/build-icons.py` (in a throwaway Python environment with `fonttools`, and `Pillow` for `--preview`: `python3 -m venv /tmp/v && /tmp/v/bin/pip install fonttools pillow`), then **Theme**. Add new icons at the end of the file — inserting one in the middle renumbers the rest.

The meters are made of icons too: `eq-0` … `eq-5` are 3-pixel columns with up to five lit segments and a 4-pixel advance, and `eq-cap` has no width at all — printed after a full column in vermilion, it repaints that column's top segment. The music meter and the CPU history are built from them, and so is the level display's wedge (at twice the size); the brightness menu and the volume menu's title use the framed-bar icons `lv-*` (cells 8 px tall).

5.3 Transitions

`kawa-veil` (C, about 750 lines in `src/kawa-veil/`, built by **Theme** with the tools that build dwl) makes the theme and wallpaper changes smooth. It takes a still of every screen (the compositor's screencopy), shows it on the topmost layer with every click passing through, runs the change underneath, waits a moment for the apps to redraw, then lets the still go block by block over about two thirds of a second, with an ease in and out. Each block is one pixel of the wallpaper pack's art on that screen (3 px at 1080p, 4 at 1440p, 6 at 4K), and a block is only ever cleared, never blended, so every frame stays crisp. The order the blocks go in is the design: an 8×8 ordered dither for a wallpaper, and for a theme a front of seigaiha crests — arcs 96 px wide at 1080p (128 at 1440p, 192 at 4K) with a dithered fringe — rising or falling across the screen. It works on rotated screens too. If anything is missing

(no compositor, no screencopy) the change simply happens without it, and a watchdog removes the veil after 20 seconds whatever happens.

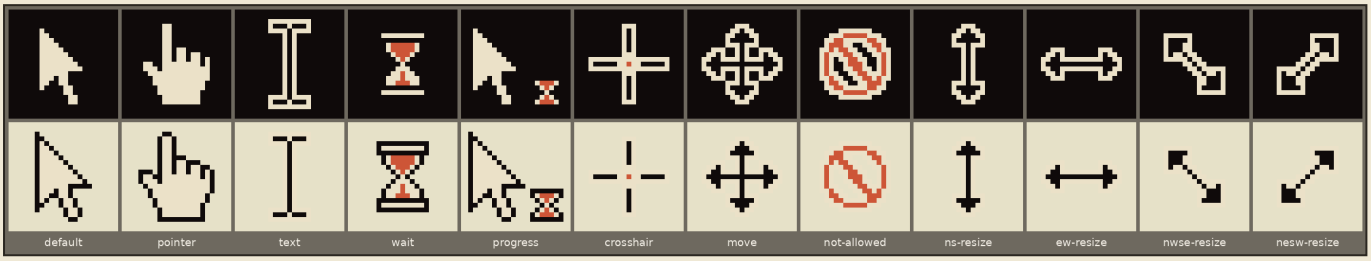
5.4 Where every app gets its colours

All theme files live in `~/.config/fedokawai/themes/dark/` and `…/light/`, written from one palette by `tools/build-themes.py`. **`~/.config/fedokawai/current`** is a link to one of them; every app reads its colours through that link, and `kawa-theme` switches it, then tells whatever is running to look again.

App	Reads the theme by	On a switch
foot (and everything in it: neovim, btop, yazi, lazygit, torrra, spotify_player, bat, nmtui, the clipboard window)	both palettes in `foot.ini`; `include=` picks the one to start in	SIGUSR1 / SIGUSR2 to the foot server switches every window; neovim notices by itself
neovim	gruvbox.nvim with the palette remapped; `background` is never forced	follows the terminal (DEC mode 2031)
yazi, lazygit, bat, torrra, the clipboard window, nmtui	the terminal's 16 colours by name (yazi's `theme.toml`, lazygit's config, bat's `ansi` theme, torrra's `ansi-dark` (right on both: the palettes keep "black" a shade of the ground), `NEWT_COLORS` for nmtui)	with foot, at once
waybar	`@import` of `current/waybar.css`	SIGUSR2 (reload)
fuzzel	`include=current/fuzzel.ini`	read fresh at every menu
mako	`include=current/mako` (outside any section, as mako requires); the file carries its own sections for low-urgency, critical and level notices	`makoctl reload`
GTK 4 / libadwaita	one `gtk.css` with both palettes, chosen by GTK's color-scheme media query	gsettings `color-scheme`
GTK 3	two GTK themes of its own, `Fedokawai` and `Fedokawai-dark` (adw-gtk3 + the palette, in `~/.local/share/themes`)	gsettings `gtk-theme` — every open window, the file dialog included
Brave	GTK mode (set by Theme): its tab strip, toolbar and menus come from the GTK theme, a few rules in it reach Brave alone (the address bar keeps Brave's own colours); the new-tab page is the ground colour (`light-dark()`); web pages follow the portal's dark / light	live
Qt	`qt6ct.conf` is a link into the theme	the link is replaced; qt6ct watches its folder
mpv	`include` of `current/mpv.conf`: on-screen text, the controller, menus	every open player loads the new file over its socket
btop	`color_theme = "fedokawai"`: `~/.config/btop/themes/fedokawai.theme` is a link through `current` (btop takes themes only by name)	SIGUSR2
zathura	`include ../fedokawai/current/zathurarc`	`SourceConfig` over D-Bus
imv, presenterm	their config file is a link into the theme	next start
swaylock	`kawa-lock` passes `-C current/swaylock`	next lock
the screenshot region	`kawa-screenshot` reads `current/colors.sh`	next screenshot
wallpaper	each theme's own choice	`kawa-wallpaper --restore`
MangoHud (games)	`MangoHud.conf` in the dark palette and Departure Mono	—
the console, boot menu	always the dark theme's colours	—

To change a colour: edit the palette at the top of `tools/build-themes.py`, run `python3 tools/build-themes.py`, then **Theme**. A few files cannot read a generated one — dwl's `config.h` (compiled in), mpv's subtitle colours, satty, MangoHud, `fetch` and the scripts' fallbacks — so they hold the colours themselves, and every run of the generator checks that each one is still a palette colour (it stops if not).

5.5 Pointers



The mouse pointers at 8× their size, on ink and on paper.

The mouse pointer is drawn for the desktop too: the theme **Fedokawai**, in `~/.local/share/icons/Fedokawai/`. The arrow and the hand are the classic shapes in ink with a paper body, so they read on both themes; the thin ones — the text cursor, the cross for screenshots, the arrows for moving and resizing a window — are ink lines with a one-pixel paper rim; the hourglass's sand, the cross's centre and the "not here" sign are vermilion. Each is a drawing in `art/cursors.txt` at the desktop's own pixel scale: 24 px, every pixel one screen pixel, like the fonts and icons, and 48 px with every pixel doubled for a 2× screen — so a pointer is never smoothed or blurred.

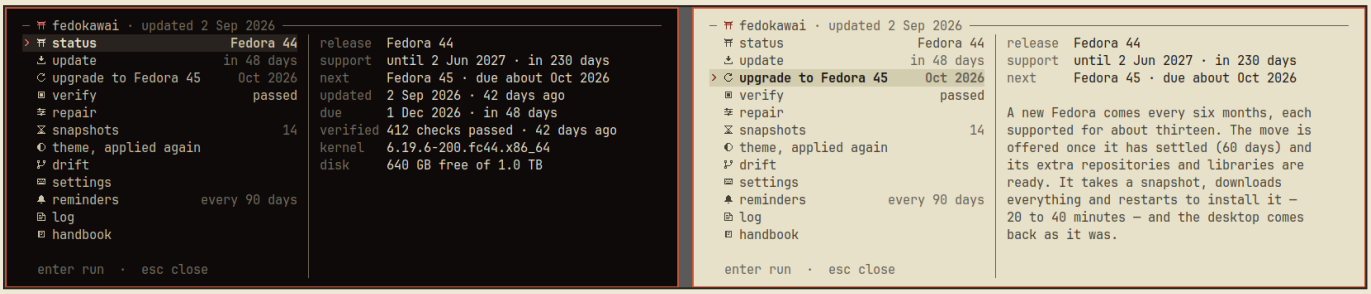
Where	What
`~/.local/share/icons/Fedokawai/cursors/`	the twelve pointers, and links for the older names programs ask for (`left_ptr`, `hand2`, `xterm`…)
`~/.local/share/icons/default/index.theme`	makes Fedokawai the default for dwl itself and older (XWayland) programs
`XCURSOR_THEME`, `XCURSOR_SIZE` · `cursor-theme`	the same for everything else: the session's environment (`start-dwl`) and GTK's setting (`/etc/dconf`)

Shapes not drawn here — the rarer ones, like a "copy" or "zoom" pointer — come from Adwaita underneath. To change one: edit its drawing, run `python3 tools/build-cursors.py` (standard Python only; `--preview out.png` draws them all), then **Theme**; the change shows at the next login.

6. Keeping it healthy

The machine is built to be left alone for months and to run for years on a toolkit that is never edited again: once a season you **update** it, about once a year you **move** it to the next Fedora, and both are one entry in one menu.

6.1 The control — Super+U



The system's control (Super+U): what is due at the right of each entry, how things stand in the pane — the status on the dark theme, the move to the next Fedora on the light.

Super+U opens it (also quick settings → *system*, or `fedokawai` in a terminal): a small window with nothing to search, only twelve entries — ↑ ↓ (or **j k**) to choose, the mark at the right of each saying what is due, and beside them a pane with how things stand for the entry under the cursor. **Enter** runs it in the same window — what changes the system asks for your password — and a key afterwards brings the menu back with its marks brought up to date; **Esc** (or **q**) closes it.

Entry	Does	Its mark
status	how things stand, on one page	the release
update	the season's update (§6.2)	when the next is due; **due** in red once it is
upgrade to Fedora N	the move to the next release (§6.4)	when it is expected or offered; **ready** in red once it has settled
verify	every check (§6.7); changes nothing	**passed**, or how many to fix
repair	verify, put right what it found — packages, services, the login, dwl (rebuilt if it will not start; work of yours in its folder is set aside as a branch or a stash, never lost), the session, the tools, neovim's plugins, the configs where the theme was chosen — and verify again; offline too, as far as it can	
snapshots	the snapshots of `/` (§7.2)	how many
theme, applied again	every config back as the toolkit has it (yours kept in the backup)	
drift	the configs that differ from the toolkit's; changes nothing	
settings	a copy of `fedokawai.conf` in neovim; when you close it every value is checked, and only a valid file replaces the toolkit's (a wrong one is shown and nothing changes); the next update applies it	
reminders	Enter changes the season: every 60 · 90 · 120 days · off	the season
log	the last run's log, at its end	
handbook	this book	

Every entry is a command too — `fedokawai status`, `update`, `upgrade` (`--now`: without waiting), `verify`, `repair`, `theme`, `drift`, `snapshots`, `log`, `settings`, `guide`, `remind [every DAYS | wait DAYS | on | off]` — and `update`, `verify` and `repair` work on their own in a terminal. On a text console (Ctrl+Alt+F3), or should fzf ever fail to start, `fedokawai` shows the same menu as a numbered list: the system can be looked after even when the desktop cannot start.

Reminders. Two minutes after you log in, and every six hours while you stay, a quiet check reads what is known — your last update, your release's end of support, whether the next release has settled — and shows at most one notice a day, only when something is due. It never goes on the network. The season is in `~/.config/fedokawai/upkeep.conf`: `EVERY` (days between updates, 90), `WAIT` (days a new release settles before the move is offered, 60), `REMIND` (`yes` or `no`).

The toolkit. What root runs — every update, repair and move — is the copy setup installed in `/usr/local/lib/fedokawai`, owned by root, so nothing running as you can change it; the `fedokawai` command is installed from it. The folder you installed from can go once setup is done. *settings* edits the toolkit's own `fedokawai.conf`. To change any other file of the toolkit, change it in a copy of the project and run an update or a repair from there (`sudo bash fedokawai.sh update`): that copy becomes the toolkit. Its `fedokawai.conf` is merged key by key: a value you saved through *settings* stays, unless the copy changed that key since the toolkit was last made from a copy — then the copy's is meant. The log names every value taken from the copy, and the toolkit's file before is kept as `/var/lib/fedokawai/fedokawai.conf.before`. The toolkit holds plain files and folders only: a copy with a symbolic link in it is refused — the toolkit stays as it was, and the update or repair stops there, before anything is applied.

6.2 The season's update

update brings everything up to date in one run, in this order:

Step	What happens	If it fails
checks	10 GB free on `/` and 300 MB on `/boot`, the clock, no earlier update waiting for a restart, dnf's own state (packages left twice by an interrupted run are tidied)	it stops before anything has changed, and says what to do
snapshot	a numbered snapshot of `/`, *before update DATE*, checked to exist	it stops: nothing is updated without one
packages	one `dnf upgrade --refresh` — Fedora, RPM Fusion, Brave, Claude Code, the COPRs — run as a system service of its own, its output followed in the window and kept in the log: closing the window, or the session ending, cannot cut it off halfway	tried again; if a third-party repository is what fails, Fedora's own updates go ahead without it and the report says so; otherwise it stops with nothing changed
`fedokawai.conf`	its system values applied again: keyboard, time zone, DNF, SSH, the performance profile	
the manifest	anything missing installed; a package a newer Fedora no longer has is listed, never fatal	
the desktop's own parts	dwl's wlroots, the pinned tools and fonts (§6.3), GE-Proton, the file dialog	each on its own: one that fails — GitHub down, say — is reported, the rest goes on, and the next update tries it again
torrra · Prowlarr	libtorrent to its newest under torrra; Prowlarr's newest image, then started and checked	
neovim's plugins	your current versions kept first (`~/.local/state/fedokawai/lazy-lock/`, the last six), then `Lazy! sync`	
verify	every check, the outcome recorded for the menu	

The dnf transaction itself also gets its pre/post pair of snapshots. At the end the update says whether a new kernel waits for a restart, and notes whether the next Fedora is out. The outcome — complete, *partial* (a part to retry next time) or *stopped* — shows in the menu; the log keeps every line (`fedokawai log`). Not included: language servers you installed through Mason in neovim (`:Mason`, then `U`).

6.3 Pinned, and why

The few tools Fedora does not package come from their official releases, each **pinned**: an exact version and the SHA-256 of its file. The desktop's configs are written for exactly these — yazi's layout code, the fonts' pixel metrics every alignment is measured in — so an update never moves them, and a download that differs by one byte is refused.

Tool	Version	Held because
yazi	26.9.1	its theme, and the file dialog's layout code, use its internals
satty · presenterm · spotify_player	0.22.0 · 0.16.1 · 0.25.1	their configs are written for these
JetBrains Mono and Departure Mono (Nerd Fonts)	3.5.1	every alignment is measured in their pixels
PixelMplus 12	2013	its only release
torrra	2.5.1	its key wrapper replaces three of its methods; libtorrent under it moves to the newest
GE-Proton	the newest; two kept	games are what it is for, and it changes nothing else

dwl is built from a pinned commit (`DWL_COMMIT`), and so is the file dialog's portal (`TFC_COMMIT`).

6.4 A new Fedora — upgrade

A new Fedora comes every six months, in late April and late October; each is supported for about thirteen, until four weeks after the release two later — Fedora 44 until 2 June 2027. The menu shows your release's date from Fedora's own record of it (`SUPPORT_END` in `/etc/os-release`), and offers the move when it is sensible:

1. **Out.** A release is out once Fedora publishes its release tree. An update finds that out, and so does opening the menu — one request to Fedora's own servers, at most once a day.
2. **Settled.** The move is offered `WAIT` days (60) after the release day, once its first fixes are in — sooner when your release ends within 60 days. `fedokawai upgrade --now` does not wait.
3. **Ready.** When you choose it, the move checks first, and stops with nothing changed unless RPM Fusion already has the new release (ffmpeg and the drivers come from there), there are 20 GB free, and every library dwl needs is in it (one that a tool of the desktop's own needs is reported). A COPR that has not built for the new release yet is left out, its packages kept as they are, until it has.
4. **The move.** Your release fully updated, a snapshot of `/` , everything downloaded — should dnf report conflicts, it lists what it would remove and asks — then a restart into Fedora's own installation: 20–40 minutes with nothing else running. Leave the computer on.
5. **After.** Once, by itself, after the restart: the manifest again, packages left twice tidied, torrra for the new Python, the theme (dwl rebuilt if needed), a new baseline snapshot for the new release, the checks — and the outcome, told once in a notice when you next log in (nobody is logged in while it runs). Configuration files the new packages could not merge (an `.rpmnew` beside yours) and packages no longer in any repository are listed in the status, for you to look at — never changed by themselves.

If the installation fails, the computer restarts into your old release as it was, and the notice at your next login says so. If the new release starts but the desktop does not, the boot menu still has your previous kernel, and the snapshot taken before the move holds the old system (§7.2).

6.5 Built to be left alone

The toolkit is never edited after it is installed, so whatever changes around it over the years is handled one way — checked for, worked around, or reported, never a silent break:

In time	What happens
Fedora moves to a newer wlroots and drops the one dwl is written for	dwl uses its own copy, kept from Fedora's build at the first install (`/usr/local/lib/fedokawai-wlroots-0.20`); a move checks the new release has every library under it
dwl cannot start — a library missing	the login starts the build before it (`dwl.prev`); with neither, a rescue screen on the console says what to do, instead of a login that loops back
a new kernel misbehaves	the one before it stays installed, in the boot menu

In time	What happens
a package is renamed or retired	listed, never fatal: the rest goes on without it
a third-party repository is down, changes its key, or has nothing yet for a new release	skipped with a warning; Fedora's own updates never wait on it
a release tool's download changes	refused by its checksum; the version in place stays
a run is interrupted — the window closed, the power cut	dnf runs apart from the window; the next run tidies a half-finished transaction; every record is written whole or not at all
the disk fills	every run checks for room first; snapshots stay within bounds (a fifth of the disk stays free)
dnf's clean-up (`autoremove`) would take a package the desktop needs	never: every package in the manifest is marked as yours. Nothing is *protected* by name, so a later Fedora can still replace one (say, sudo by sudo-rs) without holding back every update
a tool's output changes its language or format	every script reads it in one fixed language (`LC_ALL=C.UTF-8`)
two runs at once	one at a time; the menu shows when one is going on

6.6 Kernels, DNF, snapshots

DNF keeps two kernels: the newest and the one before it (`installonly_limit=2` , the DNF 5 minimum). Should a new one fail to boot or to start the desktop, choose the older one in the boot menu (hold Esc if it is hidden); the next update replaces it.

`/etc/dnf/libdnf5.conf.d/90-fedokawai.conf` holds only what differs from Fedora's defaults: ten parallel downloads, the fastest mirror, a 50 kB/s floor with a 20 s timeout (slow mirrors are dropped), and *yes* as the default answer. Setup and `verify` check all six values with `dnf --dump-main-config` .

snapper keeps timeline snapshots of `/` (and `/home` when it is its own subvolume), a *baseline* taken right after the verified build, a pre/post pair around every DNF transaction, and one before every update and every move. For a machine left alone for months they stay within bounds: at most twenty numbered ones (fewer when space runs low — a fifth of the disk stays free), the last ten marked important (those before updates and moves among them), hourly ones for a few hours, daily ones for a week. The baseline stays until you delete it — or until the next Fedora: a move takes a new baseline for the new release, and the old one then ages out with the numbered ones. `~/Downloads` is a subvolume of its own, so downloads stay out of `/home` 's snapshots: the space of a deleted download comes back at once.

The machine runs the **tuned** profile `fedokawai` : Fedora's *throughput-performance* (the CPU at full speed — this is a desktop that is always plugged in) with swap used early and big writes flushed sooner, in smaller batches, so a large copy never stalls the desktop; Wi-Fi power saving is off, so the connection never naps.

6.7 What verify proves

Every check fails if its probe fails — a broken tool can never produce a pass.

Area	Checks
system	Fedora ≥ 44 · SELinux enforcing · Btrfs root · no root-owned files in your home · groups · neovim as the editor · the tuned profile · Wi-Fi power saving off · `secrets.sh` private · linger · ntsync and i2c-dev · the SSH policy and firewall ports (6881 for torrents)
packages and redundancy	every manifest package installed · none of the forbidden ones · nothing user-installed beyond the pre-setup record and the manifest · no Flatpak or Snap
repositories and DNF	RPM Fusion, Brave, Claude Code, COPRs · all six DNF values
upkeep	the toolkit root's alone · the `fedokawai` command the toolkit's · the manifest marked as yours · the snapshots' bounds
boot	the boot-menu setting · at most two kernels, the newest running · no rescue image · the GPU kernel argument

Area	Checks
GPU and media	hardware video decoding · Vulkan · ffmpeg encoders
compositor and login	dwl version and pinned commit, XWayland, its own wlroots and every library, the patch (keys, gaps, frames, centring, layout), built from `config.h` · the session launcher and its rescue screen · the greetd SELinux module, agreety · the boot-success timer
services	greetd, Bluetooth, snapper timers, fstrim, PipeWire, Prowlarr
torrents	the key in `secrets.sh` · Prowlarr's container file current, running, answering, taking the key · its files yours · indexers (a warning when there are none) · podman's database · `~/Downloads` outside the snapshots
release binaries	present, every library resolves, `--version` runs · fonts
applications	Brave Origin actually starts · Claude Code · LazyVim · torrra loads and runs through its wrapper, with no key in its config · mpv in the bar · yt-dlp's JavaScript runtime · lazygit · qalc
theme	every file in sync · foot and fuzzel accept their configs · waybar config parses · GTK and the portal follow the current theme · no window buttons · default programs · Brave in the desktop's colours · Qt palette · fonts resolve (Arabic, Japanese, Chinese, Korean) · mpv's colours · pixel art drawn for both themes · the two C helpers built · the clipboard window · the wallpaper pack · the pixel pointers · the yazi file dialog · cava · the boot menu theme
live session	bar, notifications, terminal server, wallpaper, idle, automount, status readers, polkit agent, clipboard history and keeper, the update reminder · the graphical session known to systemd · the file-dialog and screen-sharing portals · the "Share" sound devices · the refresh rate is the best available · a monitor answers DDC/CI · night light (shown, never a fault: it is yours to switch off)

6.8 Between seasons

In between, nothing changes underneath you — nothing updates, or even checks for updates, in the background:

Would otherwise	Here
DNF fetches package lists every hour or so (its *makecache* timer)	off: lists are fetched when you update (`dnf upgrade --refresh`)
neovim's plugin manager looks for new plugin versions	off (`checker = { enabled = false }` in `lua/config/lazy.lua`); plugins move only with an update, which keeps the old versions first
Prowlarr's image is replaced as soon as a new one appears	fetched by an update only; its next start uses it
tldr refreshes its pages	at most once a quarter, and only when you run it
the release tools and fonts	pinned (§6.3)

Two programs live on the internet's schedule rather than yours, and are worth updating on their own when they ask — each command touches that one program only:

When	Command
Brave reports a security fix you want now	`sudo dnf upgrade --refresh brave-origin`
YouTube stops playing in mpv, or yt-dlp says to update	`sudo dnf upgrade --refresh yt-dlp`
security fixes only, between seasons (optional)	`sudo dnf upgrade --refresh --security`

If an update breaks neovim

An update keeps the plugin versions you had in `~/.local/state/fedokawai/lazy-lock/` (the last six). To go back: copy the newest one over `~/.config/nvim/lazy-lock.json`, open neovim and run `:Lazy restore`. Anything else an update changed, snapper undoes (§7.2): every `dnf` run is wrapped in a pre/post pair, and the update itself starts from a snapshot.

7. Keeping it clean

Chapter 6 keeps the machine working. This one keeps it tidy: where files belong, how the snapshots work and how to use them, how to keep the package list short, and how to find and free disk space. Nothing here runs by itself unless it says so.

7.1 Where things go

Each kind of file has one place. Knowing which places belong to the scripts, which to you, and which can be thrown away is most of keeping a desktop clean.

Place	Holds	Who writes it	Safe to delete?
`/etc`	system-wide settings: DNF (`/etc/dnf/libdnf5.conf.d/90-fedokawai.conf`, the snapshot hook in `/etc/dnf/libdnf5-plugins/actions.d/`), the tuned profile, NetworkManager, `/etc/dconf`, `/etc/default/grub`, snapper's configs	Setup and Theme (as root), and Fedora's packages	no. Change Fedokawai's own files through `fedokawai.conf` and the toolkit (§2); anything else with `sudoedit FILE`
`~/.config`	every program's settings	Theme owns the files listed at the start of chapter 11; the rest are yours	no — but a managed file edited here is replaced at the next Theme run
`~/.config/fedokawai/`	`env`, `themes/`, `current`, `icons.sh`, `secrets.sh`	Theme (`env` is written from `fedokawai.conf`); `secrets.sh` is yours	no
`~/.local/bin`	`kawa-*` helpers, `fetch`, `kawa-veil`, `kawa-clipkeep`, the release binaries	Theme and `update`	no. Your own scripts may live here too, under names that do not start with `kawa-`
`~/.local/share`	program data: Steam and its games, pipx's torrra, podman's images, the trash, neovim's plugins and Mason tools, the pixel art and the wallpaper pack	the programs themselves	no, not as a whole; see §7.3 and §7.4 for the parts you can trim
`~/.local/state/fedokawai/`	theme stamps, each theme's wallpaper choice, `backup/<time>/`, `lazy-lock/` (neovim's plugin versions before each update)	Theme, `update` and the session	only the old folders in `backup/`
`~/.cache`	logs, previews, the scaled wallpapers, the clipboard history, launcher counts	the programs	yes, piece by piece; every program rebuilds its part (§7.4)
`~/Downloads`	what Brave, torrra and yt-dlp fetch	the programs	yes — and its space comes back at once: it is its own Btrfs subvolume, outside every snapshot
`~/Pictures/Screenshots`	Print, Shift+Print, satty's Ctrl+S, mpv's `s`	the screenshot tools	yes, whenever you like
`~/Pictures/Wallpapers`	your own wallpapers, shown on both themes	you	yes
`~/Books`	your library: what the books window (Super+R) lists	you (and its Ctrl+A)	yes, book by book — Ctrl+D in the window sends one to the trash
`~/Notes/notes.md`	your notes (Super+N)	you	only if you mean it: it is your writing

The one rule for configuration

Do not edit a file Theme generated — `~/.config/fedokawai/env`, anything under `themes/`, a managed config. Change `fedokawai.conf` or the copy under `configs/home/` in the toolkit (§2), then run `fedokawai theme`. A file you changed in place is listed by `fedokawai drift`; the next run puts the toolkit's copy back and keeps yours in `~/.local/state/fedokawai/backup/<time>/`.

Your own files

A short, fixed layout makes them easy to find, to back up and to clean:

Folder	For	Why
`~/repos/`	every git repository, one folder each	the prompt shows the branch and changes there; lazygit and neovim find the project from the folder. The Fedokawai clone can stay at `~/fedokawai` (§3.2)
`~/Documents/`	papers, letters, spreadsheets, PDFs you keep	one place for everything you would hate to lose
`~/Pictures/` · `~/Videos/` · `~/Music/`	what you keep of each kind	the standard folders; file dialogs and yazi know them
`~/Downloads/`	arrivals only	not in snapshots — move what you keep into one of the folders above, delete the rest
`~/Books/`	books and comics, in folders of your own if you like	the books window reads it (§4.11); its Ctrl+A moves books here from `~/Downloads`, `~/Documents` and `~/Desktop`
`~/Notes/`	`notes.md` (Super+N), and any other notes you start	plain Markdown: searchable with `rg`, readable anywhere (§4.12)

Snapshots are not backups: they live on the same disk. Copy the folders in the second table to another disk or machine now and then.

7.2 Snapshots

snapper does here what Time Machine or Timeshift do elsewhere: it keeps earlier states of the system so a change can be undone. Setup made two configurations, both Btrfs snapshots:

Config	Covers	Snapshots kept in
`root`	`/` — the system, `/etc`, `/usr`, `/var`	`/.snapshots/`
`home`	`/home` (only when it is its own Btrfs subvolume, as the installer makes it) — **except** `~/Downloads`	`/home/.snapshots/`

What makes a snapshot

Trigger	Config	Cleaned up by
every DNF transaction: a *pre* before and a *post* after, described `dnf YYYY-MM-DD HH:MM`. dnf5's actions plugin runs `/usr/local/sbin/kawa-snap` from `/etc/dnf/libdnf5-plugins/actions.d/fedokawai-snapper.actions`	`root`	the *number* rule
every hour (`snapper-timeline.timer`)	`root` and `home`	the *timeline* rule
the *fedokawai baseline*, taken once after the verified build — and again for each new Fedora	`root`	none — kept until you delete it, or until the next release's baseline replaces it (then the *number* rule)
`sudo snapper -c CONFIG create -d "…"`	either	none — kept until you delete it

`snapper-cleanup.timer` applies the rules once a day. How many are kept is set in each config by setup (the bounds in §6.6: `NUMBER_LIMIT=4-20`, `NUMBER_LIMIT_IMPORTANT=4-10`, hourly 2–8, daily 2–7, monthly 0–2, `FREE_LIMIT=0.2`); read them with `sudo snapper -c root get-config`. The ones that matter are `NUMBER_LIMIT` (DNF pairs and the baseline), `TIMELINE_LIMIT_HOURLY`, `_DAILY`, `_WEEKLY`, `_MONTHLY` and `_YEARLY`. Change one with, for example:

```
sudo snapper -c root set-config NUMBER_LIMIT=20
sudo snapper -c home set-config TIMELINE_LIMIT_DAILY=5 TIMELINE_LIMIT_MONTHLY=2
sudo snapper -c home set-config TIMELINE_CREATE=no      # no hourly snapshots of /home at all
```

The baseline has no cleanup rule, so the automatic cleanup never removes it: it stays as the known-good state of the freshly built system until you delete it yourself, or until a move to the next Fedora takes a new one (*fedokawai baseline (Fedora N)*) and hands the old one to the numbered cleanup. To keep any other snapshot for good the same way: `sudo snapper -c root modify --cleanup-algorithm "" NUMBER` (the number from `snaps`).

Looking and comparing

Command	Does
`snaps`	the list for `/` (an alias for `sudo snapper -c root list`)
`sudo snapper -c root list --type pre-post`	only the DNF pairs, one line each
`sudo snapper -c home list`	the list for `/home`
`sudo snapper -c root status 41..42`	every file changed between two snapshots
`sudo snapper -c root diff 41..42 /etc/some.conf`	the change inside one file
`sudo snapper -c root status 41..0`	everything changed since 41 — `0` is the system as it is now

Undoing

You want	Do
undo one DNF transaction	`sudo snapper -c root undochange 41..42`, reboot, `verify` (§13.9)
one system file as it was	`sudo snapper -c root undochange 41..42 /etc/some.conf`
one of your files as it was	copy it out of the snapshot: `sudo ls /home/.snapshots/57/snapshot/$USER/` to find it, then `sudo cp -a /home/.snapshots/57/snapshot/$USER/Documents/letter.odt ~/Documents/` (`-a` keeps you as its owner)
a file deleted by mistake	the same, from the newest hourly snapshot that still has it
the whole system back to an earlier state	`sudo snapper -c root undochange 41..0` — every file changed since 41 goes back. There is no one-step rollback on this layout (§11.21): the boot entries name the `root` subvolume, so `snapper rollback` would boot the same system again

A whole-system undo is the same tool as undoing one transaction, only wider: it puts files back and runs no package scripts, and `/boot` (the kernels) is not in the snapshots. Reboot and run `verify` afterwards.

Before a risky change — a new repository, a driver, a big edit in `/etc` — take your own: `sudo snapper -c root create -d "before X"`. If it goes wrong, `status N..0` shows everything since, and `undochange N..0` puts it back.

Deleting

`sudo snapper -c root delete 30-40` (a range) or `delete 31 35`; the same with `-c home`. Snapshots hold every file that was there when they were taken, so deleting a large file frees nothing until the last snapshot holding it is gone.

Not in any snapshot:

- `~/Downloads` — its own subvolume (setup made it while the folder was empty; `verify` warns if it is not). A Btrfs snapshot never includes a subvolume nested inside it.
- `/boot` and `/boot/efi` — separate partitions; the previous kernel is the safety net (§6.6).
- everything else in your home **is** in the `home` snapshots: `~/.cache`, Steam's games, podman's images. Deleting them frees their space only as the hourly snapshots expire. To keep another big folder out the way `~/Downloads` is, make it a subvolume while it is still empty: `sudo btrfs subvolume create DIR && sudo chown $USER: DIR` (then `restorecon DIR`).

7.3 Keeping packages lean

Every package you want lives in `lib/manifest.sh`, and `verify` fails on anything user-installed beyond the manifest and what was there before setup. That rule keeps the list honest; these keep the disk lean.

Command	Does
`dnf repoquery --userinstalled`	packages someone asked for (the rest came in as dependencies) — what `verify` compares with the manifest
`sudo dnf autoremove`	removes dependencies nothing needs any more; read the list before saying yes. Manifest packages are user-installed, so it never touches them
`sudo dnf clean all`	empties DNF's download and metadata caches in `/var/cache/libdnf5` (they come back at the next upgrade)
`rpm -q kernel-core`	two kernels at most: the newest, and the one before it

Adding a package properly

A package you keep goes into the manifest (§13.6): its name in `PKG_APPS`, a row in `PACKAGES.md`, then `update`. A one-off — something to try for an afternoon — can be `sudo dnf install NAME`; `verify` will list it as *installed outside the manifest* until you `sudo dnf remove NAME` again. Never leave it there to be forgotten.

Taking one out

Move it from its list to `RETIRED_PKGS` in `lib/manifest.sh`. `update` then marks it as a dependency (`dnf mark dependency`) rather than removing it, so `verify` stops counting it as yours and `sudo dnf autoremove` removes it once nothing needs it.

Kernels

Two at most: the newest and the one before it, kept as the way back should a new one misbehave (§6.6). DNF removes the oldest by itself when a third arrives; there is nothing to clean by hand.

The rest of what this desktop installs

What	Where	Clean with
torrra (pipx)	`~/.local/share/pipx/venvs/torrra`	nothing to do: torrra stays at its version, and `update` moves libtorrent under it to the newest. `pipx list` shows it; do not `pipx uninstall` it — `verify` checks it
Prowlarr's image	podman's storage in `~/.local/share/containers`	`podman system df` shows the size; `podman image prune` removes layers older images left behind
GE-Proton	`~/.local/share/Steam/compatibilitytools.d`	nothing: `update` keeps the newest two
games	`~/.local/share/Steam/steamapps`	Steam's *Settings → Storage*: uninstall games there. Each game's Proton prefix (`steamapps/compatdata`) and shader cache (`steamapps/shadercache`) can grow to gigabytes; a folder there for a game you no longer have can go
neovim's tools	Mason, in neovim's data folder	`:Mason`, `X` on a tool you no longer use

7.4 Disk space and logs

How full is it? On Btrfs `df` guesses, because snapshots share space. Ask Btrfs:

Command	Shows
`sudo btrfs filesystem usage /`	the real picture: *Free (estimated)*, and *Device unallocated*
`sudo btrfs filesystem du -s ~/Downloads ~/.local/share/Steam`	how much of a folder is its own and how much is shared with snapshots
`du -xh --max-depth=1 ~ | sort -h | tail`	the biggest folders in your home (`-x` stays on one subvolume, so it skips `~/Downloads` ; run it there separately)
`fd -H -S +1g . ~`	single files over 1 GB

`compsize` , `gdu` and `ncdu` are not in the manifest; `du` , `btrfs filesystem du` and `fd` do the job.

The journal

systemd's journal keeps itself within Fedora's default limit (a share of the disk, capped at 4 GB); nothing here changes it.

Command	Does
`journalctl --disk-usage`	how much the journal uses
`sudo journalctl --vacuum-time=4weeks`	delete entries older than four weeks
`sudo journalctl --vacuum-size=500M`	shrink it to 500 MB

For a lasting limit, write a drop-in with `sudoedit /etc/systemd/journald.conf.d/size.conf` : a `[Journal]` line, then `SystemMaxUse=500M` ; `sudo systemctl restart systemd-journald` applies it.

Caches you can clear

What	Clear with	Comes back
scaled wallpapers and picker previews	`rm -r ~/.cache/kawa-wallpaper`	at the next wallpaper change, or `kawa-wallpaper --restore`
thumbnails	`rm -r ~/.cache/thumbnails`	as file dialogs show pictures again
the clipboard history	Super+V, then Ctrl+X (or `cliphist wipe`)	from your next copy
the launcher's counts	`rm -r ~/.cache/fuzzel`	the order of programs starts afresh
old config backups	`rm -r ~/.local/state/fedokawai/backup/<time>`	— keep the newest until you are sure
the trash	**g t** in yazi → *Empty trash bin*, or `rm -rf ~/.local/share/Trash/{files,info}`	—
DNF's caches	`sudo dnf clean all`	at the next upgrade

Clear single folders while you are logged in; the logs (`dwl.log` , `waybar.log`) are open by running programs and are best left alone.

Btrfs care

TRIM runs weekly (`fstrim.timer`). Two more tools exist; neither is needed often:

- **scrub** reads every block and checks it against its checksum, to find a failing disk early: `sudo btrfs scrub start /` , then `sudo btrfs scrub status /` . It runs in the background; once a month is plenty.
- **balance** rewrites block groups. It is almost never needed: only when `btrfs filesystem usage /` shows *Device unallocated* close to zero while data is far from full. Then a gentle one: `sudo btrfs balance start -dusage=50 /` . Never run a full balance as routine.

7.5 A seasonal routine

Once a season, with the update (§6.2):

Step	Command
update everything	**Super+U** → update (or `update` in a terminal) — it ends with `verify` (§6.2)
check the result	the status in the same menu: every check passed, nothing listed to look at
once a year	**Super+U** → upgrade, once the menu offers the next Fedora (§6.4)
drop unneeded dependencies	`sudo dnf autoremove` (read the list)
look at the space	`sudo btrfs filesystem usage /`
sort the downloads	move keepers out of `~/Downloads`, delete the rest; same for `~/Pictures/Screenshots`
empty the trash	**g t** in yazi → *Empty trash bin*
prune snapshots you made by hand	`snaps`, then `sudo snapper -c root delete N`
trim the journal	`sudo journalctl --vacuum-time=4weeks`
check the disk	`sudo btrfs scrub start /`
back up	copy `~/repos`, `~/Documents` and your other folders to another disk

8. Software, and why each piece

The full table is `PACKAGES.md`; these are the choices worth knowing.

Job	Tool	Why this one
compositor	dwl 0.9	small and fast; patched for gaps, retro frames, centred dialogs, Arabic-proof shortcuts, the layout in the bar
bar	waybar	its own clock; custom modules fed by one status reader
launcher and every menu	fuzzel	one look for all menus; no second menu program
terminal	foot	fast, live dark/light palettes, prompt integration, pictures (Sixel)
login	greetd + agreety	a plain text login, no extra greeter
lock / idle	swaylock · swayidle · wlopm	the padlock is a picture, the ring sits in its keyhole; wlopm powers the screens off without moving windows
transitions	`kawa-veil` (this repository, C)	nothing packaged does animated changes on dwl; built locally
clipboard	wl-clipboard · cliphist · fzf · `kawa-clipkeep` (this repository, C)	history with a searchable window and picture previews (ImageMagick, as sixels); copies that outlive their program — the one clipboard keeper for wlroots is not packaged for Fedora, and a small one is easy to prove
notifications	mako	
music meter	cava	the only extra process for the bar, asleep in silence
brightness	ddcutil	the monitor's real backlight
fetch	`fetch` (this repository)	replaces fastfetch: a few milliseconds, pictures drawn here
torrents	torrra + Prowlarr	a terminal client that searches through Prowlarr's hundreds of indexers
video	mpv (+ mpv-mpris, yt-dlp, Node.js)	the best player; Node.js solves YouTube's challenges for yt-dlp

Removed as redundant across versions: tuigreet (agreety is part of greetd), wlogout (a fuzzel menu), fastfetch (own `fetch`), a separate clock script (waybar's clock), starship (native prompt), GStreamer plugins (nothing uses them), and the Rust toolchain (release binaries instead).

9. Troubleshooting

Symptom	Look at
setup stopped	the red line on screen and `/var/log/fedokawai.log`; fix; run it again
something feels off	`verify` — it names the exact piece
the login loops back	`~/.cache/dwl.log`; `sudo journalctl -b -u greetd`
a rescue screen instead of the desktop	dwl could not start (it says why): `fedokawai repair` there, then log out (Ctrl+D) and in again
an update stopped, or did only part	the menu's status line (Super+U) and `fedokawai log`; most causes go with `fedokawai repair`, then update again
the move to a new Fedora was refused	it says why (not settled, RPM Fusion not ready, a library missing, too little room) — nothing changed; the release you have stays supported until the date in the status
the bar shows no workspaces	`~/.cache/waybar.log`; `pgrep -f kawa-dwlstatus`
the bar, the terminal server or the clipboard keeper vanished	they start again by themselves within a second; if one keeps failing, a notice says so once — `~/.cache/dwl.log`. mako is started again by the next notification
a theme or wallpaper change has no transition	`ls ~/.local/bin/kawa-veil`; re-run **Theme** (it builds it; the build log is in the theme log)
a copy is gone after closing its program	`pgrep kawa-clipkeep`; re-run **Theme**; Super+V still has it
an upload button does nothing	`verify` (the portal checks); then restart Brave — it looks for the file-dialog portal when it starts
no music meter	`cava -v`; the meter appears only while an MPRIS player plays
brightness does nothing	switch **DDC/CI on** in the monitor's own menu, then try Super+]
Brave not in the desktop's colours	it was open while Theme ran: close Brave, run **Theme** (or Settings → Appearance → Theme → GTK)
torrra says Prowlarr has no indexers	type `prowlarr` in a terminal (it starts Prowlarr and opens its page) → Indexers → Add Indexer (§12)
the boot menu has the wrong size	re-run **Theme** after changing monitors
a wallpaper-pack picture is missing	`kawa-wallpaper --pack` when online
SELinux denial	`sudo ausearch -m avc -ts recent`; never disable SELinux
a config change didn't apply	`fedokawai drift`; foot fonts and dwl changes apply at the next login
want an old config back	`~/.local/state/fedokawai/backup/<time>/`

Known issue	Status
foot does not reorder right-to-left text	Arabic is complete in GTK/Qt programs and Brave; in the terminal the letters show but are not reordered — a foot limitation
Steam's interface	not themeable
Brave's own pages	settings pages, the Shields panel, about:blank and error pages keep Brave's own greys; private windows keep its private colours
sharing a screen with system sound	carries the call's own sound too (an echo for the others); use a tab's sound or the Share output instead (§4.10)

10. Where everything lives

Every path this desktop uses, in three tables: your own files, each program's files, and what the scripts put on the system. Settings that Theme manages are replaced at its next run if edited in place (§7.1); data and caches belong to the programs.

10.1 Your files

Path	What	Notes
`~/Documents/` · `~/Pictures/` · `~/Videos/` · `~/Music/`	what you keep	the standard folders: file dialogs and yazi know them
`~/Downloads/`	arrivals: Brave, torrra and yt-dlp save here	its own Btrfs subvolume, outside every snapshot (§7.2)
`~/Books/`	your library — the books window lists it (Super+R, §4.11)	folders inside are fine
`~/Notes/notes.md`	your notes (Super+N, §4.12)	plain Markdown
`~/Pictures/Screenshots/`	Print, Shift+Print, satty's Ctrl+S, mpv's `s`	
`~/Pictures/Wallpapers/`	your own wallpapers, on both themes	
`~/repos/`	your git repositories	
`~/.config/fedokawai/secrets.sh`	API keys, as `export NAME=value` (and Prowlarr's, deliberately without `export`)	yours, mode 600; nothing else holds a key

10.2 Each program's files

Program	Settings	Data	Cache, state, logs
the desktop (dwl, bar, menus)	`~/.config/fedokawai/` (`env`, `themes/`, `current`, `icons.sh`, `keybinds.txt`, `cava.conf`, `fzf.opts` — the look of every list) · `~/.config/{waybar,mako,fuzzel,dwl}/` · dwl's `~/.local/src/dwl/config.h`	`~/.local/share/fedokawai/` (pixel art, `wallpapers.list`, the pack in `wallpapers/`, `upstream/`)	`~/.cache/dwl.log`, `waybar.log`, `fedokawai-theme.log`, `kawa-wallpaper/` · `~/.local/state/fedokawai/` (each theme's wallpaper, stamps, `backup/`, `lazy-lock/`)
upkeep (Super+U)	`~/.config/fedokawai/upkeep.conf` (the season) · `~/.config/systemd/user/fedokawai-remind.{timer,service}` (the reminder)	—	`~/.cache/fedokawai/` (when the next Fedora came out, the day of the last reminder)
foot and bash	`~/.config/foot/foot.ini` · `~/.bashrc.d/fedokawai.sh` · `~/.inputrc`	`~/.bash_history` · zoxide's folders: `~/.local/share/zoxide/`	`~/.cache/fedokawai/` (fzf's and zoxide's shell code)
the clipboard	—	`~/.cache/cliphist/db` (the history)	
books and notes	—	`~/Books/` · `~/Notes/`	`~/.cache/kawa-books/` (covers, page counts)
neovim	`~/.config/nvim/` (`lua/config/lazy.lua`, `lua/plugins/colorscheme.lua` from Theme; the rest is LazyVim's starter, yours)	`~/.local/share/nvim/` (plugins in `lazy/`, Mason's tools in `mason/`) · `~/.config/nvim/lazy-lock.json` (plugin versions)	`~/.local/state/nvim/` (undo history, swap files, logs)
lazygit	`~/.config/lazygit/config.yml`	—	`~/.local/state/lazygit/`

Program	Settings	Data	Cache, state, logs
yazi (files and dialogs)	`~/.config/yazi/theme.toml`, `init.lua` · the dialog's keys: `~/.config/fedokawai/yazi-pick/` · `~/.config/xdg-desktop-portal-termfilechooser/config`	the trash: `~/.local/share/Trash/`	`~/.local/state/yazi/`
zathura	`~/.config/zathura/zathurarc`	`~/.local/share/zathura/bookmarks.sqlite` (each book's page, bookmarks)	
mpv and yt-dlp	`~/.config/mpv/` (`mpv.conf`, `scripts/`) · `~/.config/yt-dlp/config`	—	`~/.local/state/mpv/` (where each video stopped)
torrra (torrents)	`~/.config/torrra/config.toml` — Prowlarr's address, the download folder, colours; never the key	the program: `~/.local/share/pipx/venvs/torrra/` · downloads: `~/Downloads/`	
Prowlarr	`~/.config/containers/systemd/prowlarr.container` (how it runs) · its key: `secrets.sh`	`~/.config/prowlarr/` (its database: indexers, login, history) · the image: `~/.local/share/containers/`	`journalctl --user -u prowlarr`
Brave Origin	`~/.config/BraveSoftware/Brave-Origin/` (profile, settings, extensions)	the same folder	`~/.cache/BraveSoftware/`
spotify_player	`~/.config/spotify-player/` (`app.toml`, `theme.toml`)		`~/.cache/spotify-player/` (login, cover art)
btop	`~/.config/btop/btop.conf`		
tldr	its colours and refresh in `~/.bashrc.d/fedokawai.sh`		`~/.cache/tldr/` (the pages)
Steam and games		`~/.local/share/Steam/` · GE-Proton in `compatibilitytools.d/` there	
GTK, Qt, pointers	`~/.config/gtk-4.0/` · `~/.config/qt6ct/` · `/etc/dconf`	`~/.local/share/themes/Fedokawai*` · `~/.local/share/icons/Fedokawai/` (pointers)	

10.3 On the system

Path	What
`/usr/local/lib/fedokawai/`	the toolkit: this project, root's own copy, run by every update, repair and move (§2)
`/usr/local/bin/`	`fedokawai` (the control, Super+U), `dwl` and `dwl.prev` (the build before it), `start-dwl`, `kawa-greeter`, `kawa-torrra`
`/usr/local/lib/fedokawai-wlroots-0.20/`	dwl's own copy of wlroots, kept from Fedora's build (§6.5)
`/usr/local/sbin/`	`kawa-snap` (the DNF snapshots)
`/usr/local/libexec/xdg-desktop-portal-termfilechooser` (+ its service files under `/usr/local/lib` and `/usr/local/share`)	the yazi file dialog's portal, built by setup

Path	What
`/etc/dnf/libdnf5.conf.d/90-fedokawai.conf` · `/etc/dnf/libdnf5-plugins/actions.d/`	DNF's settings · the snapshot around every DNF run
`/etc/tuned/profiles/fedokawai/` · `/etc/NetworkManager/conf.d/90-fedokawai.conf`	the performance profile · Wi-Fi power saving off
`/etc/dconf/db/local.d/00-fedokawai`	GTK's defaults: theme, icons, pointers, fonts
`/etc/greetd/fedokawai.toml` · `/etc/systemd/system/greetd.service.d/fedokawai.conf`	the login: agreety on tty1 · greetd pointed at that file
`/etc/snapper/configs/` · `/.snapshots`	the snapshot settings · the snapshots
`/boot/grub2/themes/sumi/`	the boot menu theme
`/var/lib/fedokawai/` · `/var/log/fedokawai.log` · `/var/log/fedokawai-{check,verify}.log`	setup's progress and the records the menu reads (`last-update`, `last-verify`, `last-upgrade`, `release-N`, `snapshots`, and what to look at after a move: `unavailable`, `rpmnew`) · the log · the check and verify logs (root's alone: `fedokawai log`)

Everything system-wide goes into a drop-in folder (`*.d/`) or under `/usr/local`, never into a file Fedora's packages own — so a Fedora update never fights these settings, and removing one of them removes it cleanly.

11. Using the software

Chapter 4 covers the desktop itself: the bar, the menus, the keys, the themes. This chapter covers the programs you use on it, one at a time: what each is for here, how you open it, the handful of things worth knowing, and where it trips people up. Keys are given as this desktop configures them; where a program keeps its own defaults, they are the defaults of the version Fedora 44 ships, or of the release `update` fetches.

Who owns which config

Every file under `configs/home/` in the toolkit (§2) belongs to **Theme**. Edit it there and run `fedokawai theme`; an edit made directly in `~/.config` is listed by `fedokawai drift` and replaced at the next run (the old copy goes to `~/.local/state/fedokawai/backup/`). Files the toolkit does not ship are yours and are never touched.

Managed by Theme (edit in the repository)	Yours (edit in place)
`foot/foot.ini`, `fuzzel/fuzzel.ini`, `mako/config`, `waybar/`, `btop/btop.conf`, `lazygit/config.yml`, `bat/config`, `mpv/mpv.conf` and `mpv/scripts/kawa-*.lua`, `yt-dlp/config`, `zathura/zathurarc`, `satty/config.toml`, `spotify-player/`, `yazi/theme.toml` and `yazi/init.lua`, `nvim/lua/config/lazy.lua`, `nvim/lua/plugins/colorscheme.lua`, `nvim/after/ftplugin/sh.lua`, `MangoHud/MangoHud.conf`, `mimeapps.list`, `xdg-terminals.list`, the portal files (`xdg-desktop-portal/`, `xdg-desktop-portal-wlr/`, `xdg-desktop-portal-termfilechooser/`), the pointers in `~/.local/share/icons/`, `pipewire/pipewire.conf.d/60-share.conf`, `~/.bashrc.d/fedokawai.sh`, `~/.inputrc`, `dwl/autostart.sh`, the launcher entries in `~/.local/share/applications`	every other file in `~/.config/nvim`, `yazi/yazi.toml`, `yazi/keymap.toml`, `mpv/input.conf`, `cava/config`, `~/.config/fedokawai/secrets.sh`, `~/.config/torrra/config.toml`, the browser and Steam profiles (except the two Brave settings in §2)

11.1 foot — the terminal

Super+Return opens a terminal. Each one is a window of the foot server that starts at login, which is why it appears at once. **Ctrl+Shift+N** inside a terminal opens another in the same folder. The prompt and its marks are described in §4.6.

Keys	Does
Ctrl+Shift+C · Ctrl+Shift+V	copy · paste. Selecting text with the mouse already copies it
Shift+PageUp / PageDown	scroll back through the last 20 000 lines (also the mouse wheel)
Ctrl+Shift+R	search the scrollback: type, then Ctrl+R for the older match, Ctrl+S for the newer; Ctrl+W widens the match by a word; Enter ends the search with the match selected; Esc cancels
Ctrl+Shift+O	URL mode: every link on screen gets a short label; type the label to open it in Brave; Esc leaves
Ctrl+Shift+Y	copy mode: every commit hash, IP address and file path on screen gets a label; type it and that text is on the clipboard — no careful selecting
Ctrl+Shift+Z / X	jump to the previous / next prompt
Ctrl+Shift+G	the last command's output in neovim (§4.6)
Ctrl+Shift+U	type a character by its Unicode number
Ctrl++ · Ctrl+- · Ctrl+0	larger · smaller · normal text (Ctrl+wheel too)
double / triple click	select a word / the quoted text or the whole line

Recommended. A terminal started as `footclient -a float` opens floating and centred, like the network window from quick settings, btop from the bar and the clipboard history; good for a quick look at something without disturbing the tiling. In programs that use the mouse themselves (btop, yazi, lazygit), hold **Shift** while dragging to select text anyway.

Tip: to keep a long output, press Ctrl+Shift+G and save it from neovim with `:w ~/Documents/output.txt`.

A notice when a long command ends

A command that runs 10 seconds or more sends a notice when it finishes — *done in 2m14s*, or *failed (1) after 40s*, with the folder it ran in — but only while its terminal is not the window you are looking at. Start an upgrade, go elsewhere, and hear when it is done.

Safe clipboard

Programs in the terminal may *put* text on the clipboard (neovim over SSH does that with OSC 52), but never *read* it: a file you `cat` or a server you `ssh` into cannot see what you copied (`[security] osc52=copy-enabled`).

Pitfalls.

- Every selection replaces the clipboard (`selection-target=clipboard`) and lands in the clipboard history (Super+V). Middle-click pastes the *primary* selection, which foot itself no longer fills; use Ctrl+Shift+V.
- All Super+Return windows live in one server process. If it ever stops, they all close together; a new server starts within a second, but the windows do not come back. For something that must not be cut off — a long copy, an upgrade — start the plain **Foot** entry from the launcher, which runs as a process of its own.
- Changes to the font or other `[main]` settings reach new windows only after the server restarts, which in practice means the next login (§9).
- Right-to-left text is shown but not reordered (§9, known issues).

11.2 The shell

Bash, with Fedora's own `~/.bashrc` reading `~/.bashrc.d/fedokawai.sh` (should a `~/.bashrc` not read that folder, Theme adds Fedora's few lines for it). The prompt, the OSC 7 / 133 marks and the history size are in §4.6; this is the rest of what the file and `~/.inputrc` set. A handful of small tools make it quicker, each doing one job:

Tool	Job	Replaces
fzf	find by typing a few letters of anything: history, files, folders	scrolling, `Ctrl+R` 's one-match-at-a-time search, long paths
zoxide	`cd` remembers the folders you use and jumps to them by a word	typing `cd ../../repos/project`
eza	`ls` , with folders first, sizes you can read, each file's git state, trees	`ls -la` , `tree`
tldr	the five commands you actually wanted, with examples	reading a whole man page
bat, **ripgrep**, **fd**	reading a file, searching inside files, finding files (§11.7)	`cat` , `grep -r` , `find`

Keys or input	Does
Up / Down	earlier / later commands that start with what you have already typed
Ctrl+R	search the whole history in fzf: type any part of a command, Enter puts it on the line
Ctrl+T	pick a file or folder below here (previewed: a file's start, a folder's tree) and insert its path where you are typing
Alt+C	pick a folder below here and go into it
`nvim **` , then Tab	fuzzy completion for any command: `**` + Tab opens fzf for that argument
`cd proj`	jumps to the folder you use most whose name matches *proj*, from anywhere (`~/repos/project`); `cd re pro` matches both words in order; a real path still works exactly as always
`cdi`	the folders zoxide knows, in fzf
Tab	complete, ignoring case; one press lists every choice when there are several
Ctrl+Left / Right	move by word
Ctrl+Backspace · Ctrl+Delete	delete a word backwards · forwards

Keys or input	Does
Alt+.	insert the last argument of the previous command
a folder name on its own	changes into it (`autocd`)
`**` in a path	any depth of folders: `ls **/*.pdf` (`globstar`)

Alias or setting	Does
`ls` · `ll` · `lt`	eza: names, folders first · everything, with sizes, dates (`2026-10-15 13:04`) and each file's git state (`N` new, `M` modified) · a tree, two levels deep
`tldr tar`	examples for `tar` (any command); the pages are fetched on first use and refreshed at most once a quarter, when you ask
`update` · `verify` · `repair`	§6.1
`snaps`	the snapshot list (§11.21)
`dwl-log`	the compositor's log, opened at the end
`torrra` · `prowlarr`	torrra through its key wrapper · Prowlarr's web page, started first (`prowlarr stop` when done) (§12)
`ssh`	always reaches the server as `xterm-256color` : few servers know foot's own terminal type
`history`	shows the date and time of each command
`EDITOR` , `VISUAL` , `vi`	all neovim; `man` pages open in neovim too (`q` closes; `gO` lists the sections)
`less`	colours kept, mouse wheel works, searches ignore case unless you type a capital

Fast to start

Every helper's shell code is written once to `~/.cache/fedokawai/` and read from there: a new terminal starts no extra program, and the prompt itself runs none outside a git repository. zoxide learns a folder only when you arrive in it — one quick call then, nothing on any other prompt. (`time bash -ic exit` shows the cost: a few hundredths of a second.) After an `update` changes one of the tools, its cached code is written again by itself.

History rules

A command typed with a **leading space** is not saved: use it for anything with a password or token in it. Bare `ls` , `ll` , `cd` , `cd -` , `pwd` , `exit` , `clear` and `history` are not saved either, and a repeated command keeps only its newest copy.

Secrets

API keys go in `~/.config/fedokawai/secrets.sh` (mode 600), which every new shell reads — and only while the file is yours alone: a copy others could read is not loaded, and the shell says so. Write them as `export NAME=value` — without `export` the programs you start never see them. Setup keeps Prowlarr's key there too (§12), deliberately without `export` : only the torrent search reads it, so no other program carries it. Nothing in the repository holds a key.

Pitfalls. `cdspell` quietly corrects small typos in `cd` arguments and prints the folder it chose; check it before running something destructive there. Outside foot — the text consoles (Ctrl+Alt+F2…F6), a login from another machine — the prompt uses plain characters, because only foot has the icon font.

11.3 fuzzel — the launcher



The launcher (Super+Space): Papirus icons and names, on both themes.

Super+Space, or the torii key at the left of the bar. Type part of a name, Enter starts it, Esc closes.

- It matches application **names only**: *brave* finds Brave Origin, *browser* finds nothing.
- The programs you start most often move to the top (the counts live in `~/.cache/fuzzel`).
- Terminal programs with a launcher entry, such as Neovim and btop, open in a terminal window. **Files** (yazi) and **Torrents** (torrra) are entries of this desktop's own.
- The other tools installed as release binaries — presenterm, spotify_player, satty — have no launcher entry; start them in a terminal.

11.4 yazi — files



yazi: the parent folder, the current folder, and a preview of the file under the cursor.

Super+E opens yazi in its own terminal window, starting in your home folder; `q` quits and closes the window. Typed in a terminal, `yazi` starts in the folder you are in. Three columns: the parent folder, the current one, a preview of the file under the cursor. Its colours are the terminal's, so it switches with the theme. Press **~** or **F1** for the complete key list of the installed version.

Keys	Does
h j k l (or arrows)	parent folder · down · up · open folder
gg · G · H · L	top · bottom · back · forward through visited folders
g h · g d · g c · g t	home · Downloads · `~/.config` · the trash
g Space	type a path and jump there
Enter · O	open · choose what to open it with
Space · v · Ctrl+A	select a file · select a range · select all
y · x · p · P	copy · cut · paste · paste, overwriting
d · D	move to the trash · **delete for good**
a	create a file; end the name with `/` for a folder
r	rename; with several files selected, rename them all in neovim
.	show / hide hidden files
s · S	search file names (fd) · file contents (ripgrep) below this folder
f · / · n · N	filter the list as you type · find · next · previous match

Keys	Does
z · Z	jump to any folder or file below this one by typing part of its name (fzf) · jump to a folder you often use, from anywhere, by a word (zoxide — the same list as the shell's `cd`)
c c · c f · c d	copy the file's full path · its name · the folder's path
t t · 1…9 · [ ]	new tab · go to a tab · previous / next tab
Tab	details of the file (type, size, picture dimensions)
, then a letter	sort: `m` modified, `s` size, `n` natural, `a` alphabetical (capital = reversed)
K · J	scroll the preview
; · :	run a shell command in the background · run one in this window (yazi returns when it ends)
w	running tasks (copies, moves)

Opening

Enter uses the first opener for the type: text and JSON open in neovim inside the same window (`:q` brings you back to yazi); pictures, PDFs, videos and the rest go to `xdg-open`, which hands them to the default program for that type (§4.9). **O** lists every choice for the hovered file.

Previews that work here: text and code (in the terminal's colours), JSON (jq), pictures (drawn in foot as Sixel), the first page of PDFs (pdftoppm), video stills (ffmpeg), fonts and HEIC/AVIF/JXL pictures (ImageMagick), and the contents of archives (7-Zip). SVG pictures show no preview, because resvg is not packaged for Fedora.



yazi's preview column showing a picture, drawn in the terminal as Sixel by foot.

Bulk rename

Select the files (Space or v), press **r**: neovim opens with one name per line. Change the lines, save and quit (`:wq`), and yazi asks before renaming. Keep the lines in the same order and do not add or remove any.

Trash

d moves files to the desktop trash (`~/.local/share/Trash`). **g t** opens it; there, **O** on a file offers *Restore selected files* and *Empty trash bin*. **D** skips the trash entirely.

Tip: files moved with **d** still take their space until the trash is emptied; make it part of the seasonal routine (§7.5).

Commands on files

In `;` and `:` commands, `%h` stands for the hovered file and `%s` for the selected ones, already quoted: `;` then `7z x %h` unpacks an archive here; `:` then `du -sh %s; read` shows the sizes until you press Enter (without the `read`, yazi comes back before you can read them).

Recommended: yazi, neovim and lazygit together. Work in one terminal: browse in yazi, press Enter on a file to edit it in neovim, `:q` to return; when the work is done, `:` then `lazygit` to stage and commit, and `q` in lazygit brings you back to yazi. Uploading a file in Brave opens yazi too, as the file dialog (§4.9, §13.1).

Pitfalls.

- Input boxes (rename, create, filter) work like a tiny vi: Esc first leaves typing, a second Esc — or Ctrl+C — cancels.
- Two files are managed here: `theme.toml` (colours, the file-kind icons — the desktop's own pixel icons) and `init.lua` (the frame: one free column at each edge, as every popup has, and the file dialog's title rule and key line). Your own keys and openers go in `~/.config/yazi/keymap.toml` and `yazi.toml`, which you create. yazi itself is held at the version these files are written for (§6.3).
- Search results (s, S) are a temporary list; Esc leaves it, Ctrl+S stops a search still running.

11.5 neovim and LazyVim — the editor



Neovim with LazyVim, the palette's gruvbox, on a shell script.

Neovim is the only editor here (`nvim`, `vi`, `$EDITOR`, the man pages, yazi's Enter). It runs the **LazyVim** starter that setup copied into `~/.config/nvim`; the repository adds only `lua/config/lazy.lua` (the starter's own, with the background check for new

plugin versions switched off, so plugins move only with `update`), `lua/plugins/colorscheme.lua`, the remapped gruvbox that follows the dark and light theme by itself (§5.4), and `after/ftplugin/sh.lua`, which keeps shell scripts from being reformatted on save (*Formatting*, below). Started without a file, it shows a start screen with one-key actions: `f` find file, `n` new file, `g` find text, `r` recent, `c` config, `s` restore the last session in this folder, `x` extras, `l` plugins, `q` quit.

Neovim differs from other editors in one way that matters more than all the others: most of the time, the keys you press are **commands**, not text. That is what makes it fast once learned, and baffling for the first ten minutes. This section starts from nothing — how to stay in control, a first hour of practice, the small grammar everything else is built from — and then covers what LazyVim adds. Read it with neovim open beside it.

Modes

Neovim is always in one *mode*. Its name stands in the box at the left end of the status line, each in a colour of its own. Esc leaves every other mode for Normal.

Mode	The box	You are	Get there with
Normal	NORMAL, vermilion	giving commands: moving, deleting, copying. Neovim starts here	Esc
Insert	INSERT, asagi	typing text, as in any editor	`i`, `a`, `o` and a few others (below)
Visual	VISUAL, yamabuki	selecting text, to act on it	`v`, `V`, Ctrl+V
Command-line	COMMAND, blue	typing a command after `:`, or a search after `/`; Enter runs it, Esc drops it	`:` or `/`
Replace	REPLACE, beni	typing over the existing text	`R`
Terminal	TERMINAL, asagi	in a shell inside neovim; Esc Esc, quickly, stops typing into it	Ctrl+/ (again to hide it)

`:` opens a small box near the top of the window with suggestions under it (Tab completes); a search with `/` stays at the bottom.

Never stuck

Whatever happened, one of these gets you out of it:

Situation	Do
you do not know which mode you are in, or keys do odd things	**Esc** — twice if unsure. You are in Normal mode (Ctrl+[is the same key)
save and quit	`:wq` Enter, or `ZZ`. Ctrl+S saves, also while typing
quit without saving	`:q!` Enter; `:qa!` closes every file. Plain `:q` on a changed file asks *Save changes?* — Yes, No or Cancel
undo · redo	`u` · Ctrl+R, as many times as needed
throw away everything since the last save	`:e!` Enter
the status line says *recording @q*	you pressed `q` and a letter: press `q` once more
a window of old commands opened at the bottom	you typed `q:`: Ctrl+C, or `:q` Enter
letters come out in Arabic, or Normal-mode keys do nothing	the keyboard layout changed (the bar shows AR): Alt+Shift. The desktop's shortcuts always read the US layout; neovim reads the letters you type
neovim vanished and the shell says *Stopped*	Ctrl+Z put it to sleep: type `fg`
Found a swap file on opening a file	a neovim crashed while editing it: `R` recovers the unsaved text — save it, and answer `D` (delete the old swap file) the next time the question comes. (A file open in another running neovim opens anyway, with the warning *W325*: save it in one place only)

Situation	Do
readonly or *Permission denied* on a system file	`:q!`, then `sudoedit /etc/the-file` in the shell: your own neovim edits a copy, and sudo puts it in place (§7.1)

Tip: undo survives closing the file — LazyVim keeps the undo history on disk, so `u` the next morning still takes back yesterday's edits. Space s u lists the whole history to pick a point from.

The first hour

Practise on a long file you cannot damage; a copy of this handbook's source does well:

```
cp ~/fedokawai/docs/fedokawai-guide.md /tmp/practice.txt
nvim /tmp/practice.txt
```

Spend ten minutes on each block below, in order, and do every key at least five times. Plain text files wrap long lines and underline spelling mistakes in red (Space u s hides them).

Moving. Arrows work, but h j k l sit under your right hand, and the hand never leaves the keys.

Keys	Move
`h` `j` `k` `l`	left · down · up · right
`w` · `b` · `e`	to the next word · back a word · to the end of the word
`W` · `B` · `E`	the same, counting only spaces as gaps (`~/.config/nvim` is one word)
`0` · `^` · `$`	start of the line · its first letter · its end
`gg` · `G` · `42G` (or `:42`)	first line · last line · line 42
Ctrl+D · Ctrl+U	half a screen down · up
`}` · `{`	to the next · previous blank line: a paragraph at a time
`%`	to the matching bracket
Ctrl+O · Ctrl+I	back · forward to where you were before a jump
`s`, a letter or two, a label	**flash**: every visible place with those letters gets a one-letter label; type it to land there

The numbers in the left margin count lines **from the cursor**: the line you are on shows its real number, every other line how far away it is. Any move takes a count in front — `5j` goes five lines down, and the 5 can be read off the margin.

Typing.

Keys	Start typing
`i` · `a`	before · after the cursor
`I` · `A`	at the start · at the end of the line
`o` · `O`	on a new line below · above
Esc	stop typing: back to Normal mode

Changing.

Keys	Do
`x`	delete the character under the cursor
`r`, then a character	replace one character
`dd` · `yy`	cut · copy (*yank*) the line
`p` · `P`	paste after · before the cursor; a whole line goes below · above
`J`	join the next line onto this one

Keys	Do
`~`	switch the case of a letter
`.`	**repeat the last change** — the most useful key in neovim
`u` · Ctrl+R	undo · redo

Searching. Searches ignore case unless you type a capital letter.

Keys	Do
`/word` Enter · `?word` Enter	search forward · backward
`n` · `N`	next · previous match
`*` · `#`	the word under the cursor, forward · backward
Esc	clear the highlighting

Replacing. While you type a replacement, the file shows the result live.

Command	Replaces
`:s/old/new/`	the first *old* on this line
`:s/old/new/g`	every *old* on this line
`:%s/old/new/g`	every *old* in the file (`%` means all lines)
`:%s/old/new/gc`	the same, asking at each: `y` yes, `n` no, `a` all the rest, `q` stop
`:%s/\<old\>/new/g`	whole words only — not the *old* in *golden*
`:'<,'>s/old/new/g`	in the selected lines: select, press `:`, and the `'<,'>` is filled in

Finally `:w` to save and `:q` to quit. Delete `/tmp/practice.txt` whenever you like.

The grammar: verb, count, motion

Most of neovim is three parts said in order: *what to do* (a verb — neovim calls it an operator), *how often* (a count, optional), and *over what* (a motion, or a text object). `d` deletes and `w` moves a word, so `dw` deletes a word, `d3w` three words, `d$` to the end of the line. Every verb works with every motion, so each new motion gives you a new command with every verb. A verb typed twice acts on the whole line.

Verb	Does	Twice: the whole line
`d`	delete (a cut: the text goes to the clipboard)	`dd`
`c`	change: delete, then type in its place	`cc`
`y`	yank: copy	`yy`
`>` · `<`	indent · outdent	`>>` · `<<`
`gu` · `gU` · `g~`	lower case · UPPER CASE · switch case	`guu` · `gUU` · `g~~`
`=`	re-indent	`==`
`gc`	comment out · back in	`gcc`

Any move from the first hour works as the motion — `w`, `b`, `e`, `0`, `$`, `G`, `gg`, `}`, a `/search` — and `j` or `k` make the verb take whole lines. Two more are made for verbs: `fx` reaches onto the next *x* on the way, `tx` up to just before it (any character in place of *x*; `F` and `T` look backwards). Text objects (next block) work as motions too.

You type	It does
`dw` · `d3w`	delete to the next word · three words
`D` (the same as `d$`)	delete to the end of the line

You type	It does
`dt)` · `df,`	delete up to the next `)` · up to and including the next comma
`5dd` · `dG`	delete five lines · everything to the end of the file
`cw` · `ciw`	change to the end of the word · the whole word, wherever the cursor is in it
`ci"` · `ci(`	change what is inside the quotes · the brackets
`yip` · `>ip` · `gcip`	copy · indent · comment out the paragraph
`gUiw`	the word in capitals
`d/end` Enter	delete up to the next *end*

The dot

`.` repeats the last change, verb and typed text together. To rename a word in a few chosen places: `*` to search for it, `ciw`, the new word, Esc; then `n` to the next match and `.` to change it too — or `n` again to skip one.

Tip: `f` and `t` are LazyVim's (flash): they look past the end of the line too, highlight every possible target, and pressing `f` again moves on to the next one.

Text objects

A text object names a piece of text by what it is rather than where it ends, so it works wherever the cursor sits inside it. `i` takes the inside, `a` takes it *around*, with its surroundings: `iw` is a word, `aw` the word and the space after it. They follow a verb, or `v` to select:

Object	`i` (inside)	`a` (around)
`w` · `W`	the word	the word and a space
`s` · `p`	the sentence · the paragraph	with the space · the blank line after it
`"` · `'`	inside the quotes	with the quotes
`(` `[` `{` `<`, or `b` for the nearest	inside the brackets	with the brackets
`q`	inside the nearest quotes of any kind	with them
`t`	inside an HTML or XML tag	with the tags
`a`	an argument between commas	with its comma
`f` · `c`	the body of a function · a class (in code neovim has a parser for)	the whole function · class
`g`	the whole file	the whole file

`daw` deletes a word and its space; `yi"` copies a quoted path without its quotes; `dap` deletes a paragraph; `yag` copies the whole file. LazyVim's mini.ai adds `q`, `a`, `f`, `c` and `g`, and makes all of them look ahead: `ci"` works from the start of the line, before the quotes.

Surround comes with an extra, `coding.mini-surround` (switch it on in `:LazyExtras`, below). Then `gsaiw"` puts quotes round a word — `gsa`, an object, the character — and `gsa` works on a selection too; `gsd"` removes the quotes round the cursor; `gsr"'` turns double quotes into single ones.

Visual mode and block mode

Select first, then act — the way to go when the piece has no name.

Keys	Does
`v` · `V` · Ctrl+V	select characters · whole lines · a rectangle (a block)

Keys	Does
any move or object	extends the selection: `vip` a paragraph, `v$` to the end of the line
`o` · `gv`	jump to the other end · select the last selection again
`d` · `y` · `c`	delete · copy · change the selection
`>` · `<`	indent · outdent; the selection stays, so press again for more
`u` · `U` · `~`	lower case · upper case · switch case
`J` · `gc`	join the lines · comment them out or back in
Alt+J · Alt+K	move the selected lines down · up (a single line too, in Normal mode)
`p` · `P`	replace the selection with the clipboard; `P` leaves the clipboard as it was
Ctrl+A · `g` Ctrl+A	add one to every number · number them 1, 2, 3 … down the selection
`:`	a command for the selected lines (`:'<,'>` appears)

Block mode selects a rectangle, which is how columns are edited. Press Ctrl+V, then move down (`j` , or `3j`) and across. With a block selected:

- `I` , the text, Esc — the text goes in at the left edge of the block on **every** line. It shows on the first line while you type and on the others when you press Esc: Ctrl+V, `3j` , `I` , then `- [ ]` and a space, then Esc turns four lines into a checklist.
- `$` , then `A` , the text, Esc — appended at the end of every line, whatever its length.
- `c` replaces the block's contents on every line; `d` deletes the column; `r` and a character fills it.

LazyVim lets the cursor go past the end of short lines in block mode, so a block can be as wide as the longest line. (foot's own paste is Ctrl+Shift+V, so Ctrl+V reaches neovim.)

Registers and the clipboard

Neovim keeps several clipboards, called *registers*, each named by one character. To use one, type `"` and its name before the command: `"ayy` copies the line into register `a` , `"ap` pastes it back. Typing `"` alone — or Ctrl+R in Insert mode — shows a popup of what each register holds; Space s " lists them in the picker.

Here, the plain commands use the desktop's clipboard. LazyVim sets `clipboard=unnamedplus` (except in an SSH session), so `y` , `d` , `c` , `x` and `p` all go through the system clipboard: a line yanked in neovim pastes into Brave with Ctrl+V, a link copied in Brave pastes in neovim with `p` , and every yank lands in the clipboard history (Super+V, §4.8) — and, thanks to `kawa-clipkeep` , stays on the clipboard after neovim closes. `"+y` and `"+p` , the way plain Vim reaches the clipboard, work too and do exactly the same.

Register	Holds
`"` (unnamed) and `+`	the last yank or delete — here, the system clipboard
`0`	the last **yank** only; deletes leave it alone, so `"0p` pastes what you copied even after deleting since
`1` … `9`	the last nine deletions of a line or more, newest first
`a` … `z`	your own: `"ay` stores, `"Ay` (the capital) adds to the end
`_`	the black hole: `"_dd` deletes a line without touching the clipboard
`*`	the primary selection (select, then middle-click)
`/` · `:` · `.` · `%`	the last search · the last command · the last text typed · this file's name

Tip: since deletes go to the clipboard, a `dd` replaces the link you copied in Brave a minute ago. It is still in Super+V; to keep it on the clipboard, delete with `"_d` , or paste your last yank with `"0p` .

Macros

A macro is a recording of keys that neovim plays back: any edit you can do once by hand, it does a hundred times. The recording goes into a register, like a copy.

Keys	Does
`qa`	start recording into register `a` (any letter); the status line shows *recording @a*
`q`	stop recording
`@a` · ``	play it once · play the last played macro again
`10@a`	play it ten times in a row
`qA` … `q`	record more onto the end of `a`
`:5,20normal @a`	play it once on each of lines 5 to 20
`:'<,'>normal @a`	play it once on each selected line
`:g/pattern/normal @a`	play it once on each line containing *pattern*

Three habits make a macro reliable:

- **Start from a known place.** Begin with a move that does not depend on where the cursor happened to be — `0` or `^` — and end by moving to where the next run should start, usually `j`, the next line.
- **Move by meaning, not by counting.** `t.`, `f,`, `w`, `/word` find their place on lines of any length; `llll` works only on lines exactly like the first.
- **A macro stops at the first command that fails.** On the last line `j` fails, and playback ends there — so `100@a` simply means "to the end".

A worked example: numbering photos. Select some pictures in yazi and press **r** (bulk rename, §11.4): neovim opens with one name per line.

```
IMG_20261003_101512.jpg
IMG_20261003_101530.jpg
IMG_20261003_102244.jpg
IMG_20261003_103901.jpg
```

They should become `beach_01.jpg`, `beach_02.jpg` and so on.

1. Do the first line by hand: `gg`, then `0ct.` (change everything up to the dot), type `beach_01`, Esc. It reads `beach_01.jpg`.
2. Record how to get from one line to the next. On line 1, `qa` starts the recording, then: `0yt.` copies `beach_01` (everything before the dot); `j0ct.` goes down and changes that line up to its dot; **Ctrl+R** then `0` types the text just copied; Esc; **Ctrl+A** adds one to the number under the cursor; `q` stops. Line 2 reads `beach_02.jpg`.
3. `100@a` does the rest: each run copies the line it is on, moves down, writes the name and counts up, until the last line stops it.
4. `:wq` — and yazi asks once more before renaming.

Ctrl+A keeps the leading zero (`beach_09` becomes `beach_10`). It reads a `-` just before a number as a minus sign, which is why the example uses `_`: from `beach-01`, Ctrl+A makes `beach00`.

Editing a macro. A macro is only text in a register, so it can be corrected instead of recorded again. On an empty line, `"ap` pastes it, with Esc shown as `^[` and Ctrl+A as `^A`. Edit it like any text — to type a special key into it, press Ctrl+V first in Insert mode, then the key — then `0"ay$` copies the line back into `a` (`$` rather than `yy`, so no line break comes along) and `dd` removes the line.

Without a recording: `:normal` and `:g`. `:normal` runs Normal-mode keys on every line of a range, as if you typed them there with the cursor at the start of each — often all a macro would have done. Insert mode ends by itself at the end of each line's keys.

Command	Does
`:%normal A;`	adds `;` to the end of every line
`:'<,'>normal I#`	puts `#` at the start of each selected line

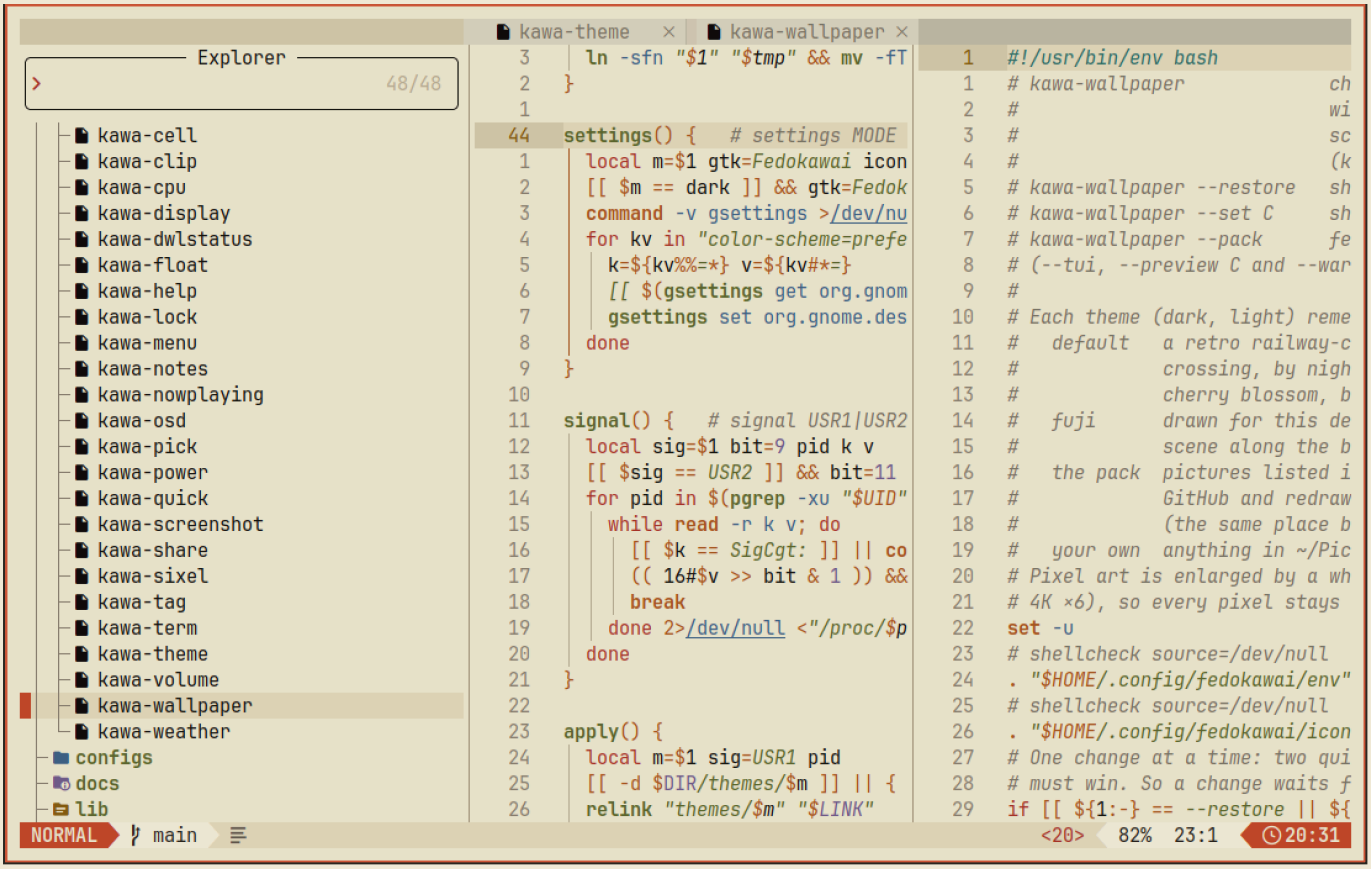
Command	Does
`:g/^TODO /normal 0cw- [ ]`	turns every line starting `TODO` into a checklist item, `- [ ] …`
`:g/^$/d`	deletes every empty line
`:v/keep/d`	deletes every line that does **not** contain *keep*
`:sort` · `:sort u`	sorts the lines of the file (or of a selection) · and drops repeated ones

`:g/pattern/` picks lines by what they contain and runs the command after it on each — `d` to delete, `normal` and any keys, `normal @a` a macro; `:v` picks the lines that do not match.

Files, buffers and windows

Every file you open becomes a *buffer* and stays open until you close it; once there are two or more, LazyVim lists them along the top. A *window* is a view onto a buffer: splitting the screen shows two files side by side, or two places in one.

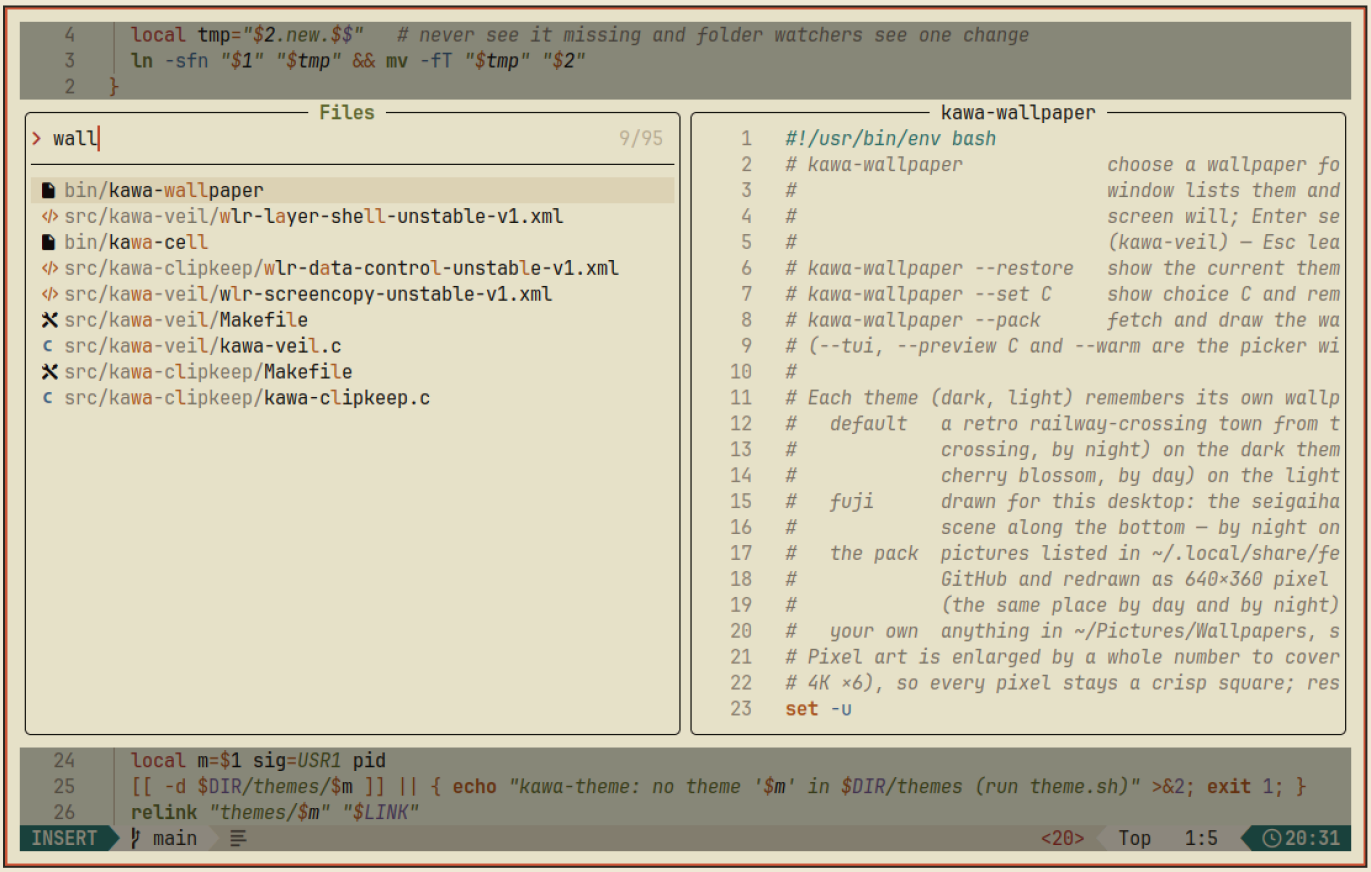
Keys	Does
Space Space · Space f r	find a file in the project · a recent file
Space , · Space b b	the open buffers · back to the one before
Shift+H · Shift+L	previous · next buffer
Space b d · Space b o	close this buffer · close all the others
`:e path`	open a file by its path; Tab completes
Space - · Space |	split: a new window below · to the right
Ctrl+H · J · K · L	to the window on the left · below · above · on the right
Ctrl+↑ · ↓ · ← · →	make the window taller · shorter · narrower · wider
Space w d · Space w m	close the window (the file stays open) · zoom it to the whole screen and back
Ctrl+W `=`	make all windows the same size
Space Tab Tab · Space Tab d	a new tab, a whole new layout of windows · close it



Two windows side by side, two shell scripts, the file explorer on the left and the open buffers listed along the top.

Finding

Space Space opens the picker: a line to type in, the matches under it, a preview of the file beside them. It matches loosely — `brc` finds `bashrc`. The same window serves every search in LazyVim (files, text, buffers, help, keys), with the same keys:



The file picker (Space Space): the typed letters, the matching files, a preview of the one selected.

Keys in a picker	Does
typing	filters the list
↑ ↓ or Ctrl+J · Ctrl+K	move in the list
Enter	open
Ctrl+V · Ctrl+S · Ctrl+T	open in a window to the right · below · in a new tab
Tab	mark several; Enter then opens them all
Alt+H · Alt+I	include hidden files (`.bashrc`, a dotfiles folder) · files git ignores
Ctrl+Q	put every result in the quickfix list (*Search and replace*, below)
Alt+P · Ctrl+F · Ctrl+B	preview on or off · scroll it down · up
Esc Esc, or Ctrl+C	close. The first Esc only stops the typing; `?` then lists every key

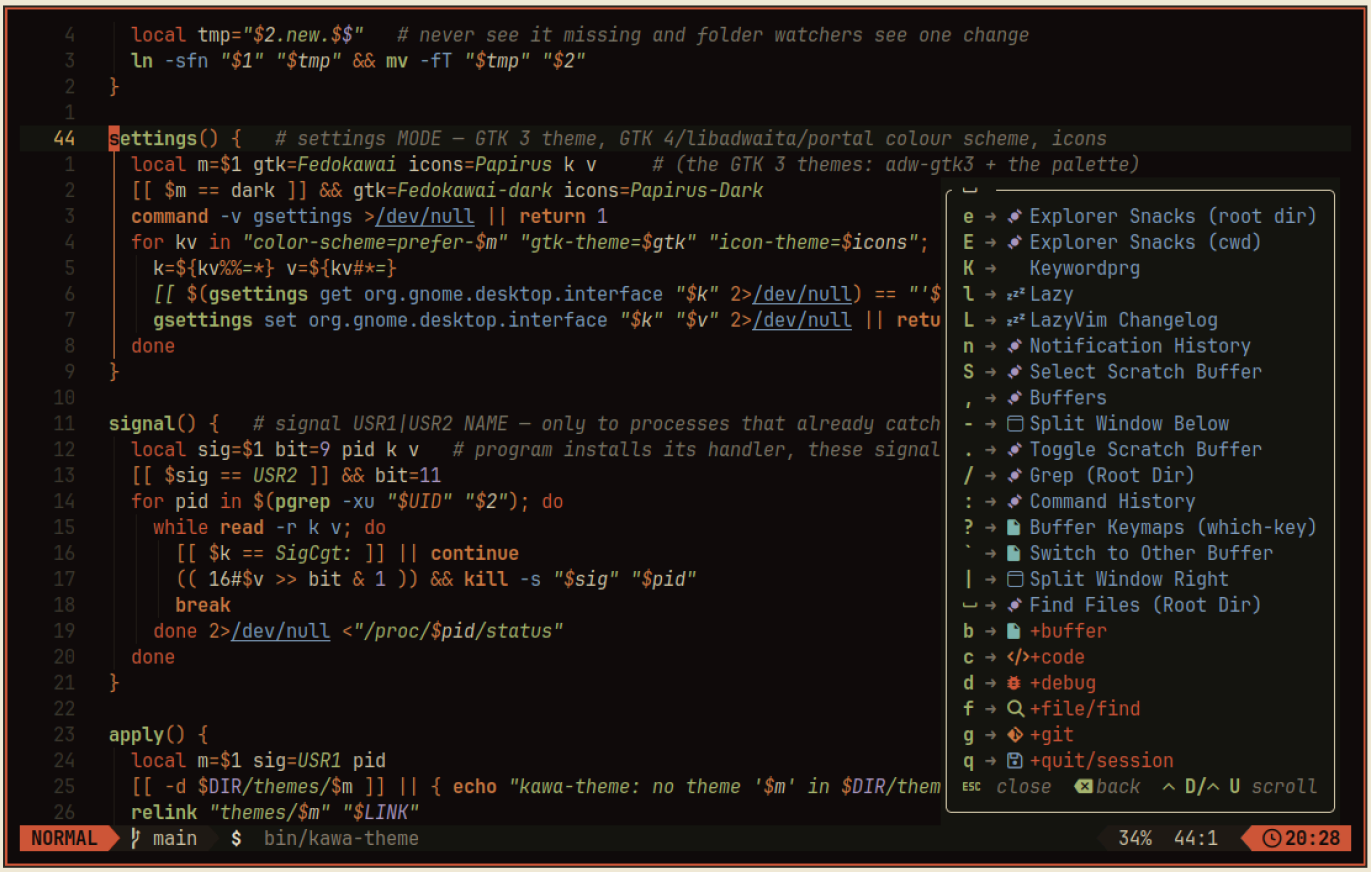
The "project" (LazyVim calls it the *root*) is the folder the language server works in, or else the git repository the file belongs to, or else the folder neovim was started in.

The file tree

Space e opens the project's folders at the left; Enter (or `l`) on a file opens it and the tree stays, `h` closes a folder. In the tree, `a` makes a new file (end the name with `/` for a folder), `r` renames, `d` deletes — to the desktop trash, after asking — `c` and `m` copy and move, `H` shows hidden files, `o` opens a file with the desktop's program for it, `q` closes the tree and `?` lists the rest.

The leader key

Most of LazyVim's own commands start with **Space**, the *leader*. Press it and wait a moment: a panel in the lower right lists what can follow, in groups — `f` files, `s` search, `g` git, `c` code, `b` buffers, `w` windows, `u` interface switches, `x` problem lists, `q` quitting and sessions. Press a letter to go down a level, Backspace to come back up, Esc to give up. The same panel helps after `g`, `z`, `[`, `]` and `"`, and Space s k searches every key by its description — type *split* or *comment*.

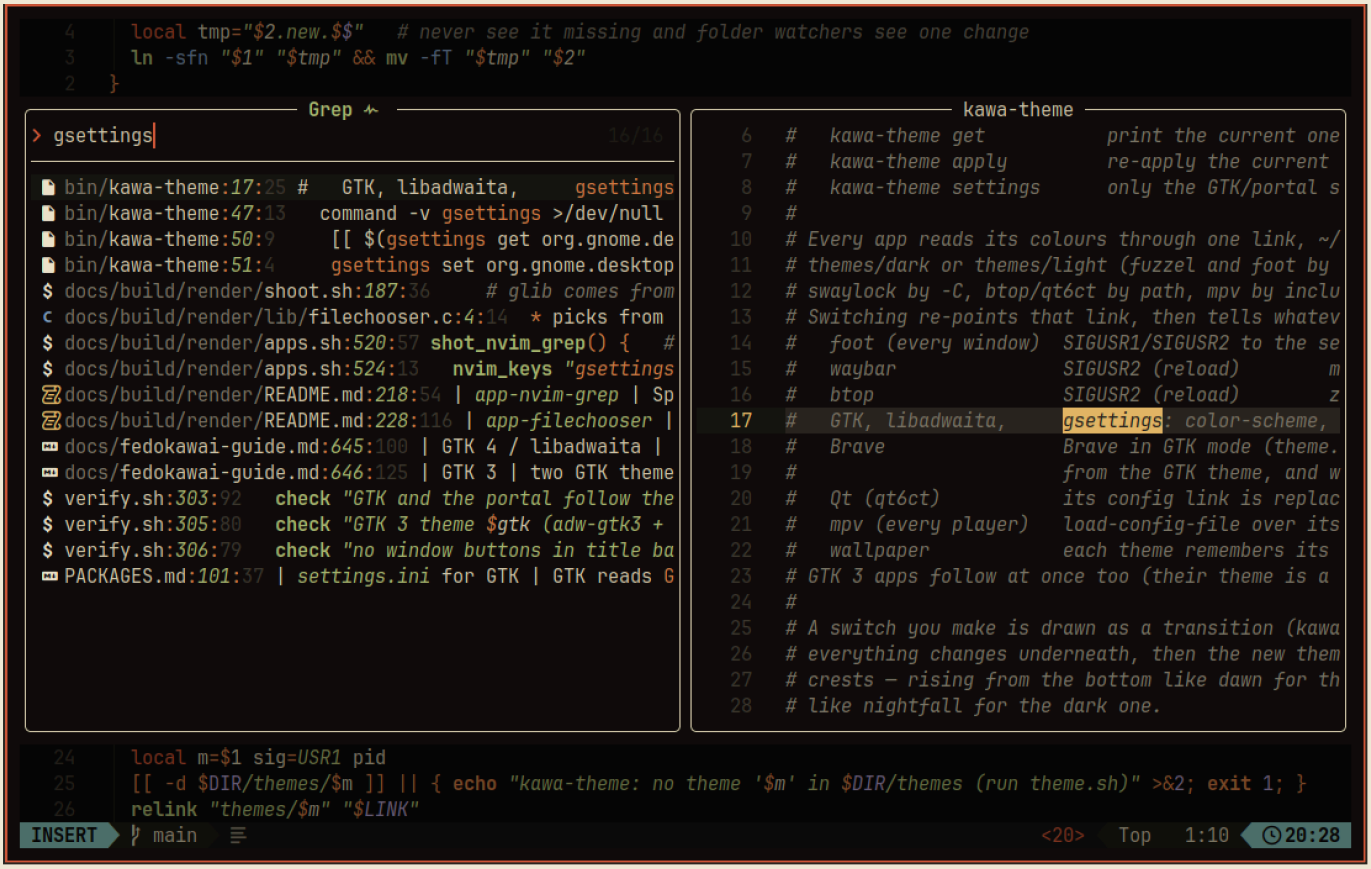


The key panel after Space: the groups that can follow, in the palette.

Keys	Does
Space Space · Space f f	find a file in the project
Space / · Space s w	search the project's text · the word under the cursor (or the selection) everywhere
Space , · Space f r	the open buffers · recent files
Space f c · Space f n	a file of neovim's own config · a new empty file
Space e	the file tree
Space g g	lazygit, in a floating window (§11.6)
Space g b · Space g f · Space g l	the history of this line · of this file · of the project
Space b d	close the buffer
Space - · Space | · Space w d	split below · split right · close the window
Space c f	format the file, or the selection
Space c a · Space c r	code actions · rename a symbol everywhere
Space c d · Space x x	the problems on this line · all problems, in a list
Space s r	search and replace across files
Space s k · Space s h	search the keys · the help pages
Space s " · Space s u	the registers · the undo history
Space u w · Space u s · Space u l	line wrap · spelling · line numbers, on or off
Space u f · Space u F	formatting on save, on or off: everywhere · this file
Space q s · Space q q	restore the last session in this folder · quit everything
Space l · Space c m	plugins (Lazy) · tools (Mason)
Space ?	the keys that belong to this buffer only

Search and replace across a project

Space / searches the text of every file in the project as you type (with ripgrep: what `.gitignore` excludes and hidden files are skipped — Alt+H and Alt+I bring them in). Each result shows the file, the line and the match; the preview shows the place around it, and Enter goes there.



Space /: every line in the project containing the typed text, the place previewed beside it.

To change text in many files, Space s r opens **grug-far**, a replace form in a window of its own:

1. Type what to find on the *Search* line. Matches from every file appear underneath as you type.
2. Move down to *Replace* (Esc, then `j`, or Tab in Normal mode) and type the new text. The results show each line as it will be — nothing is written yet.
3. *Files Filter* starts with the current file's type (`*.sh` when you came from a shell script): clear it to search every file, or write your own (`*.md`). *Flags* takes ripgrep's options: `-i` to ignore case, `-w` for whole words, `--hidden` for hidden files.
4. In Normal mode, `\r` — backslash, then r; backslash is LazyVim's *local leader* — writes every replacement into the files. `g?` lists the other keys, `\c` closes the form.

The classic way works too: Space /, then Ctrl+Q puts every match in the *quickfix list*, and `:cdo s/old/new/g | update` replaces in each listed line and saves each file. `]q` and `[q` step through the list; Space x q shows it.

Tip: run a project-wide replace in a git repository with everything committed first: lazygit then shows exactly what changed, and discarding takes it all back (§11.6).

Language servers

A *language server* is a helper program that understands one language: where a name is defined, where it is used, what is wrong. Neovim talks to it and shows the answers — a problem is underlined, with a short note at the end of the line and a sign in the margin. LazyVim brings one for Lua (the language of neovim's config) from the start; others come with *extras* and Mason (below). Space c l shows which servers serve the current file.



A language server at work: a problem underlined with its note, and the documentation that K shows.

Keys	Does
`gd` · `gr`	go to where the name under the cursor is defined · list every place it is used
`K`	the documentation of the name under the cursor (Ctrl+F · Ctrl+B scroll it)
`gK` · Ctrl+K in Insert mode	the parameters of the function being called
Ctrl+O	back to where you were before `gd`
Space c a	code actions: the fixes the server offers here
Space c r	rename the symbol everywhere it is used
`]d` · `[d`	next · previous problem (`]e` `[e` errors only, `]w` `[w` warnings)
Space c d	the full text of the problems on this line
Space x x · Space s d	every problem, in a list · in the picker
Space s s	jump to a function, heading or other symbol in this file
Space u h	the grey hints the server writes into the code (types, parameter names), on or off

For this desktop's shell scripts: `:Mason`, type `/bash` and Enter to find **bash-language-server**, press `i`; then **shellcheck** the same way. The next shell script you open has a server that checks it with shellcheck, completes variable names and finds a function's definition with `gd`. (bash-language-server is an npm package; `:checkhealth mason` says whether npm is found.)

Completion

While you type, a menu offers what could come next: names from the language server, file paths (after `./`, `~/` or `/`), snippets, and words from the open files. A box beside it shows the documentation of the highlighted entry.

Keys in Insert mode	Does
↑ ↓ or Ctrl+P · Ctrl+N	move in the menu
Enter · Ctrl+Y	take the highlighted entry

Keys in Insert mode	Does
Ctrl+E	close the menu and keep typing
Ctrl+Space	open the menu yourself; again for the documentation
Tab · Shift+Tab	in an inserted snippet: to its next · previous blank to fill in
Ctrl+B · Ctrl+F	scroll the documentation

On the command line (after `:`), Tab completes commands and file names.

Formatting

LazyVim tidies a file's layout — indentation, spacing, line breaks — **each time you save it**, when it has a formatter for the type (conform.nvim runs them; stylua, for Lua, comes installed). Shell scripts are the exception here: `after/ftplugin/sh.lua` turns formatting on save off for them, because shfmt would re-indent and re-align them, this desktop's own scripts among them.

Keys or command	Does
Space c f	format now: the file, or the selection — a shell script too, when you ask
Space u f	formatting on save on or off, everywhere
Space u F	the same for this file only (shell scripts start at off)
`:ConformInfo`	which formatters apply to this file, and whether they are installed

Extras and Mason: adding a language

Language support comes in *extras*: `:LazyExtras` (or `x` on the start screen) lists them; move to one, press `x` to switch it on or off, and restart neovim. The choice is kept in `~/.config/nvim/lazyvim.json`, which is yours.

Extra	Brings
`lang.markdown`	for notes: a language server for links and headings, Markdown checks, headings and tables drawn in place (Space u m), a live preview in the browser (Space c p)
`lang.python` · `lang.json` · `lang.yaml` · `lang.toml`	each language's server, and its formatter where it has one
`coding.mini-surround`	the `gs` keys for quotes and brackets (above)

The servers and tools then install through **Mason** (`:Mason`, or Space c m), which downloads them into neovim's data folder — binaries, PyPI and npm packages alike (Node.js is installed). In the Mason window: `i` install, `u` update, `U` update all, `X` remove, `g?` help.

Keeping it updated and healthy

`update` already runs `Lazy! sync`, which brings every plugin to its newest version and records the versions in `~/.config/nvim/lazy-lock.json`. Inside neovim, `:Lazy` (Space l) shows them: `U` update, `S` sync, `R` restore the versions in the lock file — the way back when an update breaks something. Mason's tools are **not** part of `update`: open `:Mason` and press `U` now and then. Space L shows LazyVim's changelog. `:checkhealth` explains anything that does not work: every plugin and tool reports OK, WARNING or ERROR, with advice. `:LazyHealth` loads every plugin first, so none is skipped.

A daily workflow

1. **Super+Return**, the project's folder (`~/repos/notes` — a folder name alone changes into it, §11.2), then `nvim`.
2. On the start screen, `s` brings back the files you had open here last time; or Space Space for a file, `r` for a recent one.
3. Work. Ctrl+S saves; Space / finds anything in the project; Shift+H and Shift+L move between the open files; `gd` and `K` explain code.
4. Space g g: read the diff, stage, commit, push (§11.6); `q` returns to the editor.
5. Space q q quits. The session of this folder is saved by itself, ready for step 2 tomorrow.

A learning path

Fifteen minutes a day beats an afternoon a month: the keys move into your hands only by use.

When	Do	Why
day 1	`nvim --clean +Tutor`, about 30 minutes; recent neovims continue with `:Tutor vim-02-beginner`	the official lessons, in plain neovim without LazyVim's changes, exactly as written (`vimtutor` belongs to Vim, which is not installed)
week 1	the first hour above, again each day: moving, typing, `x`, `dd`, `u`, `:w`, `:q`	until you stop reaching for the mouse and the arrows
week 2	the grammar: `dw`, `cw`, `ciw`, `ci"`, `dt)` — and `.`	every verb you know multiplies with every motion
week 3	Space Space, Space /, Space e, buffers and splits	moving between files as quickly as within one
week 4	visual block, `:%s`, one macro a day on real work	the keys that save hours
afterwards	one new key a week, from the Space panel or Space s k	small steps stay learnt

Pitfalls.

- LazyVim changes a few keys of plain Vim: `s` and `S` are flash (Vim's `s` is `cl`), Shift+H and Shift+L switch buffers (in Vim they move to the top and bottom of the screen), Ctrl+H/J/K/L move between windows, and `K` shows documentation. Tutorials written for Vim mean the plain keys; `nvim --clean` has them.
- With the completion menu open, **Enter takes the highlighted entry** instead of starting a new line. Ctrl+E closes the menu first.
- Markdown, plain text and commit messages wrap long lines — `j` and `k` then move by screen line — and check spelling: `]s` · `[s` jump between mistakes, `z=` offers corrections, `zg` adds the word to your list, Space u s turns the checking off.
- Hidden files do not show in Space Space or Space / — the dotfiles of a dotfiles repository included. Alt+H in the picker, `H` in the tree.
- `lang.markdown` also corrects Markdown files on save where its checker finds problems. For a file whose layout must stay as it is — this handbook's source, say — Space u F turns it off.
- Do not set `background` yourself, and leave Space u b alone: neovim learns light or dark from foot and changes with the theme.
- Your own plugins and settings go in `lua/plugins/*.lua` and `lua/config/*.lua`; only the two files named at the start of this section are managed by Theme.

Cheat sheet.

To	Keys
get back to Normal mode	Esc
save · quit · both · quit without saving	Ctrl+S · `:q` · `:wq` · `:q!`
undo · redo · repeat the last change	`u` · Ctrl+R · `.`
move by character · word · to the line's start, end	`h` `j` `k` `l` · `w` `b` `e` · `0` `$`
top · bottom of the file · line 42	`gg` · `G` · `42G`
half a screen down · up	Ctrl+D · Ctrl+U
jump to anything on screen	`s`, a letter or two, the label
back · forward along your jumps	Ctrl+O · Ctrl+I
type before · after · at the line's start · end · on a new line	`i` · `a` · `I` · `A` · `o` `O`
delete a character · a line · to the line's end	`x` · `dd` · `D`
change a word · inside quotes · inside brackets	`ciw` · `ci"` · `ci(`
copy a line · a paragraph · paste	`yy` · `yip` · `p` `P`
indent · comment out a line	`>>` · `gcc`

To	Keys
move a line down · up	Alt+J · Alt+K
select characters · lines · a block	`v` · `V` · Ctrl+V
search · next · previous · the word under the cursor	`/text` · `n` · `N` · `*`
replace in the whole file	`:%s/old/new/g` (`gc` to confirm each)
record · stop · play a macro	`qa` · `q` · `@a`, `` `, `` 10@a`
open the link under the cursor in Brave	`gx`
fold · unfold · open every fold	`zc` · `zo` · `zR`
find a file · text · an open buffer	Space Space · Space / · Space ,
the file tree · lazygit	Space e · Space g g
previous · next buffer · close it	Shift+H · Shift+L · Space b d
split below · right · move between · close	Space - · Space | · Ctrl+H/J/K/L · Space w d
definition · uses · documentation · rename	`gd` · `gr` · `K` · Space c r
next problem · all problems	`]d` · Space x x
format · code action	Space c f · Space c a
replace across the project	Space s r
every key	Space, then wait · Space s k
plugins · tools · diagnosis	`:Lazy` · `:Mason` · `:checkhealth`

11.6 lazygit — git



lazygit in a repository with changes: files, the diff, branches and commits.

What git is

git keeps the history of a folder. The folder becomes a *repository*: you change its files as usual, and whenever a set of changes is worth remembering, you record a *commit* — a snapshot of those changes with a one-line message saying why, your name and the time. Commits line up into a history you can read, compare and return to; nothing committed is lost to later edits. Before committing you choose what goes in: *staging* a change puts it on the tray for the next commit, so one commit can hold the alias you added and not the half-finished experiment in the same file. git keeps all of this in a hidden `.git` folder inside the repository.

A *branch* is a named line of commits. The usual one is called `main`; a second branch lets you try something — a new bar layout, a rewrite of a chapter — without disturbing it, then *merge* it in if it works or delete it if not. A *remote* is another copy of the same repository somewhere else: a private repository on GitHub or Codeberg, or another computer. *Push* sends your new commits there; *pull* brings in the commits made elsewhere — on your other machine, say. For dotfiles and notes, that means every change is recorded with its reason, any old version is a few keys away, and the same files live on every machine you use.

Once, before the first commit

```
git config --global user.name "Your Name"
git config --global user.email you@example.com
git config --global init.defaultBranch main
git config --global pull.rebase false
```

The name and address go into every commit you make. The third line names the first branch of new repositories `main`; the fourth lets `pull` join two histories that have both moved on with a merge commit, rather than stop with an error.

Opening it

Type `lazygit` inside any git repository, press **Space g g** in neovim (it opens the repository of the current file), or use `:` then `lazygit` in yazi. Started in a folder that is not a repository, it asks *Create a new git repository?* — `y`, then Enter for git's default branch name: the way to start keeping a folder under git (`mkdir -p ~/repos/notes && cd ~/repos/notes && lazygit`). A repository that already exists on a website comes with `git clone URL ~/repos/NAME`. Everything lazygit does is an ordinary git command, listed in its command log, so `git`, the prompt (§4.6) and neovim see the same thing.

The screen

Five panels on the left; on the right the *main view*, showing whatever the selected line stands for — a file's changes, a commit, a branch's log.

Panel	Key	Lists	More tabs, with [ ]
Status	1	the repository and branch; ↑ commits not pushed yet, ↓ commits waiting to be pulled	—
Files	2	every changed file, as a folder tree	Worktrees, Submodules
Branches	3	the local branches; the one you are on is marked `*`	Remotes, Tags
Commits	4	the history of the current branch, newest at the top	Reflog
Stash	5	changes put aside	—

A file's status letters are git's own: `??` new and not yet tracked, `M` modified, `A` added, `D` deleted, `R` renamed, `UU` in conflict. The left letter stands for the staged part, the right one for the part not yet staged.

Keys	Does
1 … 5 · 0	jump to a panel · to the main view
← → or h l (Tab, Shift+Tab)	previous · next panel
↑ ↓ or k j · `<` `>`	move in the panel · to its top, bottom
Enter · Esc	go into the selected item (a file's lines, a commit's files) · back out

Keys	Does
/ · n N	search or filter the panel · next, previous match
?	every key of the panel you are in, with what it does
`+` · `_`	a bigger main view (half, then the whole screen) · back
:	run any command — `git log --stat`, say
q	quit

The colours here

The repository's `config.yml` names the terminal's colours instead of fixing its own, so lazygit switches with the theme along with foot:

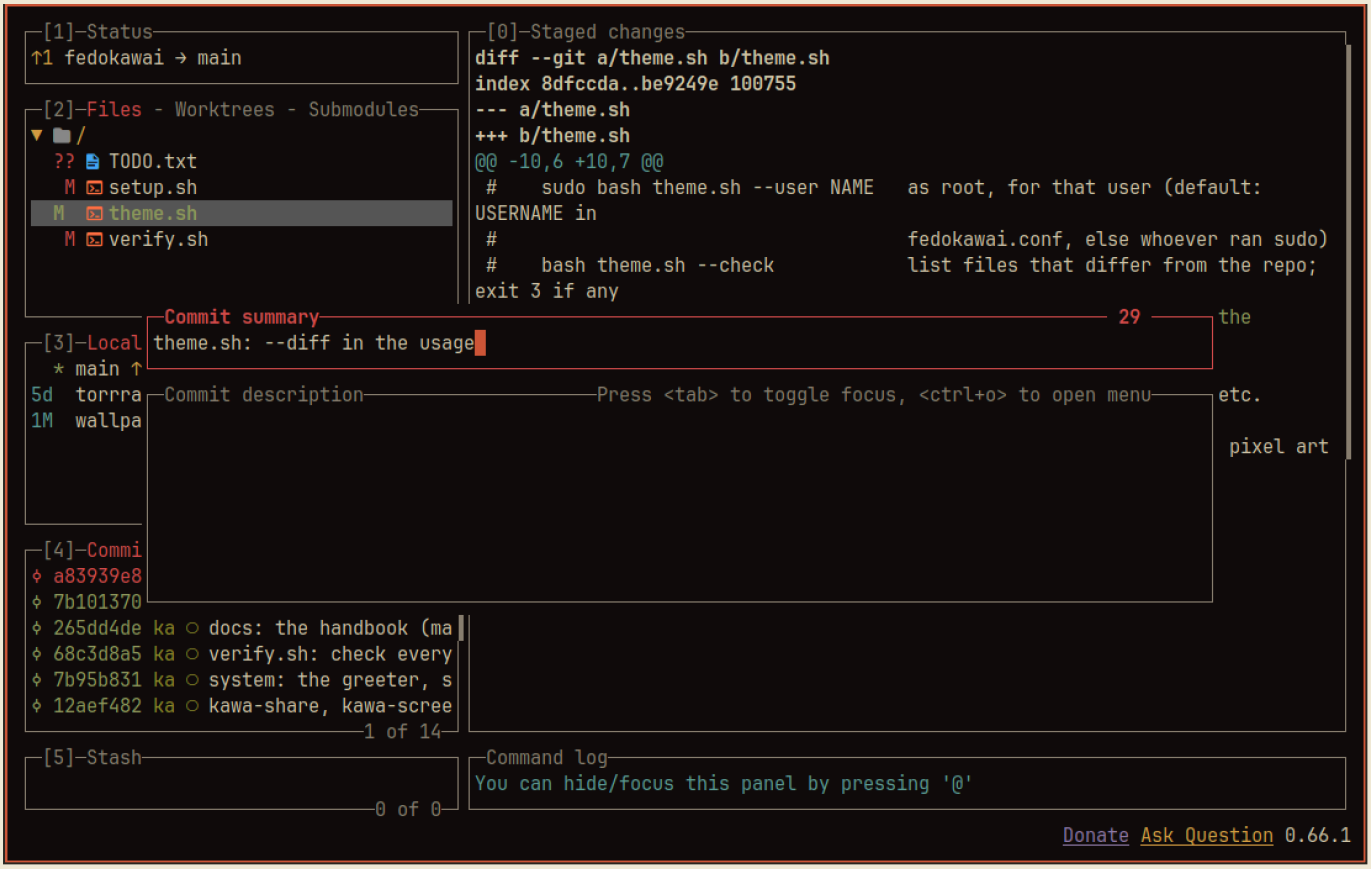
What	Colour
the frame of the panel you are in	vermilion, bold — like the frame of the focused window
the other frames and their titles	a quiet warm grey (🔳 `#877f6f`, one value that reads on ink and on paper)
the frame while you search with `/`	yamabuki
the key hints along the bottom	ai blue
the selected line	the terminal's "black": a shade off the ground, on either theme
unstaged changes	vermilion
a commit's hash	vermilion: only here, not pushed yet · yamabuki: pushed · matcha: already in `main`

Frames have square corners (`border: single`), files carry Nerd Font icons, and `overrideGpg: true` lets a signing passphrase be typed inside lazygit, should you sign commits. Inside neovim (Space g g), Snacks lays a colour file made from the editor's colours over this one, so the frames and the selected line follow the editor there.

A first commit, step by step

Say you keep notes in `~/repos/notes` and have just written `ideas.md`.

1. Start `lazygit` in the folder (or Space g g in neovim). *Files* lists `ideas.md` with `??`: git does not know the file yet.
2. With the file selected, **Space** stages it: its letters turn to `A`, in green. The main view shows what will go into the commit.
3. **c** opens the commit box. Type a short summary — what changed and why, in one line: `notes: first ideas for the bar` — and press **Enter**. For more, **Tab** moves to the description box below (Enter there starts a new line); Tab again goes back, and Enter in the summary commits both.
4. The file leaves *Files* — there is nothing left to commit — and *Commits* shows the new commit at the top, its hash in vermilion: so far it exists only on this machine.



The commit box: a one-line summary above, the description below, the staged file behind them.

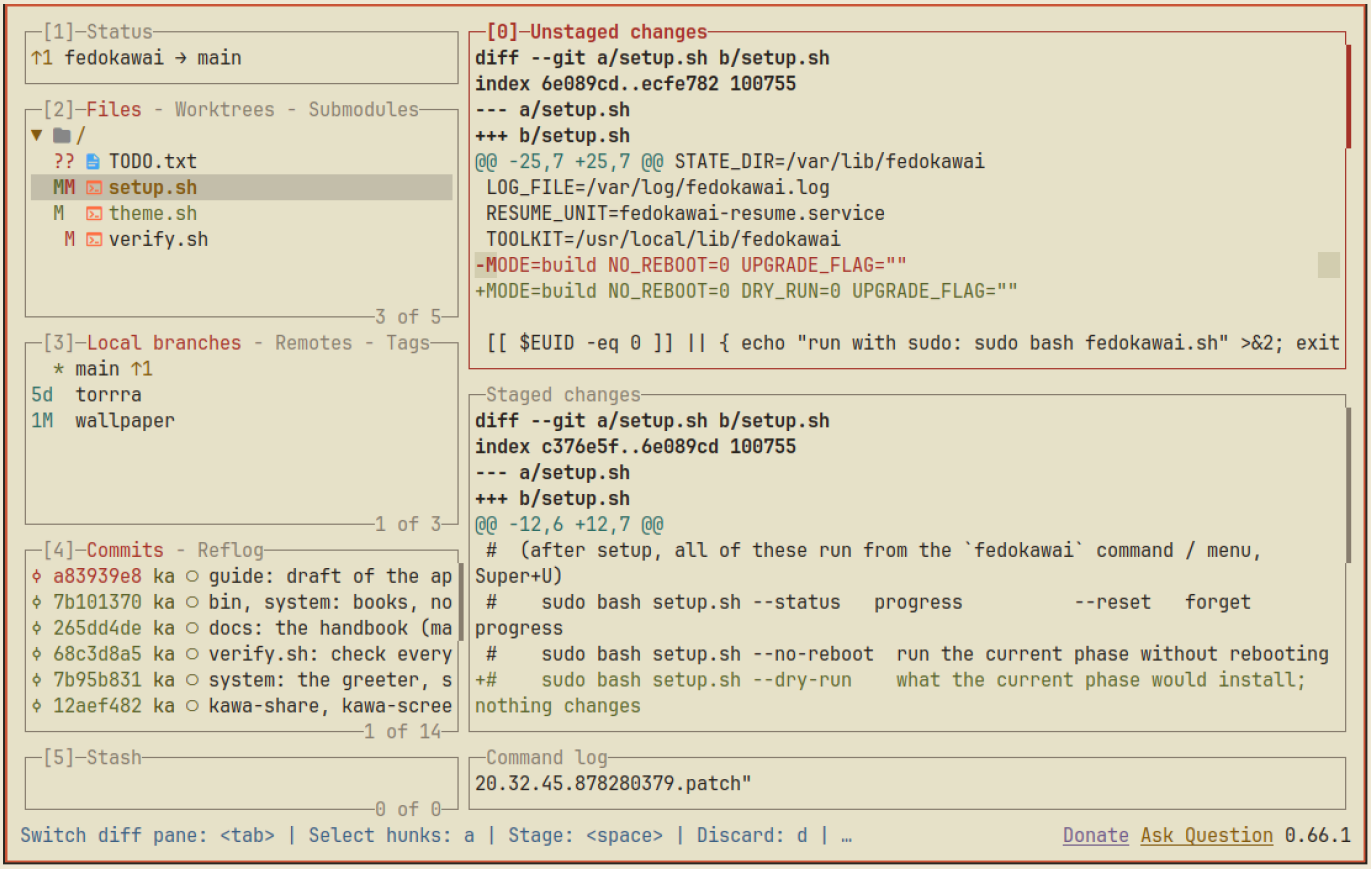
If nothing is staged when you press **c**, lazygit asks whether to commit every change.

Tip: before trying anything unfamiliar, commit or stash what you have. With nothing uncommitted, a step that goes wrong can be taken back with **z**, or from the reflog (*Recipes*, below).

Staging part of a file

A commit should hold one change you can describe in one line. When a file holds two — a fixed alias and a half-written function — stage them apart:

1. In *Files*, select the file and press **Enter**. The main view takes over, showing the changes not yet staged, with one *hunk* (a block of changed lines) selected.
2. ↑ ↓ move from hunk to hunk; **a** switches to single lines and back; **v** starts a range.
3. **Space** stages the selection, which moves to the *Staged changes* pane beside it. **Tab** switches between the two panes; Space in the staged one unstages.
4. **Esc** returns to the files, and **c** commits only what is staged. The rest stays as it is, for the next commit.



Staging by hunk: the unstaged changes on one side, the staged ones on the other, the selected hunk highlighted.

In this view **d** removes the selected change **from the file** (in the staged pane, it unstages it); **E** opens the hunk in neovim to edit it before staging. Neovim can stage too, from inside the file: the coloured bars in the margin mark changed lines, `]h` · `[h` jump between hunks, Space g h s stages the hunk under the cursor, Space g h r resets it, and Space g h p shows what changed.

Commit, amend, undo

Keys (in Files)	Does
c	commit what is staged
C	commit, writing the message in neovim — save and quit (`:wq`) to finish
A	amend: add what is staged to the last commit (it asks first)
w	commit without running the repository's hooks
z · Z	undo · redo the last git action

z reads git's own record of actions (the *reflog*) and asks before acting. Undoing a commit takes the commit back and leaves its changes staged; a checkout, a reset or a rebase is undone too. It does not bring back changes discarded with **d** that were never committed, and it does not touch what has already been pushed.

Tip: a typo in the last commit's message: select it in *Commits* and press **r**. A file forgotten from it: stage the file and press **A** in *Files*.

Stash

Stashing puts uncommitted changes aside and leaves the files as they were at the last commit — for when you need to switch branches or pull in the middle of something.

Keys	Does
s (Files)	stash the changes to tracked files; it asks for a name — Enter alone will do
S (Files)	stash options: *including untracked files*, only the staged changes, only the unstaged …
Space (Stash, 5)	apply the entry: the changes come back, the entry stays

Keys	Does
g (Stash)	pop: the changes come back, the entry goes
d (Stash)	drop the entry

Branches

Keys (in Branches, 3)	Does
n	a new branch from the selected one, and switch to it
Space · `-`	switch to the selected branch · back to the previous one
M	merge the selected branch into the one you are on (then *Regular merge*)
r	rebase the branch you are on onto the selected one (then *Simple rebase*)
R · d	rename · delete (it asks: the local branch, the remote one, or both)
f	fast-forward the selected branch to its remote
]	the *Remotes* tab: the branches of other copies

Merge or rebase. Both bring the work of one branch into another. **Merge** keeps both lines of history and joins them with a merge commit — simple, and safe on branches others share. **Rebase** replays your branch's commits on top of the other one by one, as if you had started from there: a straight history, but your commits are rewritten, so keep it to commits nobody else has. For an experiment that worked on its own branch: switch to `main` (Space), select the experiment's branch, **M**. To bring `main`'s new commits under a branch you are still working on: stay on that branch, select `main`, **r**.

Pull and push

A repository needs a remote before it can push. Create an empty private repository on GitHub or Codeberg, then in *Branches* press **]** for *Remotes*, **n**, name it `origin` and paste the address the site shows (an SSH address needs your SSH key added to the site; an HTTPS one asks for a token instead of a password).

Keys	Does
P	push the branch. The first time, lazygit asks which remote branch to track: `origin main`
p	pull: fetch the remote's new commits and merge them
f (Files)	fetch only: see what is new (↓ in *Status*) without merging it yet

lazygit also fetches by itself every minute while it is open, so the ↓ count is current. After amending or rebasing commits that were already pushed, **P** says the branch has diverged and offers a *force push*: Enter replaces the remote's history with yours. Do that only on a branch nobody else uses — on a one-person dotfiles repository, that is every branch.

Resolving a conflict

A merge, rebase or pull stops with a *conflict* when both sides changed the same lines — the same alias edited on two machines, say. Nothing is lost; git waits for you to choose.

1. lazygit reports the conflicts; *Files* lists the files concerned, marked `UU`.
2. Select one and press **Enter**. The main view shows each conflict: your side (`HEAD`), then the incoming side, between git's `<<<<<<<`, `=======` and `>>>>>>>` lines.
3. ↑ ↓ choose a side and **Space** keeps it; **b** keeps both; ← → go to the next or previous conflict; **z** takes a choice back. Or **e** edits the file in neovim: write the lines as they should be, delete the three marker lines, `:wq`.
4. When a file has no conflict left, lazygit stages it. When none remain, it asks *All merge conflicts resolved. Continue the merge?* (or the rebase) — Enter. A merge then commits; a rebase moves on to its next commit, which may stop again.

m offers *continue* and *abort* at any time; abort puts everything back as it was before the merge or rebase began.

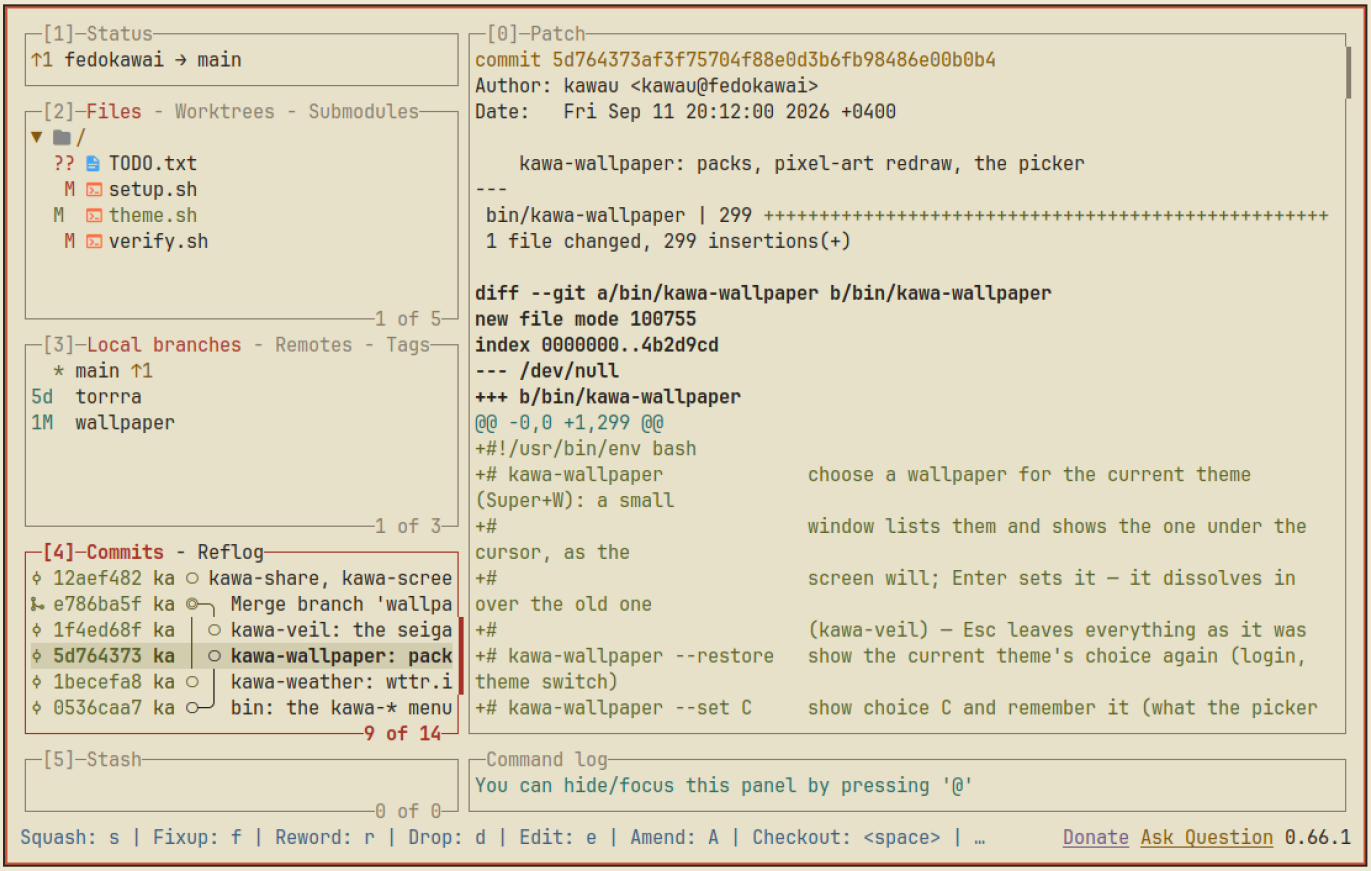
Rewriting history: interactive rebase

Commits not pushed yet — vermilion hashes — can be tidied before anyone sees them: joined, reworded, reordered, dropped. Select a commit in *Commits*:

Keys	Does
s	squash it into the commit below; both messages are kept, joined
f	fixup: into the commit below, its own message dropped
r · R	reword its message · reword it in neovim
d	drop the commit, and its changes with it
Ctrl+J · Ctrl+K	move it down · up the list
A	amend it — any commit, not only the last — with what is staged
e	stop the history at this commit to change it: edit the files, stage, **A**, then **m** → continue
i	start a rebase of the branch's commits: mark several with s, f, d (or p, to keep), then **m** → continue runs them all

Each single key runs at once, after asking. While a rebase is stopped, **m** → abort puts everything back; once it has finished, **z** undoes it.

History and diffs



The commit history: the branch graph beside the hashes, one commit's changes on the right.

Keys	Does
4, then ↑ ↓	the commits; the main view shows each one's message and changes
Enter · Enter · Esc	the files a commit changed · one file's diff · back
/	search the messages
Ctrl+S	filter the history: only the commits touching a path (a file's history), or by one author

Keys	Does
W	compare: mark the selected commit, move to another, and the main view shows the difference; **W** again to end it
a (in Status, 1)	every branch, as a graph
Ctrl+L	log options: order, graph
] (in Commits)	the *Reflog* tab: every position the branch has been in, even ones no longer in the history

Cherry-pick

To copy a commit from one branch onto another without merging the whole branch: in *Branches*, press Enter on the branch that has it, to see its commits; select the commit and press **C** (for several: **C** on each, or a range with **v**); switch to the branch where it should land; in *Commits*, **V** pastes it there as a new commit. Esc or Ctrl+R forgets the copied set.

Recipes

I made a mess — go back.

The mess	The way back
one file's uncommitted changes are wrong	*Files*: select it, **d** → *Discard all changes*. Gone for good
everything uncommitted is wrong, but might be useful later	**s**: it waits in panel 5 until you **g** it back or **d** it
the last commit is wrong, not pushed	**z** (its changes come back as staged), or fix the files and **A**
an older commit is wrong, already pushed	*Commits*: select it, **t** (revert) — a new commit that undoes it, safe to push
a merge or rebase is going badly	**m** → abort
a rebase or reset has finished, and the result is wrong	**z**; or *Commits* → **]** *Reflog*, select the line from before, **g** → *Hard reset*
everything since the last commit must go	**D** → *Nuke working tree* — think twice: untracked files go too

A hard reset also throws away whatever is uncommitted, so stash first. git keeps commits that dropped out of the history in the reflog for about a month, so a commit you can still remember is almost never really lost.

Commit only part of a file. Files, Enter on the file, select the hunk (**a** for single lines), **Space**, **Esc**, **c** — see *Staging part of a file*, above.

Move a commit to a new branch. You committed on `main` and meant to start a branch:

1. In *Branches*, with `main` selected, **n** and a name: the new branch starts here, with the commits, and you are on it.
2. Select `main` and press **Space** to go back to it.
3. In *Commits*, select the last commit that belongs on `main`, **g** → *Hard reset*. `main` is back where it was; the commits live on in the new branch.

N (*Move commits to new branch*) does the same in one step for commits not pushed yet, and asks where the new branch should start. For a single commit from the middle: cherry-pick it to the other branch (above), then **d** drops it here.

Recommended. From inside neovim (Space g g), lazygit is set up by LazyVim itself: it takes the editor's colours, and **e** opens the file in that same neovim instead of starting a second one. Started on its own, `e` runs neovim inside lazygit's window and returns when you quit it. Commit small and often, with a message that says why: `bar: show seconds` reads better in a year than `update`.

Pitfalls.

- **z** undoes git actions, not edits you have not committed — discarded changes (**d**) are gone.
- Commits are made with your git identity: set it once (above) if git complains.
- **Space** on a *commit* checks that commit out: the files show the old state, and lazygit reports a *detached HEAD*. Space on a branch in *Branches* (or **-**) brings you back.

- Rewriting commits that are already pushed — amend, squash, reword, drop — needs a force push and breaks other copies that have them. On a one-person repository that is harmless; anywhere shared, revert (**t**) instead.

Cheat sheet.

To	Keys
move between panels · jump to one	← → · 1 … 5
every key here	?
stage a file · all files · single lines	Space · a · Enter, then Space
commit · in neovim · amend the last	c · C · A
discard a file's changes · everything	d · D
stash · pop it back	s · g (in Stash)
undo · redo	z · Z
pull · push · fetch	p · P · f
new branch · switch · back to the previous	n · Space · `-`
merge into this branch · rebase onto	M · r
squash · fixup · reword · drop · move	s · f · r · d · Ctrl+J, Ctrl+K
copy commits · paste them here	C · V
revert a commit · reset to it	t · g
look inside a commit · back out	Enter · Esc
a file's history	Ctrl+S, by path
continue or abort a merge or rebase	m
quit	q

11.7 bat, ripgrep, fd, fzf, eza, zoxide and tldr

Small command-line tools, each for one job; ripgrep and fd also power neovim's finders and yazi's search, fzf the clipboard window, the shell's Ctrl+R / Ctrl+T / Alt+C and yazi's **z**, zoxide the shell's `cd` and yazi's **Z**. Their keys in the shell are in §11.2.

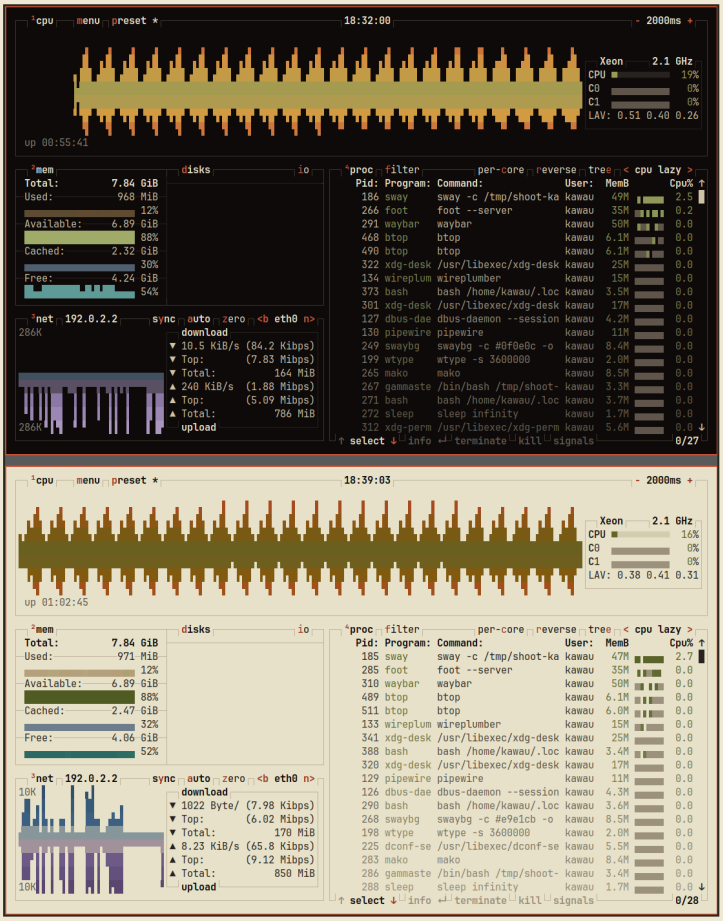
Command	Does
`bat file`	the file with line numbers, git changes in the margin and syntax colours from the terminal's palette (so it follows the theme)
`bat -p file` · `bat -A file`	plain, no decorations · show tabs, spaces and line ends
`rg text`	search below the current folder, skipping what `.gitignore` excludes
`rg -i text` · `rg -l text` · `rg -t py text`	ignore case · only file names · only Python files
`rg -uu text`	also hidden and ignored files
`fd name`	find files whose name contains *name*
`fd -e pdf` · `fd -t d conf`	by extension · folders only
`fd -H -S +1g . ~`	files over 1 GB anywhere in your home, hidden folders included
`fd -e jpg -x magick {} {.}.webp`	run a command on every match
`nvim "$(fzf)"`	pick any file below here by typing part of its name, and edit it
`ll` · `lt` · `eza -lT -L 3 --git`	the folder in detail · as a tree · a deeper tree with sizes and git state
`ll -s modified` · `ll -s size -r`	newest last · biggest first

Command	Does
`cd proj` · `cdi` · `zoxide query -l`	jump by a word · pick in fzf · every folder zoxide knows, best first
`zoxide remove ~/old/folder`	forget a folder
`tldr ffmpeg` · `tldr --update`	examples · refresh the pages now

On Fedora the fd package is called `fd-find`, but the command is `fd`.

Tips. In Ctrl+R, type words in any order — `ssh 192` finds `ssh admin@192.168.1.20`. In any fzf list, `'word` matches *word* exactly, `^start` and `end$` anchor it, and `!word` leaves out what contains it. zoxide ranks by *frecency* — how often and how recently — so the folder you live in wins after a day or two.

11.8 btop — the system monitor



btop: CPU, memory, network and the process list, in the palette.

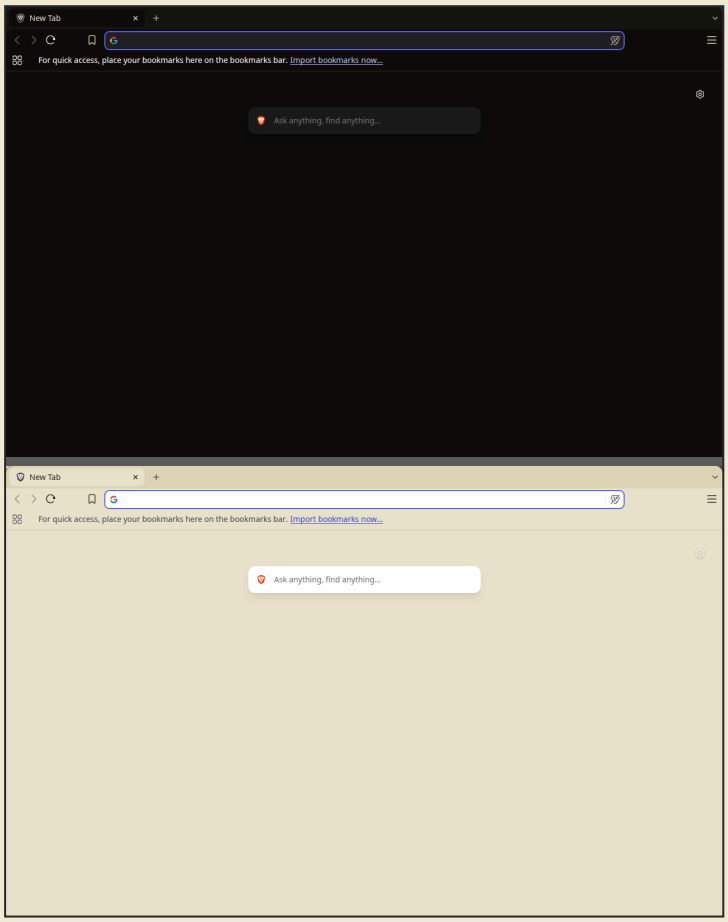
Click the **CPU** part of the bar: btop opens in a floating window. Or type `btop`.

Keys	Does
↑ ↓ · Enter	choose a process · details of it
f	filter the process list (start with `!` for a regular expression)
← → · r	sort by the previous / next column · reverse
e	process tree
t · k · s	end the chosen process (SIGTERM) · kill it (SIGKILL) · send another signal
1 2 3 4 · 5	show / hide CPU, memory, network, processes · the GPU box
d	disks in the memory box

Keys	Does
p · m · o	cycle layout presets · menu · options
q	quit

Pitfalls. Changes made in btop's own options menu last until you close it: the config says `save_config_on_exit = False`, so `~/.config/btop/btop.conf` stays exactly as the repository has it. Make lasting changes there and run Theme. On AMD the GPU box needs the ROCm SMI library, which is not installed, so it may stay empty; LACT (§11.19) shows the card instead.

11.9 Brave Origin — the browser



Brave Origin's new-tab page on both themes: the tab strip and toolbar in the palette, no window buttons; the address bar, focused, keeps Brave's own grey (white on paper) and blue ring.

Super+B. Brave Origin is Brave with Rewards, Wallet, VPN, Leo, News and the Tor windows removed from the build; the Shields (ad and tracker blocking) remain. It is free on Linux and the only browser here (`verify` fails if another appears).

What this desktop sets up

Very little, on purpose:

- **The desktop's colours.** Brave runs in its GTK mode (Theme sets it in the profile while Brave is closed): the tab strip, toolbar, menus and dialogs take the palette from the GTK theme, the new-tab page is the theme's ground, and both follow a theme switch live. Web pages that offer a dark version follow the portal's dark / light. The address bar and Brave's own pages (settings, Shields) keep Brave's greys.
- **No window buttons** — the tab strip ends at the tab list; Super+Q closes the window.
- **File dialogs.** Upload buttons and *Save as* open the desktop's file dialog, served by the portal (§4.9).
- **Screen sharing** goes through the portal and the *share what?* menu; a tab's sound always travels, and *Also share system audio* is turned on for screens and windows (§4.10).

- **No flags file** — Chromium 140 and later choose Wayland by themselves. Whether videos are decoded by the GPU is Brave's own default; `brave://gpu` → *Video Decode* shows it.
- Links opened elsewhere — foot's URL mode, lazygit, spotify_player's sign-in — open in Brave, and magnet links clicked in Brave open in torrra (§12).

Worth checking once

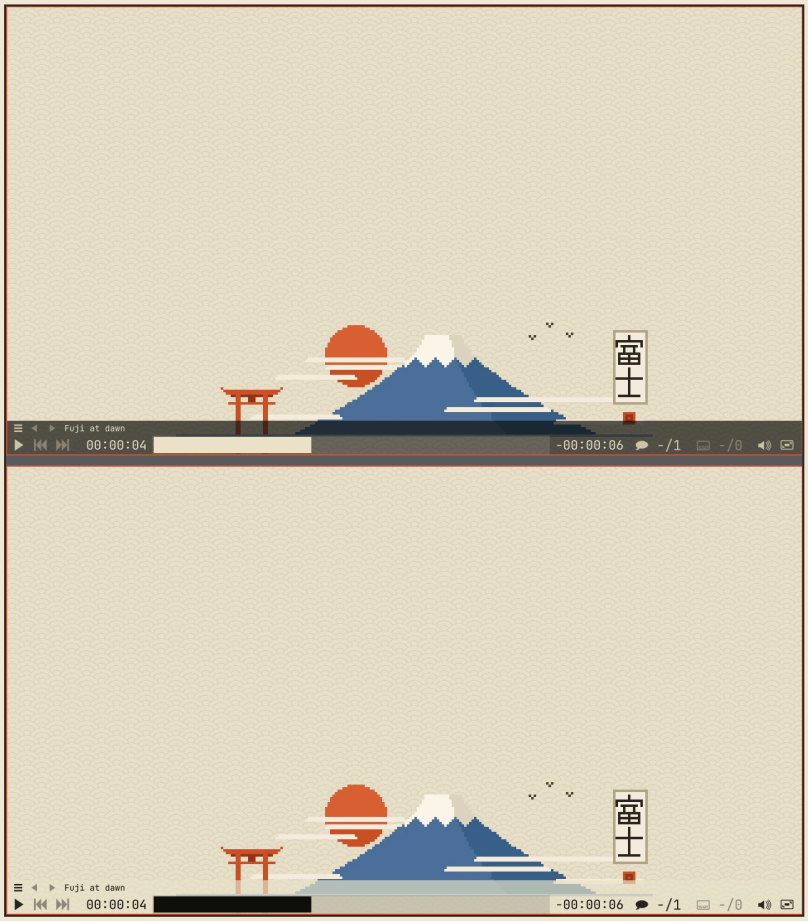
These live in Brave's own settings, are not managed here, and their names can move between versions:

- *Settings* → *Shields*: the defaults for blocking trackers and ads and for fingerprinting. The lion icon in the address bar changes them for one site — the first thing to try when a site misbehaves.
- *Settings* → *Privacy and security*: the data-collection switches. Origin removes most reporting; look rather than assume.
- *Settings* → *Search engine*, the password manager and sync, to taste.

Keys	Does
Ctrl+T · Ctrl+W · Ctrl+Shift+T	new tab · close tab · reopen the last closed tab
Ctrl+L · Ctrl+Tab · Ctrl+1…8 · Ctrl+9	address bar · next tab · that tab · the last tab
Ctrl+Shift+N	private window
Ctrl+J · Ctrl+H · Ctrl+F	downloads · history · find in page
F11 or Super+F	fullscreen

Pitfalls. Super+Q closes the whole window with all its tabs; Ctrl+W closes one, and Ctrl+Shift+T brings tabs back. Choosing a theme from the Chrome Web Store replaces GTK mode (Theme then leaves it be); *Settings* → *Appearance* → *Theme* → *GTK* brings the desktop's colours back.

11.10 mpv and yt-dlp — video



mpv with its on-screen controller in the palette.

Open a video from yazi or Brave's downloads (mpv is the default for video and sound), or type `mpv file`, `mpv folder/` or `mpv 'https://…'`. The config decodes on the GPU (VA-API) and leaves the output to mpv's own probing (Vulkan, falling back to OpenGL, so a broken driver still shows a picture); on an AMD card it scales with the high-quality filters. It smooths motion to the monitor's refresh rate, starts at 80 % volume (up to 150 %), loads subtitles whose names resemble the video's, prefers English then Arabic for sound and subtitles, and saves screenshots as PNG in `~/Pictures/Screenshots`.

It **remembers where you stopped** — and a film watched to the end opens at the beginning next time, not on its last frame (`scripts/kawa-resume.lua`). Its on-screen text, controller and menus wear the theme's colours and switch live with it (`scripts/kawa-ipc.lua` lets `kawa-theme` reach every open player); subtitles keep paper on ink, which reads on any picture. It shows in the bar's *now playing* and answers the media keys (mpv-mpris).

Keys	Does
Space · p	pause
← → · ↑ ↓	back / forward 5 s · 1 min
Shift+← →	exactly 1 s
9 0 · m	quieter / louder · mute
f · Esc	fullscreen · leave it
v · j / J · #	subtitles on / off · next / previous subtitle track · next audio track
z · Z	subtitles 0.1 s earlier · later
[ ] · Backspace	slower / faster · normal speed
s · S · Ctrl+S	screenshot · without subtitles · of the window as you see it
g p · g s · g a · g c	pick from the playlist · subtitles · audio tracks · chapters
Ctrl+P	menu of everything
i · ?	playback statistics · the key list
Ctrl+H	GPU decoding off / on
q · Q	quit (the position is kept either way)

yt-dlp

mpv plays web videos through yt-dlp, at up to 2160p. On its own, `yt-dlp URL` downloads into the current folder; `yt-dlp -F URL` lists the formats; `yt-dlp -x --audio-format mp3 URL` keeps only the sound (ffmpeg does the conversion). YouTube hides its video addresses behind JavaScript challenges: yt-dlp solves them in Node.js (`~/.config/yt-dlp/config`), inside Node's permission sandbox — no file or network access.

Pitfalls. Japanese, Chinese and Korean subtitles come out in Noto Sans CJK, Arabic in Noto Naskh. If `i` shows dropped frames on a weak GPU, try `mpv --interpolation=no file` first.

11.11 imv — pictures

Open a picture from yazi or Brave's downloads (imv is the default for pictures), or type `imv picture.jpg`, `imv folder/` (every picture in it) or `imv -r folder/` (subfolders too). It opens **floating**, on the theme's ground colour.

Keys	Does
← → or [ ]	previous · next picture
gg · G	first · last
+ or i · - or o · a · r	zoom in · out · actual size · fit again
h j k l	move around a zoomed picture
s	cycle the scaling mode
f · d	fullscreen · name, size and position overlay

Keys	Does
t · T	slideshow: one second more · less per picture (0 = off)
Space · .	play / pause an animation · next frame
x · q	take the picture off the list · quit

Pitfalls. x only removes the picture from imv's list; the file stays. imv cannot edit: to mark up a picture, `satty --filename picture.png --output-filename marked.png` (§11.13). Its colours are read at start, so an open imv keeps the old theme.

11.12 zathura — PDF, EPUB and comics



A PDF in zathura, in the palette.

Open a document from yazi or Brave's downloads (zathura is the default for PDF, EPUB and comic books), or type `zathura --fork file.pdf` — `--fork` gives the terminal back. zathura reads **PDF**, **EPUB** and **XPS** (through mupdf) and **CBZ/CBR** comic archives. Pages open fitted to the window, one per row; the status and command bars are hidden until needed.

Keys	Does
j k · h l	scroll down / up · left / right
J K · Space · Shift+Space	next / previous page · a screen down / up
gg · G · 42G	first page · last page · page 42
a · s	fit the page · fit the width
+ - =	zoom in · out · original size
d	two pages side by side
/ · ? · n · N	search forward · backward · next · previous result
Tab	table of contents (Enter opens an entry)

Keys	Does
f · F · c	follow a link by its label · show where it leads · copy its address
Ctrl+O · Ctrl+I	back · forward through your jumps
m*x* · '*x*	set mark *x* · go back to it
Ctrl+R	**recolour the page in the theme's colours** — paper text on ink, or ink on paper
r	reload the file (set here; zathura's own `r` would rotate)
p	print (set here)
Ctrl+N · Ctrl+M	show / hide the status bar (page number) · the command bar
F5 · F11	presentation · fullscreen
q	quit

Selecting text with the mouse copies it to the clipboard. `:bmark name` saves a bookmark, `:blist` lists them, `:info` shows the document's details.

Pitfalls. Rotation has no key, because `r` reloads here. A theme switch makes open zathura windows re-read their config (§5.4), and the config says `recolor false`, so a recoloured page returns to normal: press Ctrl+R again. Printing to paper needs a print service; none is in the manifest, so the print dialog can save to a file but may list no printers.

11.13 satty — annotating screenshots

Super+Print: drag a region (the selection frame is in the theme's vermilion), and satty opens with it, the arrow tool ready (Print and Shift+Print in §4.3 save and copy without annotating).

Keys	Does
z · i · r · e	arrow · line · rectangle · ellipse
t	text: type, Shift+Enter for a new line, Enter to place it, Esc to drop it
m	numbered markers (1, 2, 3 … with each click)
g · u · b	highlighter (hold Ctrl for freehand) · blur · brush
c · p	crop · pointer (select and move)
1 … 5	colour: vermilion, yamabuki, matcha, indigo, paper
Ctrl+Z · Ctrl+Y	undo · redo
Enter	**copy to the clipboard and close**
Ctrl+S	**save to `~/Pictures/Screenshots/` and close**
Ctrl+Shift+S	save under a name you choose, and close
Esc	close without keeping anything

Pitfall. Satty closes after the first copy or save (`early-exit`). Enter copies but does **not** save a file — unlike Print and Shift+Print, which do both. Pick the one you need. The copy stays on the clipboard after satty closes (§4.8).

Tip: screenshots pile up; `fd -e png --changed-before 30d . ~/Pictures/Screenshots` lists those older than a month, and adding `-X rm` deletes them.

11.14 Sound — pavucontrol and EasyEffects

The audio menu (**Super+A**, §4.2) covers everyday needs: volume, mute, the output and the input. Two programs go further.

pavucontrol, the full mixer: audio menu → *mixer*…. It opens floating. Its tabs: *Playback* — a volume and an output device for **each program** (send a call to headphones and music to speakers, or a game to the **Share** output for a call, §4.10); *Recording* — which microphone each program listens to, shown while it records; *Output* and *Input Devices* — levels and ports of each device; *Configuration* — the card's profile (for example HDMI against analogue, or turning a card off).

EasyEffects (from the launcher; on Fedora 44 it is a Qt 6 program): effects in real time. *Output* effects change what you hear — an equaliser, bass, loudness. *Input* effects change your microphone — noise reduction, a gate, a compressor — and every program that records hears the cleaned sound. Save a chain as a preset; `easyeffects -l name` loads one, `easyeffects -b 1` / `-b 2` bypasses all effects / restores them.

Pitfalls. EasyEffects works **only while it runs**, and nothing starts it at login. Its *Autostart on login* switch writes an autostart file that only full desktops read; dwl does not. To keep it running, add the line `easyeffects --service-mode --hide-window <&- &` to `configs/home/.config/dwl/autostart.sh` in the repository — above the last line, and with the `<&-` the other lines there have — and run Theme. A program you have moved to a device in pavucontrol stays there; choosing another output in the audio menu no longer moves it.

11.15 spotify_player and cava — music

spotify_player is Spotify in the terminal: type `spotify_player`. It needs a Premium account. The first start opens Spotify's sign-in page in Brave; approve, and the login is cached (`spotify_player authenticate` signs in again from scratch). It plays through its own built-in client, appears in the bar's *now playing*, and answers the media keys.

Keys	Does
Space · n · p	play / pause · next · previous
+ - · _	louder · quieter · mute
> · < · ^	forward · back · to the start of the track
Ctrl+S · Ctrl+R	shuffle · repeat mode
g s · g l · g y · z	search page · library · liked tracks · queue
/	search in the current page
Enter · a	play the selection · actions on the current track
Tab · Backspace	next panel · previous page
D	choose the playback device
?	every key
q	quit

Its colours are the terminal's own with vermilion accents (the `sumi` theme in `theme.toml`), so it switches with the desktop.

cava draws the bar's music meter (§4.1) with its own config in `~/.config/fedokawai`. Type `cava` in a terminal for the classic full-window bars: Up / Down sensitivity, Left / Right bar width, `f` / `b` colours, `q` quits. That copy reads `~/.config/cava/config`, which it creates on first run and which is yours to change.

11.16 presenterm — slides in the terminal

A presentation is a Markdown file. `presenterm talk.md` shows it full-window and **reloads it every time you save**, so you can edit in neovim beside it; `presenterm --present talk.md` turns that off for the real thing. Super+F makes the window fullscreen; Ctrl++ in foot makes the text larger. Colours follow the theme at start (`terminal-dark` or `terminal-light`).

```
---
title: Fedokawai
author: me
---

First slide

<!-- pause -->
This line appears on the next key press.

<!-- end_slide -->

Second slide
<!-- speaker_note: say hello -->
```

Keys	Does
l j → ↓ PageDown Space	next
h k ← ↑ PageUp	previous
gg · G · 7G	first · last · slide 7
Ctrl+P	slide index
Ctrl+E	run the code block on screen (only when started with `-x`)
? · q	keys · quit

Pictures show in foot. `presenterm --export-html talk.md` writes one self-contained HTML file; PDF export needs weasyprint, which is not installed. For speaker notes, run `presenterm talk.md --publish-speaker-notes` in one window and `presenterm talk.md --listen-speaker-notes` in another.

11.17 qalc — the calculator

`qalc` in a terminal, or `qalc 'expression'` for one answer (`-t` prints only the result). It knows units, currencies, bases and functions.

```
qalc '2^64'
qalc '75 km/h to mph'
qalc '255 to hex'
qalc '3.5 h to min'
qalc -e '100 USD to AED'      # -e fetches current exchange rates first
```

Inside it: `ans` is the last result, `to unit` converts it, `exrates` updates the rates, `help`, `quit`. Up and Down recall earlier lines. For a quick sum without leaving your work, `footclient -a float qalc` opens it in a small floating window.

11.18 Documents — Xournal++, AbiWord, Gnumeric and the PDF tools

Xournal++ (launcher) writes on PDFs: *File → Annotate PDF*, then the pen (Ctrl+Shift+P), highlighter (Ctrl+Shift+H), text (Ctrl+Shift+T) and eraser (Ctrl+Shift+E). It saves your marks in a separate `.xopp` file and never changes the PDF; *File → Export as PDF* writes a new PDF with the marks in it — that is the file to send.

AbiWord for letters and simple documents, **Gnumeric** for spreadsheets (both from the launcher, both the defaults for their file types). Both open Microsoft and OpenDocument files; *File → Save As* lists what each can write. The Liberation fonts stand in for Arial, Times New Roman and Courier New with the same widths, so pages break where their authors meant. From a terminal: `abiword --to=pdf letter.docx` and `ssconvert sheet.xlsx sheet.csv` convert without opening a window. Neither is a full office suite: a document with tracked changes or a complex layout is better opened in the web version of the program it came from.

PDF tools in a terminal: `qpdf --empty --pages a.pdf b.pdf -- both.pdf` joins files, `qpdf in.pdf --pages . 1-3 -- first-three.pdf` takes pages out, `pdftotext file.pdf -` prints the text, `pdfunite a.pdf b.pdf both.pdf` is the other way to join.

11.19 Hardware — OpenRGB, Solaar, liquidctl, LACT

OpenRGB (launcher) sets the lighting of RGB devices from one window: find the devices, choose colours and effects, save a *profile*. Nothing starts it at login, so the lighting falls back to the devices' own defaults after a power cut; `openrgb --profile NAME` applies a saved profile, and that command can go in the repository's `dwl/autostart.sh` (written like the EasyEffects line in §11.14). Memory and motherboard lighting is reached over SMBus; if those devices do not appear, see OpenRGB's own SMBus notes — this desktop sets up nothing beyond loading `i2c-dev`. Its window takes the Qt palette, so it follows the theme.

Solaar (launcher) manages Logitech receivers and their devices: pair a mouse or keyboard to a receiver, read battery levels, change buttons and pointer settings. `solaar show` lists everything in a terminal. Like EasyEffects, its autostart entry is for full desktops, so it runs only when you start it.

liquidctl controls AIO coolers, fan hubs and some power supplies from the terminal: `liquidctl list` finds them, `liquidctl initialize all` prepares them (needed after every boot on many devices), `liquidctl status` shows temperatures and speeds, `liquidctl set pump speed 70` and `liquidctl set fan speed 20 30 40 60 50 100` (pairs of temperature and duty) set them. Most devices forget these settings when powered off, and nothing here applies them again at boot.

LACT (launcher; AMD only) shows the graphics card — clocks, temperatures, power — and changes it: fan curve, power limit, performance level, and, thanks to the `amdgpu.ppfeaturemask` kernel argument setup added, clocks and undervolting. Its service, `lactd`, applies the settings in `/etc/lact/config.yaml` at every boot. Change one thing at a time and test it: an unstable clock or voltage can freeze the machine, and LACT's wiki has a page on recovering from a bad overclock.

11.20 podman — the Prowlarr container

podman runs one container here: **Prowlarr**, the indexer service torrra searches through (§12). It runs rootless, as you — you are its user inside too, so every file in `~/.config/prowlarr` is yours — holds no capabilities and can gain none, runs **only when wanted** (a torrra search starts it and stops it again; the command `prowlarr` starts it and opens its page — idle it would hold some 300 MB of memory), and listens only on this machine, at http://localhost:9696. systemd manages it from a *quadlet* file, `~/.config/containers/systemd/prowlarr.container`, which `update` keeps current. A health check asks it every five minutes whether it answers; three misses in a row restart it.

Command	Does
`systemctl --user status prowlarr`	is it running, and its last lines of output
`journalctl --user -u prowlarr -e`	its full log
`systemctl --user restart prowlarr`	restart it
`prowlarr stop`	stop it when you are done with its page (`prowlarr` leaves it running; it is stopped anyway at the next restart of the computer)
`podman ps` · `podman logs -f prowlarr`	running containers (with their health) · follow the log
`podman image ls lscr.io/linuxserver/prowlarr`	the Prowlarr image you have, and when it was made (`update` fetches the newest)
`podman image prune`	remove image layers left behind by updates

Pitfalls.

- Never `sudo podman`: root has its own, separate — and here empty — list of containers.
- Do not `podman rm` or `podman run` it by hand: systemd recreates the container from the quadlet at every start. To change it, edit `system/prowlarr.container` in the repository and run `update`.

11.21 snapper — snapshots

snapper keeps snapshots of `/` (and of `/home`, which is its own subvolume): one every hour, thinned out over time; the *fedokawai baseline* taken after the build; and a **pre / post pair around every dnf transaction**, described as `dnf YYYY-MM-DD HH:MM`. A snapshot costs almost nothing until files change after it. `~/Downloads` is not in them (§6.6).

Command	Does
`snaps`	list the snapshots of `/` (`sudo snapper -c root list`)
`sudo snapper -c root list --type pre-post`	only the dnf pairs, one line each
`sudo snapper -c root status 41..42`	which files the transaction changed: `+` created, `-` deleted, `c` content, `p` permissions
`sudo snapper -c root diff 41..42 /etc/some.conf`	what changed inside one file
`sudo snapper -c root undochange 41..42`	**put back every file the transaction changed**
`sudo snapper -c root undochange 41..42 /etc/some.conf`	put back one file only
`sudo snapper -c root create -d "before trying X"`	a snapshot of your own
`sudo snapper -c root delete 30-40`	delete a range
`sudo snapper -c home list`	the snapshots of `/home`

Recommended. Before an experiment — a new repository, a driver, a big config change — take a snapshot of your own with a description. If it goes wrong, `status` from that number to the newest one shows everything that changed, and `undochange` puts it back. §13.9 walks through undoing a bad update.

Pitfalls.

- **`undochange`, not `rollback`.** `snapper rollback` works by changing Btrfs's *default* subvolume, but Fedora's boot entries name the subvolume outright (`rootflags=subvol=root` — check with `cat /proc/cmdline`), so after a rollback the same old `root` would boot again. Undo with `undochange`.
- `/boot` is a separate partition and is **not** in the snapshots. For a kernel that misbehaves, pick the previous one in the boot menu (hold Esc if it is hidden) — it stays installed beside the new one (§6.6).
- snapper's own manual warns that consistency after `undochange` is not guaranteed: it puts files back; it does not re-run package scripts. Reboot afterwards, then run `verify`.
- Deleting files does not free space while snapshots still hold them (§13.10).

Tip: `sudo snapper -c root status N..0` shows everything changed since snapshot N — the quickest way to see what an experiment really touched (§7.2).

11.22 Claude Code — an assistant in the terminal

Type `claude` in a terminal, inside the folder you want to work on. It reads and edits files there and runs commands, asking first. It came from Anthropic's signed dnf repository (stable channel), so it updates with `dnf upgrade` (and `update`) and does not update itself; `claude doctor` checks the installation.

Signing in

Either sign in with a Claude account (Pro, Max, Team or Enterprise): the first start opens the sign-in page in Brave. Or use an API key — and keep it only in `~/.config/fedokawai/secrets.sh` as `export ANTHROPIC_API_KEY=…` (§11.2); Claude Code asks once whether to use the key it finds in the environment. Never put a key in the repository or in a project's settings files.

Keys or input	Does
Esc	stop Claude mid-answer, keeping what it has done

Keys or input	Does
Esc Esc	with an empty prompt: go back to an earlier point (also `/rewind`)
Shift+Tab	switch mode: ask before edits, accept edits, **plan** (it only reads and proposes)
`\` then Enter · Ctrl+J	a new line in the prompt
Ctrl+V	paste a picture from the clipboard — a screenshot, say
Ctrl+G	write the prompt in neovim
`!command`	run a shell command and show Claude the output
`@path`	point at a file
`/init`	write a `CLAUDE.md` describing the project, read at every start
`/clear` · `/compact`	start over · summarise to free room
`/model` · `/usage` · `/resume` · `/theme`	model · cost and limits · an earlier session · colours

`claude -c` continues the last conversation in this folder, `claude -r` picks an older one, and `claude -p "question"` answers once and exits, which makes it usable in pipes: `journalctl -b -p err | claude -p "what stands out here?"`.

Recommended. Use plan mode for anything touching this repository or the system, read the plan, then let it work. Run `claude` in `~/fedokawai` to ask about the desktop itself — it can read the scripts and this handbook. For colours that follow the theme, `/theme` → the *ANSI colours only* variant of dark or light.

Pitfalls. It acts as you: it can read anything you can, `secrets.sh` included, and every command it runs has your rights. Read the permission prompts instead of approving them by habit. When it edits a managed config in `~/.config`, the next Theme run replaces the change — point it at the repository copy instead.

11.23 Steam — games

Start **Steam** from the launcher. The client's main window always goes to **workspace 9** (a rule in `config.h`); press **Super+9** to reach it. Games open on whatever workspace you are on. Steam runs through XWayland, and its own interface ignores the desktop's themes (§9).

Each game's **launch options** (in Steam: the game's *Properties → General*) take the tools installed with it:

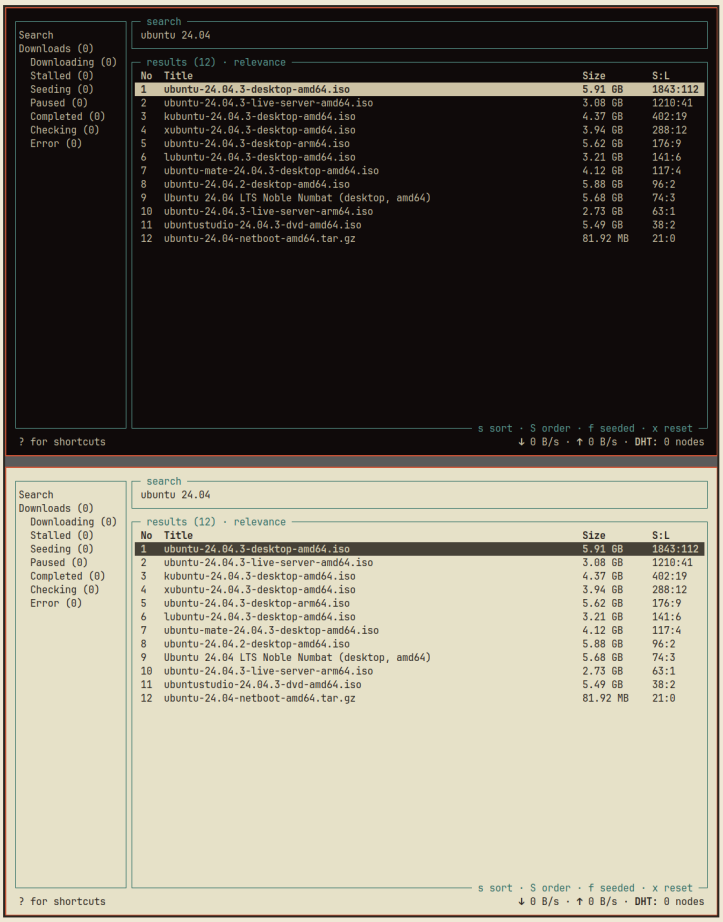
Launch options	Does
`gamemoderun %command%`	gamemode: the CPU and scheduler tuned for the game while it runs (`gamemoded -s` reports whether it is active)
`mangohud %command%`	the performance overlay, in the desktop's ink and Departure Mono; **Right Shift+F12** shows or hides it
`gamemoderun mangohud %command%`	both
`gamescope -W 2560 -H 1440 -r 120 -f -- %command%`	the game inside its own small compositor at a fixed size and refresh rate; add `-w 1920 -h 1080 -F fsr` to render lower and upscale
`gamescope --mangoapp -f -- %command%`	gamescope with the overlay — inside gamescope, use `--mangoapp` rather than `mangohud`

GE-Proton

Setup installs the newest GE-Proton, and `update` adds new releases, keeping the newest two so a game pinned to the older one still starts. Restart Steam to see a new one. Choose it per game in *Properties → Compatibility* (*Force the use of a specific Steam Play compatibility tool*), or for every game in Steam's *Settings → Compatibility*. Proton uses the kernel's `ntsync` driver (loaded at boot) for faster Windows-style synchronisation.

Pitfalls. The overlay stays off until a game's launch options ask for it, and outside gamescope `mangohud` is the switch; inside it, `--mangoapp`. MangoHud's settings are a managed file: change them in the repository's `MangoHud.conf`. `update` keeps only the newest two GE-Proton builds, so a game forced to an older one needs a new choice under *Compatibility*.

12. Torrents: searching and downloading



torrra after a search: twelve results with their sizes and seeders:leechers, in the terminal's colours.

Two pieces work together. **Prowlarr** (a small service in a container, §11.20) knows how to search hundreds of torrent sites — *indexers* — and answers one search for all of them. **torrra** is the program you use: it sends your search to Prowlarr, shows the results, and downloads what you pick with libtorrent, the engine behind most torrent clients. One API key connects them. Setup made it, and its one master copy is `~/.config/fedokawai/secrets.sh`: Prowlarr receives it as a podman *secret* (podman keeps that copy in `~/.local/share/containers/storage/secrets/`, readable by you alone, and `update` renews it from secrets.sh should you change the key there), torrra receives it from its wrapper each time it starts, and neither program writes it into its own settings.

Download only what you have the right to: Linux and BSD images, public-domain films and books, Creative Commons music, your own backups. Torrenting shares what you download with others while it runs, and your address is visible to every peer.

12.1 The first time: Prowlarr

1. Type `prowlarr` in a terminal: it starts Prowlarr and opens http://localhost:9696 in Brave. Prowlarr asks you to **create a login** — a user name and password for this page (it asks even on this computer, so no web page can reach it behind your back). Brave can remember it.
2. **Indexers → Add Indexer.** Prowlarr lists every site it knows, public and private; filter by language or type, pick the ones you use, and **Test** each one before saving. Sites marked as needing *FlareSolverr* (a browser that gets past Cloudflare checks) will not work here — none is installed — so choose others.
3. Optional: **Settings → Apps** is for connecting other programs (Sonarr, Radarr); torrra needs nothing there — it already holds the key.

That is all. `verify` warns while Prowlarr has no indexers, and torrra says so plainly instead of showing an empty list.

12.2 Searching and downloading with torrra

Open **Torrents** from the launcher, or type `torrra` in a terminal (`torrra search 'debian 13'` searches straight away; `torrra downloads` shows only the downloads). Its colours are the terminal's, so it follows the theme.

Keys	Does
type, then Enter	search all your indexers
j k · ↑ ↓ · Ctrl+D / Ctrl+U · gg · G	move · a page · top · bottom
Enter (or l)	the torrent's details; **Enter again downloads it**
s · S · f · x	sort menu · reverse the order · hide results without seeders · back to your defaults
Tab	the next panel (search, results, downloads)
p	pause or resume a download
f (on a download)	choose which files of it to download (Space selects, Enter confirms)
o	its options: speed limits, ratio, sequential download (for watching while it loads)
d · D	remove the torrent · remove it **and delete its files**
t	"turtle mode": the slow speed limits on / off
Ctrl+T · ? · Ctrl+Q	another colour theme · every key · quit

Results are sorted by **seeders** — the number of people sharing the whole file — highest first; more seeders means faster and more certain. Downloads go to `~/Downloads`, which is its own Btrfs subvolume, so a film you delete gives its space back at once rather than living on in snapshots.

Tip: when you are done with a torrent, **D** removes it and deletes its files in one step. Move anything you keep out of `~/Downloads` — nothing there is in a snapshot.

Magnet links in Brave

The first magnet link you click makes Brave ask whether to open it with *xdg-open*; tick *Always allow* and say yes. From then on a magnet link — or a `.torrent` file opened from the downloads — starts torrra in its own window, downloading it straight away. torrra runs one window at a time: if it is already open, a notice says so and the link waits on your clipboard — close torrra and click the link again (or, in a terminal, `torrra download 'PASTE'`).

12.3 How it behaves

- **Downloads run only while torrra's window is open.** Close it and they pause; open torrra again and they continue where they were. There is no background service.
- **Seeding.** A finished download keeps sharing while torrra is open, without a limit by default. To stop at a ratio or a time, set it per torrent (**o**), or for all: `torrra config set seeding.default_max_ratio 2` (shares twice what it downloaded).
- **Speed.** Unlimited by default; **t** switches to the slow limits, and `torrra config set speed_limit.download_limit "5M"` changes them (`unlimited` turns one off).
- **Ports.** libtorrent listens on **6881** (TCP and UDP, opened in the firewall by setup) and asks your router to forward it (UPnP / NAT-PMP). If 6881 is busy, it quietly uses a random port the firewall does not open, and fewer peers can reach you.
- **One torrra at a time** — a second window would share the same downloads, so it says torrra is already open.
- Settings live in `~/.config/torrra/config.toml` (yours); `torrra config list` shows them, `torrra config edit` opens the file in neovim.

12.4 When something is wrong

torrra itself exits quietly on every failure; its wrapper (`kawa-torrra`) checks Prowlarr first and says what is missing:

Message	Do
Prowlarr is not answering on 127.0.0.1:9696	`systemctl --user status prowlarr` ; `journalctl --user -u prowlarr -e`
Prowlarr refused the key in secrets.sh	`update` hands Prowlarr the key again
Prowlarr has no indexers yet	§12.1, step 2
no PROWLARR_API_KEY in secrets.sh	`update` makes one
secrets.sh must be yours and private	`chmod 600 ~/.config/fedokawai/secrets.sh`
not installed or broken	`update` reinstalls torrra
a search finds nothing although indexers are set	test them in Prowlarr (**Indexers → Test All**); one may be down or need FlareSolverr

13. Everyday recipes

13.1 Upload a file from Brave

1. In Brave, click the site's upload button. The file dialog opens: yazi, floating, in the folder you last uploaded from (§4.9).
2. Find the file: ↑ ↓ and → ← through the folders — or **z** and a few letters of its name to jump to it below this folder, **Z** and a word to jump to a folder you use often, **g d** for Downloads.
3. Press **Enter** on it. For several files (where the site takes several), **Space** on each, then Enter. **Esc** cancels.

13.2 Share your screen, with sound, in a call

1. In the call, choose to share. In Brave's picker:
2. a **tab** — its sound comes along when *Also share tab audio* is ticked;
3. a **window** or the **entire screen** — tick *Also share system audio* to send what you hear (it includes the call itself; see §4.10 for the quieter way).
4. For a window or a screen, the desktop's *share what?* menu appears; pick one. With only one choice it is taken without asking.
5. Notifications are held back while you share; critical ones still show.
6. Your microphone travels as usual; check it in the audio menu (Super+A → input).

13.3 Take, annotate and paste a screenshot

1. **Super+Print**, drag a region.
2. In satty, the arrow is ready: drag to draw. **t** for text, **m** for numbered markers, **1…5** for colours, Ctrl+Z to undo.
3. Press **Enter**: the picture is copied and satty closes. (Ctrl+S saves it to `~/Pictures/Screenshots/` instead.)
4. Paste with **Ctrl+V** in Brave, a chat, or Claude Code. **Super+V** brings back earlier copies, pictures included.

For a plain shot, **Print** (region) or **Shift+Print** (whole screen) both saves and copies.

13.4 Switch the theme and the wallpaper

1. **Super+Shift+W** switches dark ⇄ light (or quick settings → *theme*, or `kawa-theme light` in a terminal).
2. **Super+W** chooses a wallpaper **for the theme you are on**; each theme remembers its own. Pictures of a day / night pair set the other theme too.
3. For your own pictures, put them in `~/Pictures/Wallpapers`; they appear in the same menu, on both themes.

13.5 Choose which program opens a type of file

The defaults are in `~/.config/mimeapps.list`, which Theme manages (§4.9).

1. Find the type: `xdg-mime query filetype book.pdf` → `application/pdf`.
2. See what is chosen now: `xdg-mime query default application/pdf`.
3. Find the candidates: `grep -l 'application/pdf' /usr/share/applications/*.desktop`.
4. For good: change the line in `configs/home/.config/mimeapps.list` in the repository and run Theme. To try one first, `xdg-mime default NAME.desktop application/pdf` changes it at once — until the next Theme run puts the repository's choice back.

13.6 Install a program and keep `verify` happy

`verify` fails on anything user-installed that is not in the manifest, so the manifest comes first.

1. Check that it is not on the *never present* list in `PACKAGES.md`, and that it is not a second tool for a job one already does.
2. Add the package name to `PKG_APPS` in the toolkit's manifest, with a short comment: `sudoedit /usr/local/lib/fedokawai/lib/manifest.sh`.

3. Add a row to its `PACKAGES.md` (`sudoedit /usr/local/lib/fedokawai/PACKAGES.md`), so the list stays true.
4. Run `update`: it installs every package the manifest has gained, then runs `verify`.

A package from Fedora or RPM Fusion needs nothing more. One from a new COPR also needs its name in `COPRS` and `sudo dnf copr enable owner/project` by hand — `update` does not add repositories. Flatpak and Snap are refused outright.

13.7 Update everything

1. **Super+U** → *update* (its mark reads **due** once the season has come round), or type `update`. It snapshots, upgrades every package, installs what the manifest gained, checks the pinned tools and fonts, brings GE-Proton, torrra's libtorrent and Prowlarr's image up to date, syncs neovim's plugins, and finishes with `verify` (§6.2).
2. If it reports a new kernel, restart; the one before stays installed as the way back.
3. Not included: Mason's tools in neovim (`:Mason`, then `U`). The toolkit itself stays the version you installed (§6.1).

Tip: after a large update, `sudo dnf autoremove` lists dependencies nothing needs any more; read the list, then say yes (§7.3).

13.8 Move to a new Fedora

1. Wait for the menu: *upgrade to Fedora N* reads **ready** once the new release has settled (and a reminder says so). Before then it shows when it will be offered.
2. Have an hour, and 20 GB free (the status shows the space). Close what you are working on.
3. **Super+U** → *upgrade to Fedora N*, Enter, your password. It checks everything first and stops — nothing changed — if something is not ready yet, saying what.
4. If dnf reports conflicts it lists the packages it would remove and asks; *y* goes on (any the desktop needs come back afterwards), anything else stops with nothing changed.
5. The computer restarts into the installation: 20–40 minutes, then the login. A minute or two later a notice says the move is finished; the status lists anything to look at.

13.9 Undo a bad update

1. `sudo snapper -c root list --type pre-post` — find the pair for the update, by its date. Say it is 41 and 42.
2. `sudo snapper -c root status 41..42 | less` — read what it changed.
3. `sudo snapper -c root undochange 41..42` — every one of those files goes back.
4. Reboot, then `verify`.
5. The next `update` would bring the same packages back, because it upgrades everything. Until a fixed version arrives, upgrade with `sudo dnf upgrade --exclude=PACKAGE` instead.

If the problem is the new kernel, use the boot menu instead (§11.21). If the desktop does not start at all, log in on a text console (**Ctrl+Alt+F2**) and run the same commands there.

13.10 Free disk space

1. Look first: `sudo btrfs filesystem usage /` for the real picture (`df` is not reliable with snapshots), and `du -xh -d1 ~ | sort -h | tail` for the biggest folders in your home.
2. Snapshots hold deleted files: `snaps`, then delete old ranges you do not need, `sudo snapper -c root delete 30-60`. Keep the baseline (it is never cleaned up by itself).
3. The trash: **g t** in yazi, or `rm -rf ~/.local/share/Trash/{files,info}`.
4. Big files: `fd -H -S +1g . ~`.
5. Caches and logs: `podman image prune`, `sudo journalctl --vacuum-size=200M`, `sudo dnf clean all`, and Steam's own *Settings* → *Storage* for games.

Tip: chapter 7 explains each of these, and which caches are safe to clear (§7.4).

13.11 Plug in a second monitor

1. Plug it in. dwl places it beside the first, and the bar appears on it.
2. It starts at its preferred mode; for the best refresh rate up to `MONITOR_HZ`, run `kawa-display` (it runs by itself at the next login).
3. Run `kawa-wallpaper --restore`, so the wallpaper is drawn at the new screen's exact size.
4. **Super+,** and **Super+.** move the focus between screens; **Super+Shift+,** and **Super+Shift+.** send a window across.
5. Brightness keys reach the new screen after the next login, or at once after `rm "$XDG_RUNTIME_DIR/kawa-ddc-buses"` (the list of monitors is kept for the session).
6. To put it on the other side: `wlr-randr --output NAME --pos X,Y` (names from `wlr-randr`). This lasts until you log out.

If the new screen is now the one the machine starts on, re-run Theme so the boot menu matches its resolution (§4.7).

13.12 Watch or keep a video from the web

1. Copy the link in Brave.
2. In a terminal: `mpv 'LINK'` to watch it in mpv, or `yt-dlp 'LINK'` to download it into the current folder.
3. Keep the quotes: links often contain `&`, which the shell would otherwise act on.

13.13 Change a setting for good

1. Is the file managed? `fedokawai drift` lists drifted files, and the table at the start of chapter 11 shows which are.
2. Managed: edit the copy under `configs/home/` in the toolkit (§2), then run `fedokawai theme`. It reloads what it can; foot fonts and dwl keys apply at the next login.
3. Yours: edit it where it is. Nothing will overwrite it.
4. Keybindings live in `src/dwl/config.h`; add new ones to `configs/home/.config/fedokawai/keybinds.txt` too, so Super+/ stays complete.

14. Do it yourself: what websites charge for

Joining two PDFs, shrinking a video for WhatsApp, turning an iPhone photo into a JPEG: the usual answer is a website that wants the file uploaded, stops at a size or a page count, stamps a watermark on the result or asks for an account, and keeps a copy on a server you know nothing about. Every recipe in this chapter does the same job on this computer, with programs setup has already installed (`lib/manifest.sh`). Nothing leaves the machine; there are no size limits, no daily quotas, no watermarks and no sign-ups; a contract, a passport scan or a bank statement stays yours. Most recipes are one command and finish in a second.

14.1 Before you start

- Open a terminal (**Super+Return**) and go to the folder with the files: `cd ~/Documents`, or, once you have been there, a word of its name (`cd docs`, §11.7). In yazi, **c d** copies the folder's path; paste it after `cd` with Ctrl+Shift+V.
- Names with spaces go in quotes, as in the examples: `"Tenancy contract.pdf"`. Tab completes a name and adds the quoting itself.
- Every recipe writes a **new** file and leaves the original alone, unless it says otherwise. Open the result first; only then send the original to the trash (§14.10).
- A password typed into a command is kept in the shell history unless the line starts with a **space** (§11.2). The code blocks below that hold a password start with one.

Tip: `tldr qpdf`, `tldr magick` and `tldr ffmpeg` show a page of further examples for each program (§11.7).

14.2 PDFs: join, split, pick and turn pages

qpdf moves whole pages around without touching what is on them, so nothing is lost, and it is instant even on a book.

Want	Command
join files, in this order	`qpdf --empty --pages cover.pdf "Annual report.pdf" annex.pdf -- complete.pdf`
join every PDF in a folder, by name	`qpdf --empty --pages scans/*.pdf -- scans.pdf`
pages from several files	`qpdf --empty --pages report.pdf 1-3 annex.pdf 2 -- summary.pdf`
pages 3 to 7	`qpdf report.pdf --pages . 3-7 -- part.pdf`
everything but pages 2 and 5	`qpdf report.pdf --pages . 1-z,x2,x5 -- trimmed.pdf`
a new order · back to front	`qpdf report.pdf --pages . 3,1,2,4-z -- reordered.pdf` · `--pages . z-1`
odd pages · even pages	`--pages . 1-z:odd` · `--pages . 1-z:even`
one file per page	`qpdf --split-pages report.pdf page.pdf` → `page-1.pdf`, `page-2.pdf` …
files of ten pages each	`qpdf --split-pages=10 book.pdf part.pdf` → `part-01-10.pdf`, `part-11-20.pdf` …
turn every page a quarter right	`qpdf scan.pdf --rotate=+90 scan-turned.pdf`
turn some pages (`-90` turns left)	`qpdf scan.pdf --rotate=180:2,4 scan-fixed.pdf`
turn page 1 in the file itself	`qpdf --replace-input --rotate=+90:1 scan.pdf`
how many pages	`qpdf --show-npages report.pdf`

How a page list reads: `.` stands for the file named before `--pages`; `z` is the last page; `3-7` is a range; `x5` takes page 5 out of what comes before it; `:odd` and `:even` count positions in the list, not page numbers. A file given without a list comes in whole. Turning a page only changes a flag, so it costs nothing either.

Pitfalls. A wildcard sorts by name, so `scan-10.pdf` comes before `scan-2.pdf`: name files `scan-02.pdf`, or list them in the order you want. A wildcard also matches the file you made last time — keep the output outside that folder, as above. `--replace-input` is the one recipe here that changes the original.

14.3 PDFs and pictures

Pictures out of a PDF. `pdfimages -list brochure.pdf` shows what is inside. `-all` writes each picture in the format it is stored in, so a JPEG comes out byte for byte as it went in, at its full size — which no screenshot gives you:

```
mkdir brochure-images
pdfimages -all brochure.pdf brochure-images/img          # img-000.jpg, img-001.png …
pdfimages -all -f 3 -l 3 brochure.pdf brochure-images/p3  # page 3 only
```

Black-and-white scans can come out in fax formats (`.ccitt` , `.jb2e`) that few programs open; `-png` in place of `-all` turns everything into PNG.

Pages into pictures. `pdftoppm` draws each page; `-r` is the resolution — 150 for a screen or a message, 300 for printing or OCR.

Want	Command
every page as PNG	`pdftoppm -r 150 -png report.pdf page` → `page-1.png` … (`page-01.png` from ten pages up)
JPEG, smaller files	`pdftoppm -r 150 -jpeg -jpegopt quality=85 report.pdf page`
only page 2, one file	`pdftoppm -r 300 -png -f 2 -l 2 -singlefile report.pdf page2` → `page2.png`
a thumbnail 300 px tall	`pdftoppm -png -scale-to 300 -f 1 -singlefile report.pdf cover`

Pictures into a PDF. ImageMagick puts one picture on each page. `-density` sets the page size, because pixels ÷ density = inches. Left out, the picture's own value is used — 72 on a phone, which turns a 4032-pixel photo into a page 1.4 metres wide. For an A4-wide page, use the picture's width ÷ 8.3 (a phone photo: 480; a 1240-pixel scan: 150).

```
magick receipt-1.jpg receipt-2.jpg receipt-3.jpg -units PixelsPerInch -density 480 receipts.pdf
magick id-front.jpg id-back.jpg -append +repage -units PixelsPerInch -density 300 "ID card.pdf"
```

The second puts both sides of a card on one page, front above back; without `+repage` the page keeps the size of the first picture. PNGs go in without loss. JPEGs are decoded and saved again at their own quality — hard to see, but a second generation. To put JPEGs in exactly as they are, let tesseract build the PDF: it copies JPEG data untouched, and makes the text searchable as a bonus (it reads every page, so allow a few seconds each):

```
ls receipt-*.jpg > list.txt
tesseract list.txt receipts --dpi 480 -l eng+ara pdf      # writes receipts.pdf
```

14.4 PDFs: smaller, locked, unlocked

Shrink. Ghostscript writes the file anew and scales down every picture above the target resolution, saving it as JPEG. Text and drawings stay sharp, because they are not pictures.

```
gs -q -sDEVICE=pdfwrite -dPDFSETTINGS=/ebook -o "Scanned bill-small.pdf" "Scanned bill.pdf"
```

Setting	Pictures scaled to	For
`/screen`	72 dpi	the smallest file; small print in scans becomes hard to read
`/ebook`	150 dpi	email and upload forms — the usual choice
`/printer` · `/prepress`	300 dpi	printing; little smaller than before

What is lost is detail in the pictures, and only there; pictures already near the target are left as they are. In a test, two pages of 300 dpi photographs (7.8 MB) came out at 390 KB with `/ebook` and 115 KB with `/screen` ; a PDF that is mostly text hardly changes. For a middle way, add `-dColorImageResolution=200 -dGrayImageResolution=200` after `/ebook` . Resolution is counted against the page: a phone photo on a page 1.4 metres wide (§14.3) is only 72 dpi there, so no setting touches it — make such PDFs at A4 width first.

Smaller without any loss: `qpdf --recompress-flate --compression-level=9 --object-streams=generate report.pdf report-small.pdf` packs the file's structure tighter without touching a picture or a letter: anything from a few per cent to a third on text-heavy files, nothing on scans.

Lock with a password.

```
qpdf --encrypt --user-password='open sesame' --owner-password='something else' --bits=256 -- \
     contract.pdf contract-locked.pdf
```

The user password opens the file. The owner password governs printing, copying and changes; qpdf insists on one, and it should differ from the user password — if they are the same, whoever can open the file can also lift its limits. AES-256 opens in every current PDF reader, phones included. To let anyone open a file but ask readers not to print or copy it, give only the owner password and add `--print=none --extract=n` before the `--`; not every reader obeys.

Remove a password you know.

```
qpdf --password='open sesame' --decrypt contract-locked.pdf contract.pdf
```

A file that opens without a password but refuses printing or copying needs no `--password`: `qpdf --decrypt` alone lifts those limits.

Look before you open, clean before you send. `pdfinfo invoice.pdf` shows the page count and size, the program that made it, and whether it is encrypted, holds a form or carries JavaScript — which an invoice or a CV has no business carrying. `qpdf --list-attachments invoice.pdf` lists files tucked inside it, and `qpdf --show-attachment=NAME invoice.pdf > NAME` takes one out. Before passing a file on, `qpdf --remove-info --remove-metadata draft.pdf final.pdf` drops the author, title, creating program and creation date recorded in it (the date of the last change stays).

Pitfall. A PDF with a certificate-based digital signature — the kind a reader can verify — stops validating once any program here rewrites it. Keep and send it as you received it.

14.5 PDFs: text, scans and OCR

Text out. `pdftotext -layout statement.pdf` writes `statement.txt`, columns where they were; `-f 2 -l 3` limits it to pages 2–3, and `-` in place of a file name prints it (`pdftotext -layout statement.pdf - | less`). If nothing comes out, the PDF is a scan — pictures of pages — and needs OCR. There is no faithful PDF-to-Word converter here: the text is what you get, ready to paste into AbiWord.

Make a scan searchable. Tesseract reads the pages and writes an invisible text layer; qpdf lays it over the original, whose pages stay exactly as they were:

```
mkdir ocr
pdftoppm -r 300 -png "Tenancy contract.pdf" ocr/page
ls ocr/page-*.png > ocr/list.txt
tesseract ocr/list.txt ocr/text --dpi 300 -l eng+ara -c textonly_pdf=1 pdf
qpdf "Tenancy contract.pdf" --overlay ocr/text.pdf -- "Tenancy contract (searchable).pdf"
```

Afterwards zathura's `/` finds words in it and text can be selected and copied; the file grows only a little. Then `gio trash ocr`. `-l eng+ara` reads English and Arabic; name only the languages on the page, since each one adds time. Turn sideways pages upright first (§14.2): OCR reads upright text best.

A photo or a picture.

Want	Command
its text, printed in the terminal	`tesseract receipt.jpg - -l eng+ara`
its text, into `receipt.txt`	`tesseract receipt.jpg receipt -l eng+ara`
a searchable PDF, `receipt.pdf`	`tesseract receipt.jpg receipt -l eng+ara pdf`

Tip: to read the text of anything on screen, press **Print** and drag around it — the shot is copied (§4.3) — then `wl-paste -t image/png | tesseract - - -l eng` prints its text in the terminal, where selecting it copies it.

14.6 Fill in and sign a PDF

Xournal++ (§11.18) writes on top of a PDF and exports a new one — the job of the "sign a PDF" websites.

1. **Once: turn your signature into a picture.** Sign plain white paper with a dark pen, photograph it close and square-on, and cut it out with a transparent background:

```
sh magick signature-photo.jpg -colorspace gray -level 25%,75% -fuzz 30% -transparent white \ -trim +repage signature.png
```

If grey specks of paper remain, raise `-fuzz` to 40%; if the strokes break up, lower it. Keep `signature.png` somewhere private, such as `~/Documents`. 2. Start Xournal++ and choose **File → Annotate PDF**. 3. **Fill in:** the text tool (Ctrl+Shift+T); click where an answer goes and type. The toolbar changes the font and colour. The text sits on top of the page, not inside the form's own fields. 4. **Sign:** the image tool (Ctrl+Shift+I); click where the signature goes and choose `signature.png`. Drag its corners to size it; to move it later, select it with Select Rectangle (Ctrl+Shift+R). 5. **File → Export as PDF**, under a new name such as `contract-signed.pdf`: that is the file to send. Ctrl+S keeps your marks in a separate `.xopp` file, so you can change them and export again — from a terminal too: `xournalpp -p contract-signed.pdf contract.xopp`.

Pitfall. A picture of your signature is what most forms ask for, but it is not a certificate-based digital signature, which a reader can verify and which a government or bank signing service adds.

14.7 Pictures

ImageMagick's `magick` does what the picture websites do. The pattern is always `magick input [what to do] output`, and the output's extension chooses the format.

Want	Command
iPhone photo (HEIC) → JPEG	`magick IMG_2041.HEIC IMG_2041.jpg`
WebP → PNG or JPEG	`magick picture.webp picture.png`
PNG → WebP, without loss	`magick chart.png -define webp:lossless=true chart.webp`
JPEG → WebP	`magick photo.jpg -quality 80 photo.webp`
half the size	`magick photo.jpg -resize 50% photo-half.jpg`
fit inside 1920 × 1920, proportions kept	`magick photo.jpg -resize 1920x1920 photo-1920.jpg` (`'1920x1920>'` only ever shrinks)
a smaller JPEG file	`magick photo.jpg -quality 75 photo-q75.jpg`
under a size limit, for upload forms	`magick photo.jpg -resize 600x600 -define jpeg:extent=200kb photo-200k.jpg`
crop a box: width × height + left + top	`magick shot.png -crop 800x600+100+50 +repage part.png`
a square from the centre	`magick photo.jpg -gravity center -crop 1:1 +repage square.jpg`
a border	`magick chart.png -bordercolor white -border 40 framed.png`
side by side · one above the other	`magick a.jpg b.jpg +append +repage both.jpg` · `-append` in place of `+append`
a "copy for …" mark across an ID scan	`magick id-front.jpg -gravity center -pointsize 48 -fill 'rgba(200,0,0,0.45)' -annotate -25x-25+0+0 'COPY FOR VISA APPLICATION ONLY' id-front-marked.jpg`
an animated GIF	`magick -delay 50 -loop 0 frame-*.png slideshow.gif` (delay in hundredths of a second)

JPEG quality 90 looks like the original; 75 is about half the file; below 60, blocks start to show. `jpeg:extent` tries qualities until the file fits.

What a photo gives away. A phone photo records where it was taken (GPS, to a few metres), when, and on which phone, and the file carries all of it wherever it goes. `magick identify -format '%[EXIF:GPS*]' photo.jpg` shows the position; no output means there is none. To send a photo without any of it:

```
magick photo.jpg -auto-orient -strip photo-clean.jpg
```

`-strip` removes everything that is not the picture. `-auto-orient` comes first because "which way up" is stored the same way: stripped alone, a portrait photo turns on its side.

Many at once. Write the results into a new folder, so the originals stay and a second run does not pick up the first one's output:

```
mkdir -p small
for f in *.jpg; do magick "$f" -auto-orient -resize '1600x1600>' -quality 82 -strip "small/$f"; done
fd -d 1 -e heic -x magick {} {.}.jpg      # every HEIC here → a JPEG beside it
```

fd matches `.HEIC` and `.heic` alike and converts several files at once; `-d 1` keeps it to this folder (without it, subfolders too). Avoid `magick mogrify` unless you add `-path other-folder`: on its own it overwrites the originals.

Tip: `magick identify photo.jpg` prints the format, the size in pixels and the file size — check before and after.

QR codes.

Want	Command
a link as a QR code, in the terminal	`qrencode -t utf8 'https://example.org/menu'` (`-t utf8i` on the light theme)
the same as a picture	`qrencode -s 10 -o menu-qr.png 'https://example.org/menu'`
your Wi-Fi: guests join by scanning	`qrencode -s 10 -o wifi.png 'WIFI:S:Home Net;T:WPA;P:correct horse 42;;'`

`-s` is the size of one square in pixels. In the Wi-Fi text, `S:` is the network name, `T:` the security (`WPA` for a network with a password, `nopass` for an open one) and `P:` the password; a `;`, `,`, `:`, `\` or `"` inside a name or password needs a `\` in front. Start that line with a space, since it holds a password (§14.1). In the terminal, `utf8` draws the light squares in the text colour, which is light only on the dark theme; `utf8i` swaps them for the light theme.

14.8 Video and sound

ffmpeg — the full RPM Fusion build — converts nearly anything to anything. `ffprobe -hide_banner clip.mkv` first shows what a file holds; its video and audio codecs decide which recipe is needed.

An MP4 that plays everywhere (H.264 video, AAC sound) — from an iPhone's HEVC `.MOV`, a `.webm` or a `.mkv`:

```
ffmpeg -i IMG_4471.MOV -c:v libx264 -crf 23 -pix_fmt yuv420p -c:a aac -b:a 160k \
  -movflags +faststart IMG_4471.mp4
```

If ffprobe already shows `h264` and `aac`, only the container changes — instant, and nothing is lost: `ffmpeg -i "Phone video.mkv" -c copy "Phone video.mp4"`.

Shrink for WhatsApp or email.

```
ffmpeg -i "Phone video.mp4" \
  -vf scale=1280:1280:force_original_aspect_ratio=decrease:force_divisible_by=2 \
  -c:v libx264 -crf 28 -preset slow -pix_fmt yuv420p -c:a aac -b:a 96k \
  -movflags +faststart "Phone video-small.mp4"
```

The `scale` part fits the picture within 1280 pixels on its long side, upright or sideways. `-crf` sets the quality: 18 looks like the original, 23 is ffmpeg's default, 28 is small, 32 smaller still — each 6 more roughly halves the file. `-preset slow` trades time for a

smaller file; `-pix_fmt yuv420p` is the colour format every phone plays; `+faststart` lets it start playing before it has fully arrived.

Want	Command
cut 1:30–2:10, no re-encoding	`ffmpeg -ss 00:01:30 -to 00:02:10 -i talk.mp4 -c copy talk-cut.mp4`
the same, cut to the frame	`ffmpeg -ss 00:01:30 -to 00:02:10 -i talk.mp4 -c:v libx264 -crf 18 -c:a aac talk-cut.mp4`
the sound alone, untouched	`ffmpeg -i talk.mp4 -vn -c:a copy talk.m4a` (`.opus` when ffprobe says `opus`)
the sound as MP3	`ffmpeg -i talk.mp4 -vn -c:a libmp3lame -q:a 2 talk.mp3`
a WhatsApp voice note as MP3	`ffmpeg -i PTT-20261015-WA0003.opus -c:a libmp3lame -q:a 4 voice.mp3`
without sound	`ffmpeg -i clip.mp4 -c copy -an clip-silent.mp4`
one frame as a picture	`ffmpeg -ss 00:00:12 -i clip.mp4 -frames:v 1 still.png`

The keyframe catch. A copied cut can only begin on a keyframe — a complete picture, which videos carry every few seconds. ffmpeg starts at the one before your time and tells the player to skip ahead; most players start cleanly, some show a still picture for a moment. When the cut must land on the exact frame, use the second row, which re-encodes.

A GIF from a clip — four seconds from 0:05, 12 frames a second, 480 pixels wide:

```
ffmpeg -ss 5 -t 4 -i clip.mp4 \
  -vf "fps=12,scale=480:-1:flags=lanczos,split[a][b];[a]palettegen[p];[b][p]paletteuse" \
  -loop 0 clip.gif
```

`palettegen` picks the GIF's 256 colours from the clip itself, so it does not come out grainy. GIFs are large: keep them short and narrow.

Join clips without re-encoding:

```
printf "file '%s'\n" "part 1.mp4" "part 2.mp4" "part 3.mp4" > list.txt
ffmpeg -f concat -safe 0 -i list.txt -c copy joined.mp4
```

The clips must share codec, picture size and frame rate, as clips from one phone or camera do; convert mixed clips with the MP4 recipe at one size first. `-safe 0` allows spaces and full paths in the list; rename files whose names contain an apostrophe.

14.9 Download and record

Video and sound from the web, with yt-dlp (§11.10) — only what you have the right to keep:

Want	Command
the best version, into this folder	`yt-dlp 'LINK'`
into `~/Videos`	`yt-dlp -P ~/Videos 'LINK'`
no larger than 1080p	`yt-dlp -S res:1080 'LINK'`
only the sound, as MP3	`yt-dlp -x --audio-format mp3 'LINK'`
only 1:30–2:10 of it	`yt-dlp --download-sections '*1:30-2:10' 'LINK'`
one video from a playlist link	`yt-dlp --no-playlist 'LINK'`
which formats exist	`yt-dlp -F 'LINK'`

Keep the link in single quotes (§13.12). When YouTube changes its site, downloads fail until a fixed yt-dlp reaches Fedora; `sudo dnf upgrade --refresh yt-dlp` fetches exactly that, without a full `update`.

Record the screen with wf-recorder. **Ctrl+C** in its terminal stops it and finishes the file; keep recordings in `~/Videos`:

```
wf-recorder -f ~/Videos/screen-$(date +%F-%H%M).mp4                    # the whole screen
wf-recorder -g "$(slurp)" -f ~/Videos/clip.mp4                         # drag a region first
wf-recorder -g "$(slurp)" -a -f ~/Videos/howto.mp4                     # … with your microphone
wf-recorder -g "$(slurp)" --audio=@DEFAULT_MONITOR@ -f ~/Videos/demo.mp4 # … with what you hear
```

With two monitors and no region, it asks which one to record; `-g "$(slurp -o)"` lets you click one instead. For a file meant for a phone or a website, add `-x yuv420p`. A recording of the whole screen includes the terminal you started it from: start it, switch workspace (Super+2) and work there. An existing file is not overwritten without asking, which the date in the first name avoids.

Tip: for a short animation of something on screen, record an MP4 and make the GIF from it (§14.8) — smaller and smoother than recording a GIF directly.

14.10 Files: archives, checksums, trash, renaming, text

Archives, with 7-Zip (`7z`):

Want	Command
zip a folder (opens anywhere)	`7z a "Tax 2026.zip" "Tax 2026"`
7z with a password, names hidden too	`7z a "Tax 2026.7z" "Tax 2026" -p -mhe=on`
zip with a password (AES-256)	`7z a -tzip "Tax 2026.zip" "Tax 2026" -p -mem=AES256`
look inside · test it	`7z l archive.7z` · `7z t archive.7z`
unpack here · into a folder	`7z x archive.zip` · `7z x archive.zip -o"Tax 2026"` (no space after `-o`)
a `.tar.gz` or `.tar.xz`	`tar xf archive.tar.gz` (7z would only peel off the outer layer)
a `.rar` · look inside it	`bsdtar -xf archive.rar` · `bsdtar -tf archive.rar`

`-p` with nothing after it asks for the password, so it never reaches the history. A password-protected zip shows its file names to anyone, and its AES encryption may need 7-Zip on the other side; a `.7z` with `-mhe=on` hides the names as well. 7z also opens ISO images and most other formats, but not RAR: Fedora's 7-Zip leaves RAR out for licence reasons, so RAR files open with `bsdtar` (libarchive, which the books window and zathura use for comics; it unpacks RAR, though it cannot make one). In yazi, `;` then `7z x %h` unpacks the file under the cursor (§11.4).

Checksums. To check a download against the checksum its website publishes:

```
sha256sum Fedora-Server-dvd-x86_64-44.iso                              # prints it
echo 'PASTED-CHECKSUM  Fedora-Server-dvd-x86_64-44.iso' | sha256sum -c  # OK or FAILED
sha256sum -c --ignore-missing Fedora-Server-44-x86_64-CHECKSUM         # a checksum file
```

Two spaces go between the checksum and the name. A signed checksum file adds a warning about "improperly formatted lines" — its signature — which is harmless; the `OK` is what counts.

Delete safely. `gio trash old-draft.docx` (or **d** in yazi) moves a file to the trash instead of deleting it; **g t** in yazi shows the trash, restores from it and empties it. Only emptying gives the space back (§11.4).

What fills the disk. `fd -H -S +500m . ~ -X du -h | sort -h` lists every file over 500 MB in your home, the largest last; `du -xh -d1 ~ | sort -h | tail` shows the largest folders (§13.10).

Rename many files. In yazi, select them (Space, or v for a range) and press **r**: neovim opens with one name per line. Edit them as text — `:%s/IMG_/Holiday-/` changes every line, Ctrl+V edits a column — then `:wq`, and yazi asks before renaming. Keep the order and the number of lines.

Text files.

Want	Command
Arabic subtitles show as gibberish	`iconv -f WINDOWS-1256 -t UTF-8 film.ar.srt > film.ar.utf8.srt`

Want	Command
... or just play them right	`mpv --sub-codepage=cp1256 film.mkv`
which encoding is it	`file notes.txt`
Windows line endings (`^M`) out, in place	`sed -i 's/\r$//' notes.txt`
compare two texts side by side	`nvim -d old.txt new.txt`
compare two PDFs	`pdftotext -layout old.pdf; pdftotext -layout new.pdf; nvim -d old.txt new.txt`

`file` reports old Arabic files as "ISO-8859" or "Non-ISO extended-ASCII"; if the letters are garbled, Windows-1256 is almost always the answer.

14.11 Office documents and spreadsheets

AbiWord and Gnumeric (§11.18) convert without opening a window. AbiWord writes its result next to the original; `ssconvert` writes the name you give.

Want	Command
Word → PDF	`abiword --to=pdf "Offer letter.docx"` → `Offer letter.pdf`
Word → OpenDocument · and back	`abiword --to=odt "Offer letter.docx"` · `abiword --to=docx "Offer letter.odt"`
Word → plain text	`abiword --to=txt "Offer letter.docx"`
every Word file here → PDF	`for f in *.docx; do abiword --to=pdf "$f"; done`
Excel → CSV (the first sheet)	`ssconvert "Expenses October.xlsx" "Expenses October.csv"`
every sheet to its own CSV	`ssconvert -S budget.xlsx 'budget-%s.csv'` (`%s` is the sheet's name)
CSV → Excel	`ssconvert "Expenses October.csv" "Expenses October.xlsx"`
Excel → PDF · → OpenDocument	`ssconvert budget.xlsx budget.pdf` · `ssconvert budget.xlsx budget.ods`

Pitfalls. Ordinary letters and sheets convert well; tracked changes, text boxes, macros and elaborate layouts do not survive. Gnumeric writes dates into a CSV the way it displays them, so `2026-10-01` can come back as `2026/10/01`: check a CSV before you feed it to something strict.

14.12 Quick conversions: units, money, time

qalc (§11.17) answers in one line; `-t` prints the result alone.

Want	Command
units	`qalc '75 km/h to mph'` · `qalc '5 ft 11 in to cm'` · `qalc '30 °C to °F'` · `qalc '1500 sqft to m^2'`
per cent	`qalc '4500 * 20%'` · `qalc '4500 + 5%'` (adds 5 %)
money, at today's rate	`qalc -e '100 USD to AED'`
days until a date	`qalc 'days(today; 2026-12-25)'`
data sizes	`qalc '3 GiB to MB'`

`-e` (long form `-exrates`) fetches current exchange rates first; without it, qalc uses the rates it fetched last, and offline it says so and carries on with those. Write "20 % of" as a multiplication: qalc misreads `of`.

Time zones. `date` knows every zone and its summer time:

```
TZ=Asia/Tokyo date                                          # the time in Tokyo now
date -d 'TZ="Europe/London" 2026-10-20 15:00'               # 15:00 in London, in your time
TZ=Europe/London date -d 'TZ="Asia/Dubai" 2026-10-20 15:00' # 15:00 in Dubai, in London's time
```

Zone names come from `timedatectl list-timezones | rg -i york`.

Tip: the password-generator websites, offline: `tr -dc 'A-Za-z0-9'